

By:
Adarsh Kumar Singh -180
Syed Usman-172

Cohort-2

Hospital Management System



Table of Contents:

1. Introduction

- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Project Objectives
- 1.4 Project Scope

2. Requirements Analysis

- 2.1 Functional Requirements
 - 2.1.1 User Management
 - 2.1.2 Patient Registration & Appointment Booking
 - 2.1.3 Doctor and Staff Management
 - 2.1.4 Prescription and Treatment Logging
 - 2.1.5 Pharmacy and Billing
 - 2.1.6 Lab Test and Service Management
 - 2.1.7 Admin Controls
- 2.2 Non-Functional Requirements
 - 2.2.1 Security
 - 2.2.2 Performance
 - 2.2.3 Availability
 - 2.2.4 Scalability
 - 2.2.5 Maintainability
 - 2.2.6 Usability
 - 2.2.7 Data Integrity
- 2.3 Technical Requirements

3. System Design

- 3.1 Use Case Diagram
- 3.2 Class Diagram
- 3.4 Sequence Diagrams
- 3.5 Database Design

4.OOP principles

- 4.1 Classes, Objects, Inheritance, Polymorphism
- 4.2 Java GUI: JFrame, Panels, Components
- 4.3 Exception Handling
- 4.4 JDBC Integration
- 4.5 Package Structure
- 4.6 Collections (if used)
- 4.7 Abstraction
- 4.8 Exceptions
- 4.9 Access Specifiers and their availability

7. Code Snippets

7.1 Java Backend Code

7.2 Java GUI Code

7.3 MySQL Table Creation and Queries

6. Testing

6.1 Test Cases & Scenarios

6.1.1 Patient Registration & Booking

6.1.2 Prescription Entry

6.1.3 Medicine Dispensing

6.2 Acceptance Criteria

8. Results & Demonstration

8.1 GUI Screenshots and Workflow

8.2 Database Output & Sample Records

9. Conclusion & Future Enhancements

9.1 Summary

9.2 Challenges Encountered

9.3 Future Scope

10. How AI helped in my project

11. References

Introduction

1.1 Project Overview

The Hospital Management System (HMS) is an integrated software application built using Java Swing for the GUI, JDBC for database connectivity, and MySQL as the backend RDBMS. It aims to digitize and streamline the various operational aspects of a hospital, including patient management, appointment scheduling, treatment recording, pharmacy handling, and administrative monitoring. The system ensures secure multi-user access with role-based authentication for patients, medical staff, receptionists, and administrators, promoting data consistency and efficient hospital operations.

1.2 Problem Statement

Healthcare facilities often suffer from inefficiencies caused by manual systems or fragmented digital tools. Key challenges include:

- **Fragmented Data Storage:** Patient records are scattered across departments (e.g., registration desk, pharmacy, lab), leading to inconsistencies and duplication of data.
- **Manual Errors:** Human errors in record-keeping (e.g., appointment mismatches, misplaced files, incorrect dosage logging) can lead to severe consequences in medical treatment.
- **Delayed Access to Information:** Doctors and staff often waste time searching for patient histories or lab results, leading to longer consultation times and operational delays.
- **No Real-Time Synchronization:** Manual processes cannot support real-time updates, which is critical in emergency and time-sensitive medical situations.
- **Lack of Security & Privacy:** Paper-based or poorly secured systems do not ensure HIPAA-like compliance, leaving sensitive patient information vulnerable to breaches.
- **Poor Scalability:** As patient numbers grow, manual systems or basic spreadsheets cannot scale to meet demand or provide analytics for decision-making.

1.3 Project Objectives

The main goals of this software solution are:

- **Digitization of Hospital Workflow:** Replace manual paper-based processes with a centralized and fully digital system.

- **Role-Based Access Control:** Ensure secure logins and access to information based on the user type—Admin, Doctor, Receptionist, or Patient.
- **Real-Time Information Flow:** Maintain live updates of appointments, prescriptions, medical records, and billing to avoid redundancy and confusion.
- **Error Minimization:** Use structured data input forms, input validation, and exception handling to reduce manual errors in medical and billing operations.
- **Improved Patient Experience:** Allow patients to register, book appointments, and view prescriptions conveniently.
- **Database Integration:** Store and manage patient histories, staff records, and transaction logs in a secure and structured database.
- **Support for Reporting & Analytics:** Generate reports on hospital operations such as total patients, staff activities, prescriptions issued, medicines dispensed, etc.

1.4 Project Scope

This project covers a wide spectrum of hospital operations in a modular and extensible architecture:

- **Patient Module:** Handles patient onboarding, demographic data, history, appointment scheduling, and prescription retrieval.
- **Doctor Module:** Enables doctors to view assigned appointments, log treatments, prescribe medicines, and refer lab tests.
- **Receptionist Interface:** Facilitates patient registration, appointment bookings, queue management, and initial consultation setup.
- **Pharmacy Module:** Tracks medicine stock, manages billing, and processes prescriptions issued by doctors.
- **Laboratory Module:** Records test results, links them to patient records, and updates treatment history.
- **Administrative Control:** Central admin authority can manage users (add/remove/edit), generate system-wide reports, and ensure overall monitoring.
- **Data Persistence & Backup:** All records are stored in MySQL tables with normalized schemas, and regular backups are envisioned for data recovery.

- **Usability and Extensibility:** The application is designed with a layered structure to support the addition of new modules like emergency services, room booking, or insurance integration in the future

Requirements:

1. Functional Requirements

These describe what the system should do:

1.1 User Management

- Add, update, or delete doctor, nurse, and admin records.
- Secure login for admin (username/password based).
- Role-based access control (optional: restrict certain features to admin).

1.2 Patient Management

- Add new patient records (name, age, gender, symptoms, etc.).
- Update patient information.
- Discharge a patient and maintain discharge summaries.
- View/search patient records.

1.3 Doctor Management

- Add, update, and delete doctor profiles (name, specialization, availability).
- Assign doctors to patients.
- View list of all doctors.

1.4 Appointment Management

- Schedule patient appointments.
- Cancel or reschedule appointments.
- View today's appointments.

1.5 Billing System

- Generate patient bills.
- Print or export bills (optional).
- Maintain payment records.

1.6 Medical Records

- Store and update patient diagnosis and treatment.
- Attach reports or prescriptions (basic or advanced feature).
- Search medical history by patient ID.

2. Non-Functional Requirements

These define system performance and quality:

2.1 Performance

- Fast GUI response for up to 100 concurrent records.
- Load time for screens: <2 seconds.

2.2 Usability

- User-friendly interface (buttons, menus, and layout clearly visible).
- Input validation (no blank fields, correct formats for age, dates, etc.).

2.3 Security

- Password protection for login.
- Optional encryption of sensitive data (e.g., medical history).

2.4 Maintainability

- Modular code with well-organized classes (e.g., Patient.java, Doctor.java).

- Easy to update or add new features.

2.5 Scalability

- The system should support easy migration to a database backend (e.g., MySQL) in future.

2.6 Portability

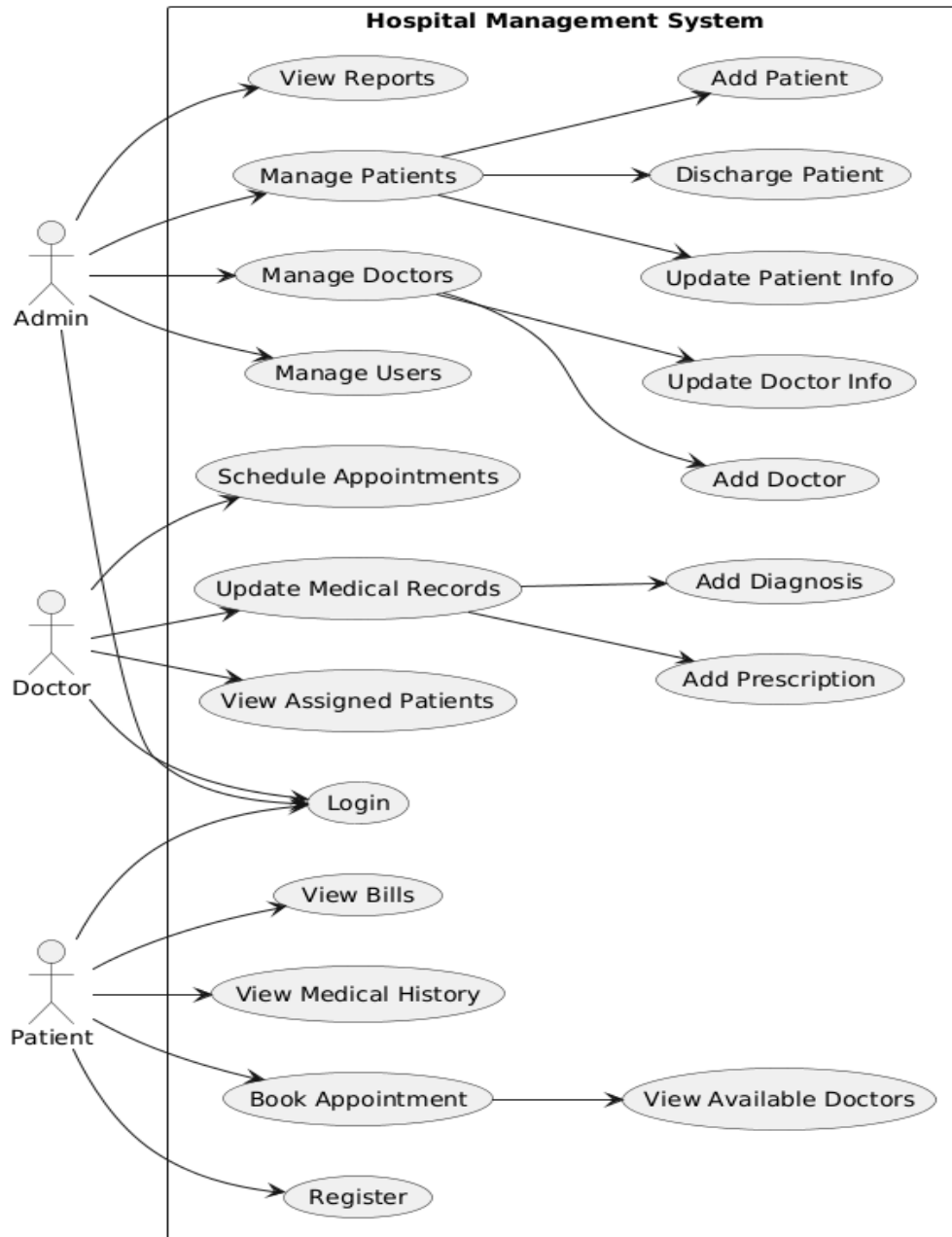
- Runs on any system with Java Runtime Environment.

3. Technical Requirements

- Language: Java
- Framework: Java Swing for GUI
- JDK Version: Java SE 8 or above
- IDE: IntelliJ / Eclipse / NetBeans
- Database (optional): Flat file or future MySQL integration

Design:

1. Use Case:



Explanation:

Actors (Users)

1. Admin
 - Manages the system's core data (users, doctors, patients).
 - Has access to almost all operations.
2. Doctor
 - Focuses on patient care: viewing assigned patients, updating records, prescribing.
3. Patient
 - Interacts with the system for registration, appointments, billing, and viewing records.

Login:

- Shared by all three actors.
- Authenticates users before allowing access to specific features.

Admin Use Cases:

1. **Manage Patients**
 - Enter new patient details.
 - Edit personal/medical info.
 - Mark patients as discharged.
2. **Manage Doctors**
 - Add new doctor profiles.
 - Edit their specialization, contact, etc.
3. **Manage Users**

- Add/edit admin-level or hospital staff users.
- 4. **Schedule Appointments**
 - Create or edit patient-doctor appointment schedules.
- 5. **View Reports**
 - Generate summaries such as patient load, billing, and appointment stats.

Doctor Use Cases:

1. **View Assigned Patients**
 - See a list of patients they are responsible for.
2. **Update Medical Records**
 - Enter illness or condition.
 - List medicines or treatment advice.

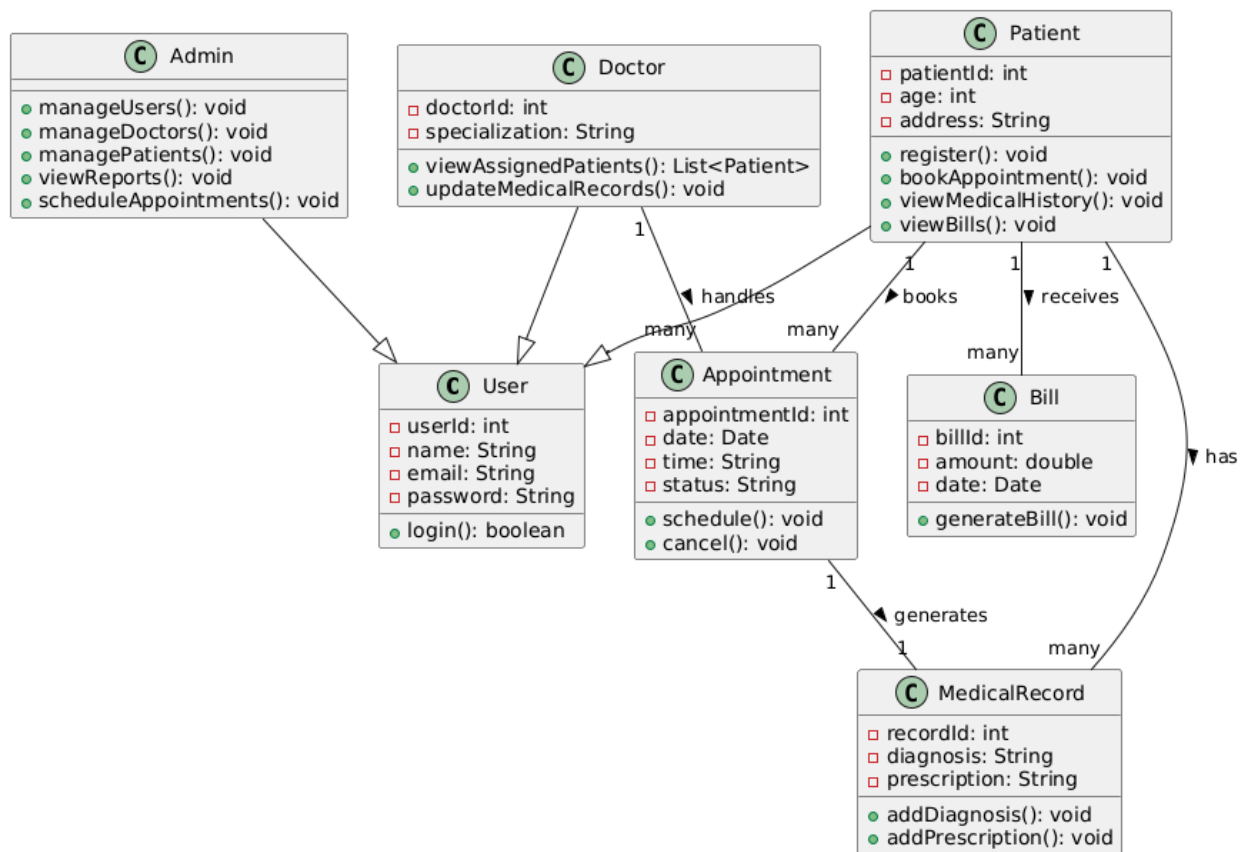
Patient Use Cases:

1. **Register**
 - Self-registration or by admin.
2. **Book Appointment**
 - ➤ : Choose a doctor based on availability.
3. **View Medical History**
 - View past diagnoses, prescriptions, visits.
4. **View Bills**
 - Access their billing information.

Relationships:

- → arrows represent **"include"** relationships, where a primary use case includes another
- Shared **Login** indicates a common entry point for all roles.
- All use cases are organized within the system boundary box labeled **"Hospital Management System"**.

2.Class Diagram:



Explanation:

Key Classes and Their Roles

User (Base Class):

- **Attributes:** userId, name, email, password
- **Methods:** login()
- **Purpose:** Acts as a **parent class** for all types of users—Admin, Doctor, and Patient.

Admin (Inherits User)

- **Methods:**
 - manageUsers()
 - manageDoctors()
 - managePatients()
 - viewReports()
 - scheduleAppointments()
- **Purpose:** Performs high-level administrative operations.

Doctor (Inherits User)

- **Attributes:** doctorId, specialization
- **Methods:**
 - viewAssignedPatients()
 - updateMedicalRecords()
- **Purpose:** Responsible for diagnosing and treating patients.

Patient (Inherits User)

- **Attributes:** patientId, age, address
- **Methods:**
 - register()
 - bookAppointment()
 - viewMedicalHistory()
 - viewBills()
- **Purpose:** Represents individuals receiving treatment.

Appointment

- **Attributes:** appointmentId, date, time, status
- **Methods:**
 - schedule()
 - cancel()
- **Relationships:**
 - Patient **books** many Appointments.
 - Doctor **handles** many Appointments.
- **Purpose:** Captures the scheduling details between doctors and patients.

MedicalRecord

- **Attributes:** recordId, diagnosis, prescription
- **Methods:**
 - addDiagnosis()

- addPrescription()
- **Relationships:**
 - Created from Appointment
 - Linked to Patient (many records per patient)
- **Purpose:** Stores diagnosis and treatment details.

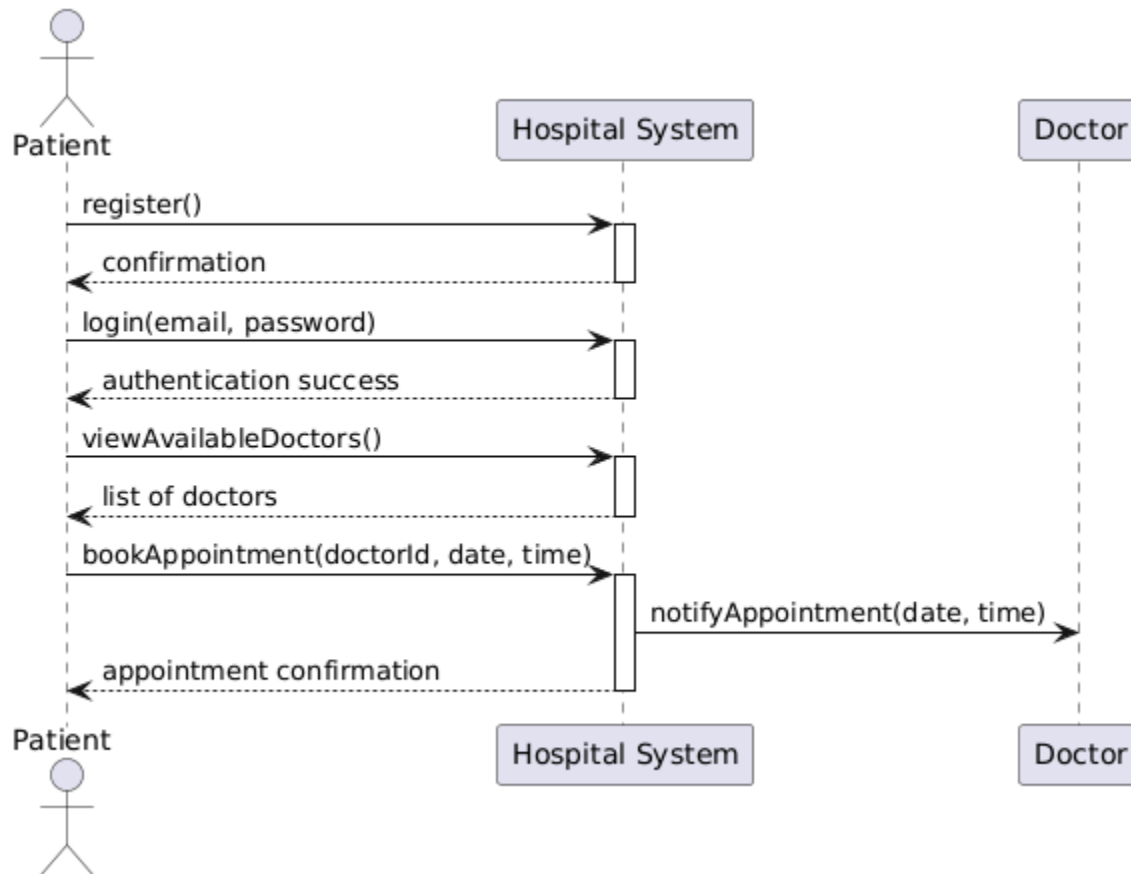
Bill:

- **Attributes:** billId, amount, date
- **Method:** generateBill()
- **Relationships:**
 - Each Patient **receives** many Bills.
- **Purpose:** Handles payment and billing information.

Relationships Summary

- **Inheritance:**
 - Admin, Doctor, Patient inherit from User.
- **Associations:**
 - Patient ↔ Appointment: 1-to-many (books)
 - Doctor ↔ Appointment: 1-to-many (handles)
 - Appointment → MedicalRecord: 1-to-1 (generates)
 - Patient ↔ MedicalRecord: 1-to-many (has)
 - Patient ↔ Bill: 1-to-many (receives)

Sequence Diagram:



Explanation:

This diagram shows the interaction between three key components:

- Patient (the user)
- Hospital System (software/application)
- Doctor (the healthcare provider)

Step-by-Step Explanation:

1. register()

- The Patient sends a request to the Hospital System to register.
- The system processes the registration and sends a confirmation response.

2. login(email, password):

- The Patient logs into the system by sending credentials.
- The Hospital System authenticates the user and sends back authentication success.

3. viewAvailableDoctors():

- The Patient requests to see the list of available doctors.
- The Hospital System fetches and returns the list of doctors.

4. bookAppointment(doctorId, date, time):

- The Patient chooses a doctor and sends a request to book an appointment, providing doctor ID, date, and time.

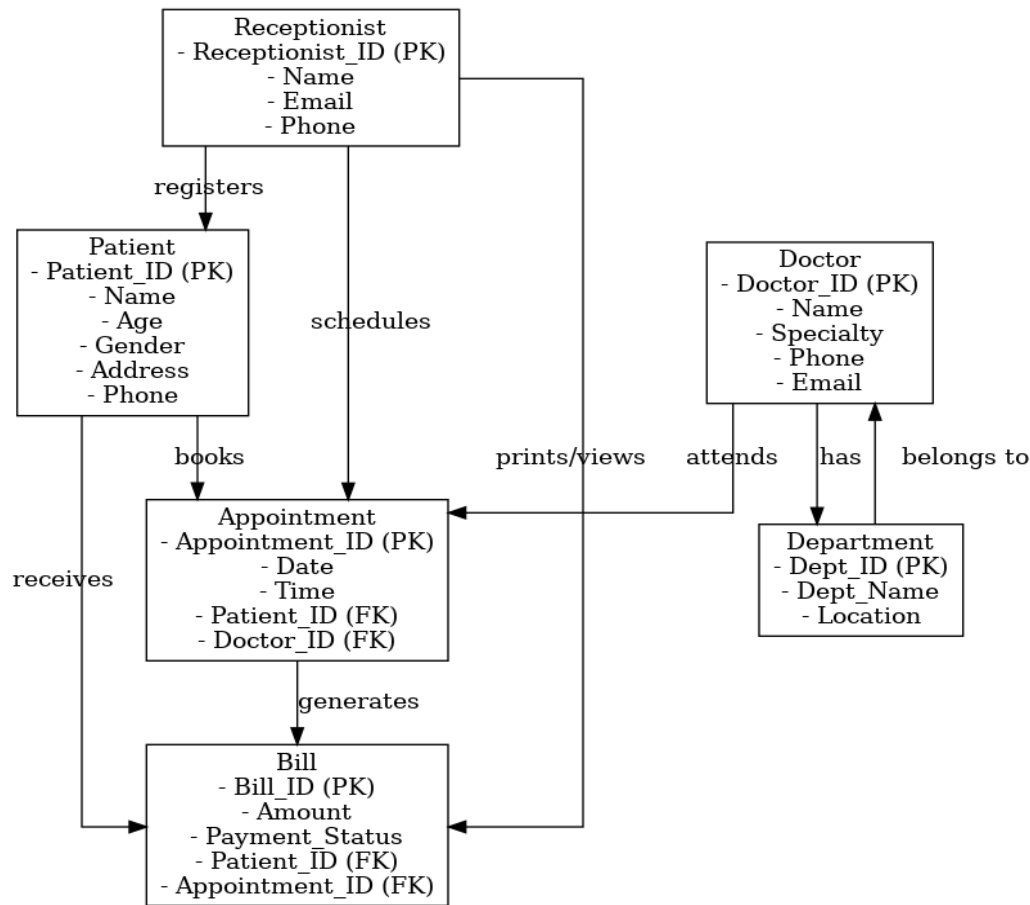
5. notifyAppointment(date, time):

- The Hospital System notifies the selected Doctor about the scheduled appointment.

6. appointment confirmation:

- After notifying the doctor, the system sends a final confirmation back to the patient that the appointment has been successfully booked.

ER-Diagram:



Explanation:

- Receptionist registers Patients and schedules their Appointments with Doctors.
- Patients have details like name, age, gender, etc.
- Appointments link Patients and Doctors, and include date and time.
- Doctors belong to a Department (like Cardiology or Neurology), and have specialties.
- After an appointment, a Bill is generated for the Patient, based on that appointment.
- The Receptionist can also view or print appointment details.
- Bills show payment status and amount.

OOPS Principles:

1.Classes: A class is a blueprint for creating objects. It defines properties (variables) and behaviors (methods).

Example:

```
class Patient {  
  
    String name;  
  
    int age;  
  
  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

2. Objects: An object is an instance of a class. You use it to access the class's variables and methods.

Example:

```
public class Main {  
  
    public static void main(String[] args) {  
        Patient p1 = new Patient(); // Creating an object of Patient  
        p1.name = "John";  
        p1.age = 30;  
        p1.displayInfo(); // Output: Name: John, Age: 30  
    }  
}
```

3. Inheritance: Inheritance allows a class to inherit properties and methods from another class.

Example:

```
class Doctor extends Person {  
    String specialization;  
  
    void showDetails() {  
        System.out.println("Name: " + name + ", Specialization: " + specialization);  
    }  
}
```

```
class Person {  
    String name;  
}
```

Now, Doctor inherits the name property from Person.

```
public class Main {  
    public static void main(String[] args) {  
        Doctor d1 = new Doctor();  
        d1.name = "Dr. Smith";  
        d1.specialization = "Cardiologist";  
        d1.showDetails(); // Output: Name: Dr. Smith, Specialization: Cardiologist  
    }  
}
```

4. Polymorphism:

Polymorphism means "many forms." It lets you use the same method in different ways.

Example of Method Overriding (Runtime Polymorphism):

```
class Person {  
    void showRole() {  
        System.out.println("Person");  
    }  
}
```

```
class Nurse extends Person {  
    void showRole() {  
        System.out.println("Nurse");  
    }  
}
```

Access Specifiers and Their Availability:

Specifier	Within Same Class	Within Same Package	Subclass in Other Package	Other Classes
private	✓ Yes	✗ No	✗ No	✗ No
default (no modifier)	✓ Yes	✓ Yes	✗ No	✗ No
protected	✓ Yes	✓ Yes	✓ Yes	✗ No
public	✓ Yes	✓ Yes	✓ Yes	✓ Yes

1. private Access Modifier:

- Visible only inside the class
- Used for sensitive data like passwords, internal counters, or helper methods

Example:

```
public class BankAccount {  
  
    private double balance; // can only be accessed within this class  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Use Case: Helps in **data hiding** and controlling access to critical fields.

2.Default (Package-Private)

- When no modifier is specified
- Visible to other classes in the same package
- Good for internal communication between package-level components

Example:

```
class Logger {  
    void log(String message) {  
        System.out.println("Log: " + message);  
    }  
}
```

Use Case: Useful when you want to expose functionality only within a specific module/package.

3. protected Access Modifier:

- Visible within the same package and to subclasses in any package
- Often used in inheritance to allow subclass customization

Example:

```
public class Animal {  
    protected void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
public class Dog extends Animal {  
    public void bark() {  
        eat(); // valid: protected method accessed in subclass  
        System.out.println("Dog is barking");  
    }  
}
```

Use Case:Used when designing frameworks or base classes that allow customization in derived classes.

4. public Access Modifier:

- Accessible from anywhere
- Ideal for APIs, libraries, or services meant for public consumption

Example:

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Use Case:Expose widely-used utilities, constants, or services.

Short Explanation:

Modifier	Analogy
private	A Personal diary: only you can read it.
default	A household fridge: anyone in your house (package) can use it.
protected	A family recipe: known within family (package) and inherited by children (subclasses).
public	A public library: anyone can access it.

Exceptions:

1. **NullPointerException**

- When you try to access methods or variables of an object that hasn't been initialized.

Example:

Patient p = null;

p.getName(); // NullPointerException

2. **ArrayIndexOutOfBoundsException**

- When you try to access an invalid index of an array.

Example:

Doctor[] doctors = new Doctor[5];

doctors[5] = new Doctor(); // ArrayIndexOutOfBoundsException

3.NumberFormatException

- When you try to convert a string to a number, but the string is not a valid number.

Example:

```
int age = Integer.parseInt("twenty"); // NumberFormatException
```

4.ClassCastException

- When you try to cast an object to a subclass it doesn't actually belong to.

Example:

```
Object obj = new Patient();
```

```
Doctor d = (Doctor) obj; // ClassCastException
```

5.InputMismatchException

- When an incorrect data type is entered while using a Scanner.

Example:

```
Scanner sc = new Scanner(System.in);
```

```
int id = sc.nextInt(); // InputMismatchException if user enters "abc"
```

6.IllegalArgumentException

- When you pass an invalid argument to a method.
- Often custom-thrown

Example:

```
setPatientAge(-5); // IllegalArgumentException
```

8.IOException (input/output)

- If your system reads/writes files, this may occur due to I/O failure.

Example:

```
BufferedReader br = new BufferedReader(new FileReader("file.txt")); // File not found
```

Abstraction – Abstract Class & Interface:

An abstract class:

- Can have both abstract methods (without body) and concrete methods (with body).
- Cannot be instantiated.
- Is used when you want to provide a common base class with shared code but enforce subclasses to define specific behavior.

Example: Abstract Class in Hospital System

```
abstract class Person {  
  
    String name;  
  
    int age;  
  
  
    // Concrete method  
    void displayInfo() {  
        System.out.println("Name: " + name);  
    }  
  
  
    // Abstract method  
    abstract void role(); // must be implemented by subclasses  
}
```

```
class Doctor extends Person {  
    void role() {  
        System.out.println("Role: Treat Patients");  
    }  
}
```

```
class Nurse extends Person {  
    void role() {  
        System.out.println("Role: Assist Doctors");  
    }  
}
```

2. Interface

An interface:

- Can only contain **abstract methods** (Java 7), or abstract + default/static methods (Java 8+).
- Is used to define a **contract** (what a class must do, but not how).
- A class can **implement multiple interfaces**, supporting **multiple inheritance**

Example: Interface in Hospital System

```
interface Schedulable {  
    void scheduleAppointment();  
}
```

```
interface Billable {  
    void generateBill();  
}  
  
class Receptionist implements Schedulable, Billable {  
    public void scheduleAppointment() {  
        System.out.println("Appointment scheduled.");  
    }  
  
    public void generateBill() {  
        System.out.println("Bill generated.");  
    }  
}
```

Code:

//File Name AdminService.java

//Accessing Package comhms.src.hms.admin.controller

```
package comhms.src.hms.admin.controller;
```

```
import comhms.src.hms.admin.model.Admin;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import comhms.src.hms.auth.model.AuditLog;
```

```

import comhms.src.hms.auth.controller.UserLoginService;

import java.util.ArrayList;

import java.util.List;

import java.util.Objects;


public class AdminService {

    private List<Schedule> scheduleDatabase; // Simulated Schedule table

    private UserLoginService loginService; // For user and audit log operations

    private int nextScheduleID;


    // Constructor

    public AdminService(UserLoginService loginService) {

        this.scheduleDatabase = new ArrayList<>();

        this.loginService = loginService;

        this.nextScheduleID = 1;

        initializeDefaultSchedules();

    }


    // Overloaded Constructor

    public AdminService(UserLoginService loginService, List<Schedule> initialSchedules) {

        this.scheduleDatabase = initialSchedules != null ? initialSchedules : new ArrayList<>();

        this.loginService = loginService;

        this.nextScheduleID = initialSchedules != null ? initialSchedules.size() + 1 : 1;

    }

```

```
// Inner Class for Admin Operation Result
```

```
private class OperationResult {
```

```
    private boolean success;
```

```
    private String message;
```

```
    OperationResult(boolean success, String message) {
```

```
        this.success = success;
```

```
        this.message = message;
```

```
    }
```

```
    String getMessage() {
```

```
        return message;
```

```
    }
```

```
}
```

```
// Method with Object Reference
```

```
public OperationResult createSchedule(Admin admin, Schedule schedule) throws  
Admin.AdminAccessException {
```

```
    if (!admin.isSuperAdmin()) {
```

```
        throw new Admin.AdminAccessException("Only super admins can create schedules.");
```

```
    }
```

```
    scheduleDatabase.add(schedule);
```

```
    nextScheduleID++;
```

```
    return new OperationResult(true, "Schedule created: " + schedule);
```

```
}
```

// Method without Object Reference (Static)

```
public static String getServiceDescription() {  
    return "AdminService: Manages schedules, users, and audit logs for the Admin module.";  
}
```

// Overloaded Method: Update Schedule Status

```
public String updateScheduleStatus(Admin admin, int scheduleID, String newStatus)  
    throws Admin.AdminAccessException, Schedule.InvalidScheduleException {  
    if (!admin.isSuperAdmin()) {  
        throw new Admin.AdminAccessException("Only super admins can update schedules.");  
    }  
    Schedule schedule = getScheduleByID(scheduleID);  
    return schedule.updateStatus(newStatus);  
}
```

// Overloaded Method: Update Schedule Status with Reason

```
public String updateScheduleStatus(Admin admin, int scheduleID, String newStatus, String  
reason)  
    throws Admin.AdminAccessException, Schedule.InvalidScheduleException {  
    if (!admin.isSuperAdmin()) {  
        throw new Admin.AdminAccessException("Only super admins can update schedules.");  
    }  
    Schedule schedule = getScheduleByID(scheduleID);  
    return schedule.updateStatus(newStatus, reason);  
}
```


// Method to Manage User (Activate/Deactivate)

```
public String manageUser(Admin admin, String userID, boolean activate)
    throws Admin.AdminAccessException, UserLoginService.InvalidUserIDException {
    UserLogin user = loginService.getUserByID(userID);
    return admin.manageUser(user, activate);
}
```

// Method to Manage User (Update Role)

```
public String manageUser(Admin admin, String userID, String newRole)
    throws Admin.AdminAccessException, UserLoginService.InvalidUserIDException {
    UserLogin user = loginService.getUserByID(userID);
    return admin.manageUser(user, newRole);
}
```

// Method to Get Audit Logs

```
public List<AuditLog> getAuditLogs(Admin admin) throws Admin.AdminAccessException {
    return loginService.getAuditLogs();
}
```

// Method to Get Schedule by ID

```
public Schedule getScheduleByID(int scheduleID) throws Schedule.InvalidScheduleException
{
    for (Schedule schedule : scheduleDatabase) {
        if (schedule.getScheduleID() == scheduleID) {
            return schedule;
        }
    }
}
```

```
    }  
    throw new Schedule.InvalidScheduleException("Schedule ID " + scheduleID + " not  
found.");  
}
```

// Method to Get Next Schedule ID

```
public int getNextScheduleID() {  
    return nextScheduleID;  
}
```

// Method to Add Schedule (For Testing)

```
public void addSchedule(Schedule schedule) {  
    scheduleDatabase.add(schedule);  
    nextScheduleID++;  
}
```

// User Defined Exception

```
public static class AdminServiceException extends Exception {  
    public AdminServiceException(String message) {  
        super(message);  
    }  
}
```

// Private Helper Method

```
private void initializeDefaultSchedules() {  
    try {
```

```

        Schedule schedule = new Schedule(nextScheduleID, 2, "2025-05-04 09:00",
                                          "2025-05-04 10:00", "Available", "Room 101");

        scheduleDatabase.add(schedule);

        nextScheduleID++;
    } catch (Schedule.InvalidScheduleException e) {

        // Ignore for now
    }
}

```

```

@Override

public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof AdminService)) return false;

    AdminService that = (AdminService) o;

    return nextScheduleID == that.nextScheduleID &&
           Objects.equals(scheduleDatabase, that.scheduleDatabase);
}

```

```

@Override

public int hashCode() {
    return Objects.hash(scheduleDatabase, nextScheduleID);
}
}

```

//File Name Admin.java

//Accessing Package comhms.src.hms.admin.model

```
package comhms.src.hms.admin.model;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import comhms.src.hms.auth.model.AuditLog;
```

```
import java.util.List;
```

```
public class Admin extends UserLogin {
```

```
    private String department; // E.g., "Hospital Management"
```

```
    private boolean superAdmin; // Extra privileges
```

```
    // Constructor
```

```
    public Admin(int userID, String username, String password, String name, int age, String department, boolean superAdmin) {
```

```
        super(userID, username, password, "Admin", name, age);
```

```
        this.department = department;
```

```
        this.superAdmin = superAdmin;
```

```
    }
```

```
    // Overloaded Constructor
```

```
    public Admin(int userID, String username, String password, String name, int age, String department, boolean superAdmin,
```

```
        String securityQuestion, String securityAnswer) {
```

```
        super(userID, username, password, "Admin", null, name, age, securityQuestion, securityAnswer);
```

```
this.department = department;  
this.superAdmin = superAdmin;  
}
```

```
// Inner Class for Admin Permissions
```

```
private class Permissions {  
    private boolean canManageUsers;  
    private boolean canViewAuditLogs;  
    private boolean canManageSchedules;
```

```
    Permissions(boolean superAdmin) {  
        this.canManageUsers = true;  
        this.canViewAuditLogs = true;  
        this.canManageSchedules = superAdmin; // Only super admins  
    }
```

```
    String getPermissionSummary() {  
        return "Manage Users: " + canManageUsers + ", View Audit Logs: " +  
canViewAuditLogs +  
        ", Manage Schedules: " + canManageSchedules;  
    }  
}
```

```
// Method with Object Reference
```

```
public String viewAuditLogs(List<AuditLog> logs) throws AdminAccessException {  
    Permissions perms = new Permissions(superAdmin);
```

```

        if (!perms.canViewAuditLogs) {
            throw new AdminAccessException("Insufficient permissions to view audit logs.");
        }

        StringBuilder result = new StringBuilder("Audit Logs:\n");

        for (AuditLog log : logs) {
            result.append(log.toString()).append("\n");
        }

        return result.toString();
    }

// Method without Object Reference (Static)
public static String getAdminRoleDescription() {
    return "Admin: Manages users, schedules, and system reports.";
}

// Overloaded Method: Manage User (Activate)
public String manageUser(UserLogin user, boolean activate) throws AdminAccessException {
    Permissions perms = new Permissions(superAdmin);

    if (!perms.canManageUsers) {
        throw new AdminAccessException("Insufficient permissions to manage users.");
    }

    String status = activate ? "activated" : "deactivated";

    return "User " + user.getUsername() + " has been " + status + ".";
}

```

// Overloaded Method: Manage User (Update Role)

```
public String manageUser(UserLogin user, String newRole) throws AdminAccessException {  
    Permissions perms = new Permissions(superAdmin);  
    if (!perms.canManageUsers) {  
        throw new AdminAccessException("Insufficient permissions to manage users.");  
    }  
    return "User " + user.getUsername() + " role updated to " + newRole + ".";  
}
```

// User Defined Exception

```
public static class AdminAccessException extends Exception {  
    public AdminAccessException(String message) {  
        super(message);  
    }  
}
```

// Getters and Setters

```
public String getDepartment() { return department; }  
public boolean isSuperAdmin() { return superAdmin; }  
public void setDepartment(String department) { this.department = department; }  
public void setSuperAdmin(boolean superAdmin) { this.superAdmin = superAdmin; }
```

// Override toString

@Override

```
public String toString() {
```

```

        Permissions perms = new Permissions(superAdmin);

        return "Admin [Username=" + getUsername() + ", Department=" + department +
            ", SuperAdmin=" + superAdmin + ", Permissions=" + perms.getPermissionSummary()
        + "];"
    }
}

```

//File Name Schedule.java

//Accessing Package comhms.src.hms.admin.model

```

package comhms.src.hms.admin.model;

import comhms.src.hms.auth.model.UserLogin;
import java.sql.Timestamp;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Objects;

public class Schedule {
    private int scheduleID;
    private int staffID; // Links to UserLogin.userID
    private Timestamp startTime;
    private Timestamp endTime;
    private String status; // E.g., "Booked", "Available", "Cancelled"
    private String location; // E.g., "Room 101"
}

```


// Constructor

```
public Schedule(int scheduleID, int staffID, String startTimeStr, String endTimeStr, String
status, String location)
```

```
    throws InvalidScheduleException {
    this.scheduleID = scheduleID;
    this.staffID = staffID;
    this.status = status;
    this.location = location;
    setTime(startTimeStr, endTimeStr);
}
```

// Overloaded Constructor

```
public Schedule(int scheduleID, int staffID, Timestamp startTime, Timestamp endTime, String
status, String location)
```

```
    throws InvalidScheduleException {
    this.scheduleID = scheduleID;
    this.staffID = staffID;
    this.startTime = startTime;
    this.endTime = endTime;
    this.status = status;
    this.location = location;
    validateTime();
}
```

// Inner Class for Schedule Details

```

private class ScheduleDetails {

    private String staffRole;

    private String purpose;

    ScheduleDetails(String staffRole, String purpose) {

        this.staffRole = staffRole;

        this.purpose = purpose;

    }

    String getSummary() {

        return "Role: " + staffRole + ", Purpose: " + purpose;

    }

}

```

// Method with Object Reference

```

public boolean assignStaff(UserLogin staff, String role, String purpose) throws
InvalidScheduleException {

    if (!status.equals("Available")) {

        throw new InvalidScheduleException("Schedule is not available for assignment.");

    }

    if (staff.getUserID() != this.staffID) {

        throw new InvalidScheduleException("Staff ID does not match schedule.");

    }

    ScheduleDetails details = new ScheduleDetails(role, purpose);

    this.status = "Booked";

    System.out.println("Assigned: " + details.getSummary());
}

```

```
    return true;
}
```

```
// Method without Object Reference (Static)
```

```
public static String getValidStatusList() {
    return "Available, Booked, Cancelled";
}
```

```
// Overloaded Method: Update Status
```

```
public String updateStatus(String newStatus) throws InvalidScheduleException {
    if (!isValidStatus(newStatus)) {
        throw new InvalidScheduleException("Invalid status: " + newStatus);
    }
    this.status = newStatus;
    return "Schedule " + scheduleID + " status updated to " + newStatus;
}
```

```
// Overloaded Method: Update Status with Reason
```

```
public String updateStatus(String newStatus, String reason) throws InvalidScheduleException
{
    if (!isValidStatus(newStatus)) {
        throw new InvalidScheduleException("Invalid status: " + newStatus);
    }
    this.status = newStatus;
    return "Schedule " + scheduleID + " status updated to " + newStatus + " (Reason: " + reason
+ ")";
}
```

```
}
```

```
// User Defined Exception
```

```
public static class InvalidScheduleException extends Exception {  
    public InvalidScheduleException(String message) {  
        super(message);  
    }  
}
```

```
// Getters and Setters
```

```
public int getScheduleID() { return scheduleID; }  
public int getStaffID() { return staffID; }  
public Timestamp getStartTime() { return startTime; }  
public Timestamp getEndTime() { return endTime; }  
public String getStatus() { return status; }  
public String getLocation() { return location; }  
  
public void setStaffID(int staffID) { this.staffID = staffID; }  
public void setLocation(String location) { this.location = location; }
```

```
// Private Helper Methods
```

```
private void setTime(String startTimeStr, String endTimeStr) throws  
InvalidScheduleException {  
    try {  
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm");  
        Date startDate = sdf.parse(startTimeStr);
```

```

        Date endDate = sdf.parse(endTimeStr);
        this.startTime = new Timestamp(startDate.getTime());
        this.endTime = new Timestamp(endDate.getTime());
        validateTime();
    } catch (ParseException e) {
        throw new InvalidScheduleException("Invalid date/time format. Use yyyy-MM-dd
HH:mm.");
    }
}

```

```

private void validateTime() throws InvalidScheduleException {
    if (endTime.before(startTime) || endTime.equals(startTime)) {
        throw new InvalidScheduleException("End time must be after start time.");
    }
}

```

```

private boolean isValidStatus(String status) {
    return status != null && getValidStatusList().contains(status);
}

```

@Override

```

public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Schedule)) return false;
    Schedule schedule = (Schedule) o;
    return scheduleID == schedule.scheduleID;
}

```

```
}
```

```
@Override
```

```
public int hashCode() {
```

```
    return Objects.hash(scheduleID);
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Schedule [ID=" + scheduleID + ", StaffID=" + staffID + ", Start=" + startTime +
```

```
        ", End=" + endTime + ", Status=" + status + ", Location=" + location + "];
```

```
}
```

```
}
```

//File Name Schedule.java

//Accessing Package comhms.src.hms.admin.view

```
package comhms.src.hms.admin.view;
```

```
import comhms.src.hms.admin.model.Admin;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import comhms.src.hms.auth.controller.UserLoginService;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import java.util.Scanner;
```

```
public class AdminView {
```

```

private Scanner scanner;

private Admin admin;

private UserLoginService loginService; // For audit logs and user management

private AdminService adminService; // Placeholder for schedule management


// Constructor

public AdminView(Admin admin, UserLoginService loginService) {

    this.scanner = new Scanner(System.in);

    this.admin = admin;

    this.loginService = loginService;

    this.adminService = new AdminService(); // Placeholder
}


// Overloaded Constructor

public AdminView(Admin admin, UserLoginService loginService, AdminService
adminService) {

    this.scanner = new Scanner(System.in);

    this.admin = admin;

    this.loginService = loginService;

    this.adminService = adminService;
}


// Inner Class for Menu Options

private class MenuOption {

    private String description;

    private Runnable action;

```

```

MenuOption(String description, Runnable action) {
    this.description = description;
    this.action = action;
}

void display(int index) {
    System.out.println(index + ". " + description);
}

void execute() {
    action.run();
}
}

// Method with Object Reference
public void displayAdminMenu(Admin admin) {
    this.admin = admin; // Update admin reference
    MenuOption[] options = {
        new MenuOption("View Audit Logs", this::handleViewAuditLogs),
        new MenuOption("Manage Users", this::handleManageUsers),
        new MenuOption("Manage Schedules", this::handleManageSchedules),
        new MenuOption("Logout", () -> System.out.println("Logging out..."))
    };
}

```



```

while (true) {

    System.out.println("\n=== Admin Dashboard ===");

    System.out.println("Welcome, " + admin.getUsername() + " (" + admin.getDepartment()
+ ")");

    for (int i = 0; i < options.length; i++) {

        options[i].display(i + 1);

    }

    int choice = getValidChoice(options.length);

    if (choice == options.length) { // Logout

        options[choice - 1].execute();

        break;

    }

    options[choice - 1].execute();

}
}

```

```

// Method without Object Reference (Static)

public static String getWelcomeMessage() {

    return "Welcome to the Admin Dashboard!";

}

```

```

// Handle View Audit Logs

private void handleViewAuditLogs() {

    try {

        String logs = admin.viewAuditLogs(loginService.getAuditLogs());

```

```
        System.out.println(logs);
    } catch (Admin.AdminAccessException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

// Handle Manage Users

```
private void handleManageUsers() {
    System.out.println("\n=== Manage Users ===");
    System.out.println("1. Activate/Deactivate User");
    System.out.println("2. Update User Role");
    System.out.println("3. Back");
    int choice = getValidChoice(3);

    if (choice == 3) return;

    System.out.print("Enter User ID (e.g., AD0001): ");
    String userID = scanner.nextLine();

    try {
        UserLogin user = loginService.getUserByID(userID);
        if (choice == 1) {
            System.out.print("Activate user? (y/n): ");
            boolean activate = scanner.nextLine().trim().toLowerCase().startsWith("y");
            String result = admin.manageUser(user, activate);
        }
    }
```

```

        System.out.println(result);
    } else if (choice == 2) {
        System.out.println("Enter new role (Admin, Receptionist, Staff, Patient): ");
        String newRole = scanner.nextLine();
        String result = admin.manageUser(user, newRole);
        System.out.println(result);
    }
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}

```

// Handle Manage Schedules

```

private void handleManageSchedules() {
    System.out.println("\n=== Manage Schedules ===");
    System.out.println("1. Create Schedule");
    System.out.println("2. Update Schedule Status");
    System.out.println("3. Back");
    int choice = getValidChoice(3);

    if (choice == 3) return;

    try {
        if (choice == 1) {
            System.out.print("Enter Staff ID (e.g., 2): ");

```

```

int staffID = Integer.parseInt(scanner.nextLine());

System.out.print("Enter Start Time (yyyy-MM-dd HH:mm): ");

String startTime = scanner.nextLine();

System.out.print("Enter End Time (yyyy-MM-dd HH:mm): ");

String endTime = scanner.nextLine();

System.out.print("Enter Location (e.g., Room 101): ");

String location = scanner.nextLine();


Schedule schedule = new Schedule(adminService.getNextScheduleID(), staffID,
startTime, endTime, "Available", location);

adminService.addSchedule(schedule);

System.out.println("Schedule created: " + schedule);

} else if (choice == 2) {

System.out.print("Enter Schedule ID: ");

int scheduleID = Integer.parseInt(scanner.nextLine());

System.out.print("Enter New Status (Available, Booked, Cancelled): ");

String newStatus = scanner.nextLine();

System.out.print("Enter Reason (or press Enter to skip): ");

String reason = scanner.nextLine();


Schedule schedule = adminService.getScheduleByID(scheduleID);

String result = reason.isEmpty() ? schedule.updateStatus(newStatus) :
schedule.updateStatus(newStatus, reason);

System.out.println(result);

}

} catch (Exception e) {

```

```

        System.out.println("Error: " + e.getMessage());
    }
}

// Helper Method
private int getValidChoice(int max) {
    while (true) {
        try {
            System.out.print("Enter choice (1-" + max + "): ");
            int choice = Integer.parseInt(scanner.nextLine());
            if (choice >= 1 && choice <= max) {
                return choice;
            }
            System.out.println("Invalid choice! Try again.");
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number!");
        }
    }
}
}

```

```

// Placeholder AdminService
class AdminService {
    private int nextScheduleID = 1;
}

```

```

public int getNextScheduleID() {
    return nextScheduleID++;
}

public void addSchedule(Schedule schedule) {
    // Simulate adding to database

    System.out.println("Added to database: " + schedule);
}

public Schedule getScheduleByID(int scheduleID) throws Schedule.InvalidScheduleException
{
    // Simulate fetching from database

    return new Schedule(scheduleID, 2, "2025-05-04 09:00", "2025-05-04 10:00", "Available",
"Room 101");
}
}

```

//File Name UserLoginService.java

//Accessing Package comhms.src.hms.auth.controller

```

package comhms.src.hms.auth.controller;

import comhms.src.hms.auth.model.UserLogin;
import comhms.src.hms.auth.model.AuditLog;
import comhms.src.hms.auth.util.DashboardRouter;
import comhms.src.hms.auth.util.DashboardRouter.InvalidRoleException;

```

```

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;


public class UserLoginService {

    private List<UserLogin> userDatabase; // Simulated UserLogin table

    private List<AuditLog> auditLogDatabase; // Simulated Audit_Log table

    private Map<String, Integer> idCounters; // Track ID counts per role

    private DashboardRouter router; // To be implemented


    // Constructor

    public UserLoginService() {

        this.userDatabase = new ArrayList<>();

        this.auditLogDatabase = new ArrayList<>();

        this.idCounters = new HashMap<>();

        this.router = new DashboardRouter(); // Placeholder

        initializeDefaultUsers();

    }


    // Overloaded Constructor

    public UserLoginService(DashboardRouter router) {

        this.userDatabase = new ArrayList<>();

        this.auditLogDatabase = new ArrayList<>();
    }

```

```
this.idCounters = new HashMap<>();  
  
this.router = router;  
  
initializeDefaultUsers();  
  
}
```

```
// Inner Class for Authentication Result
```

```
private class AuthResult {  
  
    private boolean success;  
  
    private String message;  
  
    AuthResult(boolean success, String message) {  
  
        this.success = success;  
  
        this.message = message;  
  
    }  
  
}
```

```
// Method with Object Reference
```

```
public void login(UserLogin inputUser) throws UserLogin.InvalidCredentialsException {  
  
    AuthResult result = authenticate(inputUser);  
  
    logAuditAttempt(inputUser, result.success, result.message);  
  
    if (!result.success) {  
  
        throw new UserLogin.InvalidCredentialsException(result.message);  
  
    }  
  
}
```


// Method without Object Reference (Static)

```
public static String generateIDPrefix(String role) {  
    return role.substring(0, 2).toUpperCase(); // E.g., "Admin" -> "AD"  
}
```

// Overloaded Method: Register User

```
public void registerUser(UserLogin user) throws DuplicateUserException {  
    if (isUsernameTaken(user.getUsername())) {  
        throw new DuplicateUserException("Username already exists!");  
    }  
    userDatabase.add(user);  
    logAuditAttempt(user, true, "User registered successfully");  
    idCounters.put(user.getRole(), idCounters.getOrDefault(user.getRole(), 0) + 1);  
}
```

// Overloaded Method: Register User with Custom ID

```
public void registerUser(UserLogin user, String customID) throws DuplicateUserException {  
    if (isUsernameTaken(user.getUsername())) {  
        throw new DuplicateUserException("Username already exists!");  
    }  
    userDatabase.add(user);  
    logAuditAttempt(user, true, "User registered with ID: " + customID);  
}
```

// Method to Generate User ID

```

public String generateUserID(String role) {
    String prefix = generateIDPrefix(role);
    int count = idCounters.getOrDefault(role, 0) + 1;
    return prefix + String.format("%04d", count); // E.g., AD0001
}

```

// Method to Get User by ID

```

public UserLogin getUserByID(String userID) throws InvalidUserIDException {
    try {
        int id = Integer.parseInt(userID.substring(2));
        for (UserLogin user : userDatabase) {
            if (user.getUserID() == id && userID.startsWith(generateIDPrefix(user.getRole()))) {
                return user;
            }
        }
    } catch (NumberFormatException e) {
        throw new InvalidUserIDException("Invalid User ID format!");
    }
    throw new InvalidUserIDException("User ID not found!");
}

```

// Method to Verify Security Answer

```

public boolean verifySecurityAnswer(UserLogin user, String answer) {
    boolean success = user.getSecurityAnswer().equalsIgnoreCase(answer);
    logAuditAttempt(user, success, success ? "Security answer verified" : "Incorrect security answer");
}

```

```
        return success;
    }

// Method to Route to Dashboard
public void routeToDashboard(UserLogin user) throws InvalidRoleException {
    router.redirect(user); // Delegates to DashboardRouter
}

// User Defined Exception
public static class DuplicateUserException extends Exception {
    public DuplicateUserException(String message) {
        super(message);
    }
}

// User Defined Exception
public static class InvalidUserIDException extends Exception {
    public InvalidUserIDException(String message) {
        super(message);
    }
}

// Private Helper Methods
private void initializeDefaultUsers() {
    // Add a default admin for testing
```

```

    try {
        UserLogin admin = new UserLogin(1, "admin1", "Pass@123", "Admin", "Nurse", "John
Doe", 30,
            "What is your pet's name?", "Max");
        userDatabase.add(admin);
        idCounters.put("Admin", 1);
    } catch (Exception e) {
        // Ignore for now
    }
}

```

```

private boolean isUsernameTaken(String username) {
    return userDatabase.stream().anyMatch(u -> u.getUsername().equals(username));
}

```

```

private AuthResult authenticate(UserLogin inputUser) {
    for (UserLogin user : userDatabase) {
        try {
            if (user.validateCredentials(inputUser)) {
                return new AuthResult(true, "Login successful");
            }
        } catch (UserLogin.InvalidCredentialsException e) {
            return new AuthResult(false, e.getMessage());
        }
    }
    return new AuthResult(false, "Invalid username or password");
}

```

```
}
```

```
private void logAuditAttempt(UserLogin user, boolean success, String message) {  
    int logID = auditLogDatabase.size() + 1;  
    String role = user.getRole() + (user.getSubRole() != null ? "/" + user.getSubRole() : "");  
    AuditLog log = new AuditLog(logID, user.getUserID(), role, "Login Attempt",  
                                "192.168.1.1", "Desktop", success);  
    auditLogDatabase.add(log);  
    System.out.println("Audit Log: " + log); // For testing  
}
```

```
// Getter for Audit Logs (for Admin)  
public List<AuditLog> getAuditLogs() {  
    return new ArrayList<>(auditLogDatabase);  
}  
}
```

//File Name AuditLog.java

//Accessing Package comhms.src.hms.auth.model

```
package comhms.src.hms.auth.model;
```

```
import java.sql.Timestamp;
```

```
import java.util.Objects;
```

```

public class AuditLog {
    private int logID;
    private Integer userID; // Nullable for guest actions
    private String role;
    private String action;
    private ActionDetails details;
    private Timestamp timestamp;

    // Constructor
    public AuditLog(int logID, Integer userID, String role, String action, String
ipAddress, String deviceInfo, boolean success) {
        this.logID = logID;
        this.userID = userID;
        this.role = role;
        this.action = action;
        this.details = new ActionDetails(ipAddress, deviceInfo, success);
        this.timestamp = new Timestamp(System.currentTimeMillis());
    }

    // Overloaded Constructor
    public AuditLog(int logID, Integer userID, String role, String action) {
        this(logID, userID, role, action, "192.168.1.1", "Unknown Device", false); //
Default values
    }

```

```
// Inner Class for Action Details
```

```
private class ActionDetails {
```

```
    private String ipAddress;
```

```
    private String deviceInfo;
```

```
    private boolean success;
```

```
    ActionDetails(String ipAddress, String deviceInfo, boolean success) {
```

```
        this.ipAddress = ipAddress;
```

```
        this.deviceInfo = deviceInfo;
```

```
        this.success = success;
```

```
    }
```

```
    String getSummary() {
```

```
        return "IP: " + ipAddress + ", Device: " + deviceInfo + ", Success: " +  
success;
```

```
    }
```

```
}
```

```
// Method with Object Reference
```

```
public boolean matchesUser(AuditLog otherLog) throws  
InvalidAuditLogException {
```

```
    if (otherLog == null) {
```

```
        throw new InvalidAuditLogException("Comparison log cannot be null.");
```

```

    }

    return this.userID != null && otherLog.userID != null &&
this.userID.equals(otherLog.userID);

}

// Method without Object Reference (Static)

public static String formatAction(String action) {

    return action.toUpperCase().replace(" ", "_"); // E.g., "Login Attempt" ->
"LOGIN_ATTEMPT"

}

// User Defined Exception

public static class InvalidAuditLogException extends Exception {

    public InvalidAuditLogException(String message) {

        super(message);

    }

}

// Getters

public int getLogID() { return logID; }

public Integer getUserID() { return userID; }

public String getRole() { return role; }

public String getAction() { return action; }

public String getDetailsSummary() { return details.getSummary(); }

```



```
public Timestamp getTimestamp() { return timestamp; }
```

```
@Override
```

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (!(o instanceof AuditLog)) return false;  
    AuditLog auditLog = (AuditLog) o;  
    return logID == auditLog.logID;  
}
```

```
@Override
```

```
public int hashCode() {  
    return Objects.hash(logID);  
}
```

```
@Override
```

```
public String toString() {  
    return "AuditLog [ID=" + logID + ", UserID=" + (userID != null ? userID :  
"Guest") +  
        ", Role=" + role + ", Action=" + action + ", Details=" +  
getDetailsSummary() +  
        ", Timestamp=" + timestamp + "];"  
}  
}
```

//File Name UserLogin.java

//Accessing Package comhms.src.hms.auth.model

```
package comhms.src.hms.auth.model;
```

```
import java.sql.Timestamp;
```

```
import java.util.Objects;
```

```
import java.util.regex.Pattern;
```

```
public class UserLogin {
```

```
    private int userID;
```

```
    private String username;
```

```
    private String password; // Plain text
```

```
    protected String role;
```

```
    private String subRole; // For Staff (Doctor, Nurse)
```

```
    private String name;
```

```
    private int age;
```

```
    private String securityQuestion;
```

```
    private String securityAnswer;
```

```
    private Timestamp lastLogin;
```

```
    private int failedAttempts;
```

```
    private boolean accountLocked;
```

```
    // Constructor
```

```
public UserLogin(int userID, String username, String password, String role, String name, int
age) {
    this.userID = userID;
    this.username = username;
    this.password = password;
    this.role = role;
    this.name = name;
    this.age = age;
    this.failedAttempts = 0;
    this.accountLocked = false;
}
```

// Overloaded Constructor

```
public UserLogin(int userID, String username, String password, String role, String subRole,
String name, int age, String securityQuestion, String securityAnswer) {
    this(userID, username, password, role, name, age);
    this.subRole = subRole;
    this.securityQuestion = securityQuestion;
    this.securityAnswer = securityAnswer;
}
```

// Inner Class for Login Attempt

```
private class LoginAttempt {
    private String ipAddress;
    private String deviceInfo;
    private boolean success;
```

```
private Timestamp timestamp;
```

```
LoginAttempt(String ipAddress, String deviceInfo, boolean success) {  
    this.ipAddress = ipAddress;  
    this.deviceInfo = deviceInfo;  
    this.success = success;  
    this.timestamp = new Timestamp(System.currentTimeMillis());  
}  
}
```

```
// Method with Object Reference
```

```
public boolean validateCredentials(UserLogin inputUser) throws InvalidCredentialsException  
{  
    if (accountLocked) {  
        throw new InvalidCredentialsException("Account is locked due to multiple failed  
attempts.");  
    }  
    if (inputUser.getUsername().equals(this.username) &&  
inputUser.getPassword().equals(this.password)) {  
        resetFailedAttempts();  
        this.lastLogin = new Timestamp(System.currentTimeMillis());  
        return true;  
    }  
    incrementFailedAttempts();  
    throw new InvalidCredentialsException("Invalid username or password.");  
}
```

// Method without Object Reference (Static)

```
public static boolean isValidPassword(String password) {  
    // 8-12 chars, at least one letter, number, special char  
    String regex = "^(?=.*[A-Za-z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,12}$";  
    return Pattern.matches(regex, password);  
}
```

// User Defined Exception

```
public static class InvalidCredentialsException extends Exception {  
    public InvalidCredentialsException(String message) {  
        super(message);  
    }  
}
```

// Getters and Setters

```
public int getUserID() { return userID; }  
public String getUsername() { return username; }  
public String getPassword() { return password; }  
public String getRole() { return role; }  
public String getSubRole() { return subRole; }  
public String getName() { return name; }  
public int getAge() { return age; }  
public String getSecurityQuestion() { return securityQuestion; }  
public String getSecurityAnswer() { return securityAnswer; }
```

```

public Timestamp getLastLogin() { return lastLogin; }

public int getFailedAttempts() { return failedAttempts; }

public boolean isAccountLocked() { return accountLocked; }


public void setSecurityQuestion(String question) { this.securityQuestion = question; }

public void setSecurityAnswer(String answer) { this.securityAnswer = answer; }

public void lockAccount() { this.accountLocked = true; }

public void unlockAccount() { this.accountLocked = false; }


private void incrementFailedAttempts() {
    failedAttempts++;
    if (failedAttempts >= 3) {
        lockAccount();
    }
}

private void resetFailedAttempts() {
    failedAttempts = 0;
    unlockAccount();
}


@Override

public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof UserLogin)) return false;

```

```
UserLogin user = (UserLogin) o;  
  
return userID == user.userID && username.equals(user.username);  
  
}  
  
@Override  
  
public int hashCode() {  
  
    return Objects.hash(userID, username);  
  
}  
  
}
```

//File Name CaptchaValidator.java

//Accessing Package comhms.src.hms.auth.util

```
package comhms.src.hms.auth.util;
```

```
import java.util.Random;
```

```
import java.util.Scanner;
```

```
public class CaptchaValidator {
```

```
    private Random random;
```

```
    private Scanner scanner;
```

```
    private Captcha currentCaptcha;
```

```
    // Constructor
```

```
public CaptchaValidator() {  
    this.random = new Random();  
    this.scanner = new Scanner(System.in);  
}
```

// Overloaded Constructor

```
public CaptchaValidator(Random random, Scanner scanner) {  
    this.random = random;  
    this.scanner = scanner;  
}
```

// Inner Class for Captcha

```
private class Captcha {  
    private int number1;  
    private int number2;  
    private int answer;  
  
    Captcha() {  
        this.number1 = random.nextInt(10) + 1; // 1-10  
        this.number2 = random.nextInt(10) + 1; // 1-10  
        this.answer = number1 + number2;  
    }  
  
    String getQuestion() {  
        return "What is " + number1 + " + " + number2 + "?";  
    }  
}
```



```
}
```

```
boolean isCorrect(int userAnswer) {
```

```
    return userAnswer == answer;
```

```
}
```

```
}
```

```
// Method with Object Reference
```

```
public boolean validateCaptcha(CaptchaValidator otherValidator) throws  
InvalidCaptchaException {
```

```
    if (otherValidator == null) {
```

```
        throw new InvalidCaptchaException("Validator cannot be null.");
```

```
    }
```

```
    return validateCaptcha(); // Delegate to main validation
```

```
}
```

```
// Method without Object Reference (Static)
```

```
public static String getCaptchaInstruction() {
```

```
    return "Please solve the math captcha to proceed.";
```

```
}
```

```
// Main Validation Method
```

```
public boolean validateCaptcha() throws InvalidCaptchaException {
```

```
    currentCaptcha = new Captcha();
```

```
    System.out.println(getCaptchaInstruction());
```

```
    System.out.println(currentCaptcha.getQuestion());
```

```

try {
    System.out.print("Enter answer: ");

    int userAnswer = Integer.parseInt(scanner.nextLine());

    if (currentCaptcha.isCorrect(userAnswer)) {
        return true;
    } else {
        throw new InvalidCaptchaException("Incorrect captcha answer!");
    }
} catch (NumberFormatException e) {
    throw new InvalidCaptchaException("Please enter a valid number!");
}
}

// User Defined Exception

public static class InvalidCaptchaException extends Exception {
    public InvalidCaptchaException(String message) {
        super(message);
    }
}
}

```

//File Name DashboardRouter.java

//Accessing Package comhms.src.hms.auth.util

```
package comhms.src.hms.auth.util;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class DashboardRouter {
```

```
    private Map<String, String> dashboardMap;
```

```
    // Constructor
```

```
    public DashboardRouter() {
```

```
        this.dashboardMap = new HashMap<>();
```

```
        initializeDashboardMap();
```

```
    }
```

```
    // Overloaded Constructor
```

```
    public DashboardRouter(Map<String, String> customDashboardMap) {
```

```
        this.dashboardMap = customDashboardMap != null ? customDashboardMap : new  
HashMap<>();
```

```
        if (customDashboardMap == null) {
```

```
            initializeDashboardMap();
```

```
        }
```

```
    }
```

```
// Inner Class for Dashboard Info
```

```
private class DashboardInfo {
```

```
    private String name;
```

```
    private String description;
```

```
    DashboardInfo(String name, String description) {
```

```
        this.name = name;
```

```
        this.description = description;
```

```
    }
```

```
    String getRedirectMessage() {
```

```
        return "Redirecting to " + name + ": " + description;
```

```
    }
```

```
}
```

```
// Method with Object Reference
```

```
public void redirect(UserLogin user) throws InvalidRoleException {
```

```
    if (user == null) {
```

```
        throw new InvalidRoleException("User cannot be null.");
```

```
    }
```

```
    String roleKey = user.getRole() + (user.getSubRole() != null ? "/" + user.getSubRole() : "");
```

```
    String dashboard = dashboardMap.getOrDefault(roleKey, "Unknown Dashboard");
```

```
    DashboardInfo info = new DashboardInfo(dashboard, getDashboardDescription(roleKey));
```

```
    System.out.println(info.getRedirectMessage());
```

```
}
```

```
// Method without Object Reference (Static)
```

```
public static String getDefaultDashboard(String role) {  
    return role + "Dashboard"; // E.g., "Admin" -> "AdminDashboard"  
}
```

```
// Overloaded Method: Redirect with Custom Message
```

```
public void redirect(UserLogin user, String customMessage) throws InvalidRoleException {  
    if (user == null) {  
        throw new InvalidRoleException("User cannot be null.");  
    }  
  
    String roleKey = user.getRole() + (user.getSubRole() != null ? "/" + user.getSubRole() : "");  
    String dashboard = dashboardMap.getOrDefault(roleKey, "Unknown Dashboard");  
    System.out.println(customMessage + " -> " + dashboard);  
}
```

```
// User Defined Exception
```

```
public static class InvalidRoleException extends Exception {  
    public InvalidRoleException(String message) {  
        super(message);  
    }  
}
```

```
// Private Helper Method
```

```
private void initializeDashboardMap() {  
    dashboardMap.put("Admin", "AdminDashboard");  
    dashboardMap.put("Receptionist", "ReceptionistDashboard");  
    dashboardMap.put("Staff/Doctor", "StaffDashboard (Doctor)");  
    dashboardMap.put("Staff/Nurse", "StaffDashboard (Nurse)");  
    dashboardMap.put("Patient", "PatientDashboard");  
}
```

```
private String getDashboardDescription(String roleKey) {  
    switch (roleKey) {  
        case "Admin":  
            return "Manage users, schedules, and reports";  
        case "Receptionist":  
            return "Handle patient registration and appointments";  
        case "Staff/Doctor":  
            return "View appointments and write prescriptions";  
        case "Staff/Nurse":  
            return "Record patient vitals and assist doctors";  
        case "Patient":  
            return "View medical history and appointments";  
        default:  
            return "Unknown dashboard";  
    }  
}
```

//File Name LoginView.java

//Accessing Package comhms.src.hms.auth.view

```
package comhms.src.hms.auth.view;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import comhms.src.hms.auth.controller.UserLoginService;
```

```
import comhms.src.hms.auth.util.CaptchaValidator;
```

```
import comhms.src.hms.auth.util.CaptchaValidator.InvalidCaptchaException;
```

```
import comhms.src.hms.auth.util.DashboardRouter.InvalidRoleException;
```

```
import java.util.Scanner;
```

```
import java.util.regex.Pattern;
```

```
public class LoginView {
```

```
    private Scanner scanner;
```

```
    private UserLoginService loginService; // To be implemented
```

```
    private CaptchaValidator captchaValidator; // To be implemented
```

```
    private int failedAttempts;
```

```
    // Constructor
```

```
    public LoginView() {
```

```
        this.scanner = new Scanner(System.in);
```

```
        this.loginService = new UserLoginService(); // Placeholder
```

```
        this.captchaValidator = new CaptchaValidator(); // Placeholder
```

```
        this.failedAttempts = 0;
    }

// Overloaded Constructor
public LoginView(UserLoginService loginService, CaptchaValidator captchaValidator) {
    this.scanner = new Scanner(System.in);
    this.loginService = loginService;
    this.captchaValidator = captchaValidator;
    this.failedAttempts = 0;
}

// Inner Class for Menu Options
private class MenuOption {
    private String description;
    private Runnable action;

    MenuOption(String description, Runnable action) {
        this.description = description;
        this.action = action;
    }

    void display(int index) {
        System.out.println(index + ". " + description);
    }
}
```



```
void execute() {  
    action.run();  
}  
}
```

// Method with Object Reference

```
public void displayLoginMenu(UserLoginService service) {  
    this.loginService = service; // Update service reference  
    MenuOption[] options = {  
        new MenuOption("Login", () -> {  
            try {  
                handleLogin();  
            } catch (InvalidCaptchaException | InvalidRoleException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    ),  
        new MenuOption("Create Account", this::handleCreateAccount),  
        new MenuOption("Forgot Password", () -> {  
            try {  
                handleForgotPassword();  
            } catch (InvalidCaptchaException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    )  
    }  
}
```

```

    }),
    new MenuOption("Exit", () -> System.exit(0))
};

while (true) {
    System.out.println("\n=== Hospital Management System ===");
    System.out.println("Welcome! Please choose an option:");
    for (int i = 0; i < options.length; i++) {
        options[i].display(i + 1);
    }

    int choice = getValidChoice(options.length);
    options[choice - 1].execute();
}
}

// Method without Object Reference (Static)
public static String getRolePrompt() {
    return "Select Role:\n1. Admin\n2. Receptionist\n3. Staff\n4. Patient";
}

// Handle Login
private void handleLogin() throws InvalidCaptchaException, InvalidRoleException {
    System.out.println("\n=== Login ===");
    System.out.println(getRolePrompt());
}

```

```
String role = getRoleInput();
```

```
String subRole = role.equals("Staff") ? getSubRoleInput() : null;
```

```
System.out.print("Enter Username: ");
```

```
String username = scanner.nextLine();
```

```
System.out.print("Enter Password: ");
```

```
String password = scanner.nextLine();
```

```
if (failedAttempts >= 3) {
```

```
    if (!captchaValidator.validateCaptcha()) {
```

```
        System.out.println("Captcha validation failed!");
```

```
        return;
```

```
    }
```

```
}
```

```
try {
```

```
    UserLogin user = new UserLogin(0, username, password, role, subRole, "", 0, password, password); // Temporary user
```

```
    loginService.login(user);
```

```
    failedAttempts = 0;
```

```
    System.out.println("Welcome back, " + username + "!");
```

```
    loginService.routeToDashboard(user); // Simulated redirect
```

```
} catch (UserLogin.InvalidCredentialsException e) {
```

```
    failedAttempts++;
```

```
    System.out.println("Error: " + e.getMessage());
```

```
    if (failedAttempts >= 3) {
```

```

        System.out.println("Too many failed attempts. Please complete the captcha.");
    }
}
}

```

// Handle Account Creation

```

private void handleCreateAccount() {
    System.out.println("\n=== Create Account ===");
    System.out.println(getRolePrompt());
    String role = getRoleInput();
    String subRole = role.equals("Staff") ? getSubRoleInput() : null;

    System.out.print("Enter Name: ");
    String name = scanner.nextLine();
    System.out.print("Enter Age: ");
    int age = getValidAge();

    String username;
    do {
        System.out.print("Enter Username: ");
        username = scanner.nextLine();
    } while (!isValidUsername(username));

    String password;
    do {

```

```

System.out.print("Enter Password (8-12 chars, letter, number, special char): ");
password = scanner.nextLine();
if (!UserLogin.isValidPassword(password)) {
    System.out.println("Invalid password format!");
    continue;
}
System.out.print("Confirm Password: ");
String confirmPassword = scanner.nextLine();
if (!password.equals(confirmPassword)) {
    System.out.println("Passwords do not match! ");
} else {
    System.out.println("Passwords match! ");
    break;
}
} while (true);

System.out.println("Select Security Question:");

String[] questions = {"What is your pet's name?", "What was your first school?",
"Custom"};

for (int i = 0; i < questions.length; i++) {
    System.out.println((i + 1) + ". " + questions[i]);
}

int qChoice = getValidChoice(questions.length);

String securityQuestion = qChoice == questions.length ? getCustomQuestion() :
questions[qChoice - 1];

System.out.print("Enter Security Answer: ");

```

```

String securityAnswer = scanner.nextLine();

try {

    String userID = loginService.generateUserID(role); // Simulated ID

    UserLogin user = new UserLogin(Integer.parseInt(userID.substring(2)), username,
password, role, subRole, name, age, securityQuestion, securityAnswer);

    loginService.registerUser(user);

    System.out.println("Account created! Your User ID is: " + userID);

} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());

}

}

// Handle Forgot Password

private void handleForgotPassword() throws InvalidCaptchaException {

    System.out.println("\n=== Forgot Password ===");

    System.out.print("Enter User ID: ");

    String userID = scanner.nextLine();

    if (!captchaValidator.validateCaptcha()) {

        System.out.println("Captcha validation failed!");

        return;

    }

    try {

        UserLogin user = loginService.getUserByID(userID);

```

```

        System.out.println("Security Question: " + user.getSecurityQuestion());

        System.out.print("Enter Answer: ");

        String answer = scanner.nextLine();

        if (loginService.verifySecurityAnswer(user, answer)) {

            System.out.println("Password sent to your registered email. (Dev mode: Your password is: " + user.getPassword() + ")");

        } else {

            System.out.println("Incorrect security answer!");

        }

    } catch (Exception e) {

        System.out.println("Error: " + e.getMessage());

    }

}

```

// Helper Methods

```

private String getRoleInput() {

    int choice = getValidChoice(4);

    switch (choice) {

        case 1: return "Admin";

        case 2: return "Receptionist";

        case 3: return "Staff";

        case 4: return "Patient";

        default: return "Patient"; // Fallback

    }

}

```

```

private String getSubRoleInput() {
    System.out.println("Select Sub-Role:\n1. Doctor\n2. Nurse");
    int choice = getValidChoice(2);
    return choice == 1 ? "Doctor" : "Nurse";
}

```

```

private int getValidChoice(int max) {
    int choice;
    while (true) {
        try {
            System.out.print("Enter choice (1-" + max + "): ");
            choice = Integer.parseInt(scanner.nextLine());
            if (choice >= 1 && choice <= max) {
                return choice;
            }
            System.out.println("Invalid choice! Try again.");
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number!");
        }
    }
}

```

```

private int getValidAge() {
    while (true) {

```



```
try {  
    int age = Integer.parseInt(scanner.nextLine());  
    if (age > 0 && age < 150) {  
        return age;  
    }  
    System.out.println("Invalid age! Enter a valid age.");  
} catch (NumberFormatException e) {  
    System.out.println("Please enter a number!");  
}  
}  
}
```

```
private boolean isValidUsername(String username) {  
    return Pattern.matches("^[A-Za-z0-9]{3,20}$", username);  
}
```

```
private String getCustomQuestion() {  
    System.out.print("Enter Custom Security Question: ");  
    return scanner.nextLine();  
}  
}
```

//File Name PatientService.java

//Accessing Package comhms.src.hms.patient.controller

```
package comhms.src.hms.patient.controller;
```

```
import comhms.src.hms.patient.model.Patient;
```

```
import comhms.src.hms.patient.model.MedicalRecord;
```

```
import comhms.src.hms.reception.model.Appointment;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import comhms.src.hms.auth.controller.UserLoginService;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import java.util.Objects;
```

```
public class PatientService {
```

```
    private List<Appointment> appointmentDatabase; // Simulated Appointment table
```

```
    private List<Schedule> scheduleDatabase; // Simulated Schedule table
```

```
    private List<MedicalRecord> medicalRecordDatabase; // Simulated MedicalRecord table
```

```
    private UserLoginService loginService; // For authentication and user data
```

```
    // Constructor
```

```
    public PatientService(UserLoginService loginService) {
```

```
        this.appointmentDatabase = new ArrayList<>();
```

```
        this.scheduleDatabase = new ArrayList<>();
```

```
        this.medicalRecordDatabase = new ArrayList<>();
```

```
this.loginService = loginService;  
initializeDefaultData();  
}
```

// Overloaded Constructor

```
public PatientService(UserLoginService loginService, List<Appointment>  
initialAppointments,  
List<Schedule> initialSchedules, List<MedicalRecord> initialMedicalRecords) {  
    this.appointmentDatabase = initialAppointments != null ? initialAppointments : new  
ArrayList<>();  
    this.scheduleDatabase = initialSchedules != null ? initialSchedules : new ArrayList<>();  
    this.medicalRecordDatabase = initialMedicalRecords != null ? initialMedicalRecords : new  
ArrayList<>();  
    this.loginService = loginService;  
}
```

// Inner Class for Service Operation Result

```
private class OperationResult {  
    private boolean success;  
    private String message;  
  
    OperationResult(boolean success, String message) {  
        this.success = success;  
        this.message = message;  
    }  
}
```

```
String getMessage() {  
    return message;  
}  
}
```

// Method with Object Reference

```
public String viewAppointments(Patient patient) throws Patient.PatientAccessException {  
    OperationResult opResult = new OperationResult(true,  
        patient.viewAppointments(appointmentDatabase));  
    return opResult.getMessage();  
}
```

// Method without Object Reference (Static)

```
public static String getServiceDescription() {  
    return "PatientService: Manages patient tasks like viewing appointments and medical  
records.";  
}
```

// Overloaded Method: View Medical Record (Basic)

```
public String viewMedicalRecord(Patient patient) throws Patient.PatientAccessException {  
    StringBuilder result = new StringBuilder("Medical Records for Patient " +  
patient.getPatientID() + ":\n");  
    boolean hasRecords = false;  
    for (MedicalRecord record : medicalRecordDatabase) {  
        if (patient.getPatientID().equals(record.getPatientID())) {  
            result.append(record.getRecordDetails()).append("\n");  
            hasRecords = true;  
        }  
    }  
    return result.toString();  
}
```

```

        hasRecords = true;
    }
}

if (!hasRecords) {
    result.append("No medical records found.\n");
}

OperationResult opResult = new OperationResult(true,
    result + patient.viewMedicalRecord());
return opResult.getMessage();
}

// Overloaded Method: View Medical Record (With Appointment)
public String viewMedicalRecord(Patient patient, int appointmentID)
    throws Patient.PatientAccessException {
    Appointment appointment = findAppointmentByID(appointmentID);
    if (appointment == null || !patient.getPatientID().equals(appointment.getPatientID())) {
        throw new Patient.PatientAccessException("Invalid or mismatched appointment ID.");
    }

    StringBuilder result = new StringBuilder("Medical Records for Patient " +
        patient.getPatientID() +
        " (Appointment " + appointmentID + "):\n");

    boolean hasRecords = false;
    for (MedicalRecord record : medicalRecordDatabase) {
        if (patient.getPatientID().equals(record.getPatientID()) &&
            record.getAppointmentID() == appointmentID) {

```

```

        result.append(record.getRecordDetails()).append("\n");
        hasRecords = true;
    }
}

if (!hasRecords) {
    result.append("No medical records found for this appointment.\n");
}

OperationResult opResult = new OperationResult(true,
    result + patient.viewMedicalRecord(appointment));
return opResult.getMessage();
}

// Method to View Appointment Schedule

public String viewAppointmentSchedule(Patient patient, int appointmentID)
    throws Patient.PatientAccessException {
    Appointment appointment = findAppointmentByID(appointmentID);
    if (appointment == null || !patient.getPatientID().equals(appointment.getPatientID())) {
        throw new Patient.PatientAccessException("Invalid or mismatched appointment ID.");
    }

    OperationResult opResult = new OperationResult(true,
        patient.viewAppointmentSchedule(appointment, scheduleDatabase));
    return opResult.getMessage();
}

// User Defined Exception

```

```
public static class PatientServiceException extends Exception {  
    public PatientServiceException(String message) {  
        super(message);  
    }  
}
```

// Helper Method to Find Appointment by ID

```
private Appointment findAppointmentByID(int appointmentID) {  
    for (Appointment appointment : appointmentDatabase) {  
        if (appointment.getAppointmentID() == appointmentID) {  
            return appointment;  
        }  
    }  
    return null;  
}
```

// Private Helper Method to Initialize Default Data

```
private void initializeDefaultData() {  
    try {  
        scheduleDatabase.add(new Schedule(1, 2, "2025-05-04 09:00", "2025-05-04 10:00",  
            "Available", "Room 101"));  
        appointmentDatabase.add(new Appointment(1, "PAT001", 1, "Confirmed"));  
        medicalRecordDatabase.add(new MedicalRecord(1, "PAT001", "Type 2 Diabetes",  
            "Metformin 500mg daily", null, 1));  
    } catch (Schedule.InvalidScheduleException e) {
```

```
        // Ignore for now
    }
}
```

```
@Override
```

```
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof PatientService)) return false;
    PatientService that = (PatientService) o;
    return Objects.equals(appointmentDatabase, that.appointmentDatabase) &&
        Objects.equals(scheduleDatabase, that.scheduleDatabase) &&
        Objects.equals(medicalRecordDatabase, that.medicalRecordDatabase);
}
```

```
@Override
```

```
public int hashCode() {
    return Objects.hash(appointmentDatabase, scheduleDatabase, medicalRecordDatabase);
}
}
```


//File Name MedicalRecord.java

//Accessing Package comhms.src.hms.patient.model

```
package comhms.src.hms.patient.model;
```

```
import comhms.src.hms.reception.model.Appointment;
```

```
import java.sql.Timestamp;
```

```
import java.text.SimpleDateFormat;
```

```
public class MedicalRecord {
```

```
    private int recordID; // E.g., 1
```

```
    private String patientID; // E.g., PAT001
```

```
    private String diagnosis; // E.g., "Type 2 Diabetes"
```

```
    private String treatment; // E.g., "Metformin 500mg daily"
```

```
    private Timestamp recordDate; // When the record was created/updated
```

```
    private int appointmentID; // Links to Appointment (optional)
```

```
// Constructor
```

```
public MedicalRecord(int recordID, String patientID, String diagnosis, String treatment) {
```

```
    this.recordID = recordID;
```

```
    this.patientID = patientID;
```

```
    this.diagnosis = diagnosis != null ? diagnosis : "No diagnosis provided";
```

```
    this.treatment = treatment != null ? treatment : "No treatment provided";
```

```
    this.recordDate = new Timestamp(System.currentTimeMillis());
```

```
    this.appointmentID = 0; // Default: no linked appointment
```

```
}
```

// Overloaded Constructor

```
public MedicalRecord(int recordID, String patientID, String diagnosis, String treatment,
                    Timestamp recordDate, int appointmentID) {
    this.recordID = recordID;
    this.patientID = patientID;
    this.diagnosis = diagnosis != null ? diagnosis : "No diagnosis provided";
    this.treatment = treatment != null ? treatment : "No treatment provided";
    this.recordDate = recordDate != null ? recordDate : new
Timestamp(System.currentTimeMillis());
    this.appointmentID = appointmentID;
}
```

// Inner Class for Record Update

```
private class RecordUpdate {
    private String updateType;
    private String details;

    RecordUpdate(String updateType, String details) {
        this.updateType = updateType;
        this.details = details;
    }

    String getUpdateSummary() {
        return "Update: " + updateType + ", Details: " + details;
    }
}
```

```
}
```

```
// Method with Object Reference
```

```
public String updateDiagnosis(Appointment appointment, String newDiagnosis)
    throws MedicalRecordException {
    if (appointment != null && appointment.getAppointmentID() != this.appointmentID) {
        throw new MedicalRecordException("Mismatched appointment ID for record update.");
    }
    if (newDiagnosis == null || newDiagnosis.trim().isEmpty()) {
        throw new MedicalRecordException("Diagnosis cannot be null or empty.");
    }
    this.diagnosis = newDiagnosis;
    this.recordDate = new Timestamp(System.currentTimeMillis());
    RecordUpdate update = new RecordUpdate("Diagnosis Update", "Updated to: " +
newDiagnosis);
    return "Medical record " + recordID + " updated for patient " + patientID + ". " +
        update.getUpdateSummary();
}
```

```
// Method without Object Reference (Static)
```

```
public static String getMedicalRecordDescription() {
    return "MedicalRecord: Stores patient diagnosis and treatment details.";
}
```

```
// Overloaded Method: Update Treatment (Basic)
```

```
public String updateTreatment(String newTreatment) throws MedicalRecordException {
```

```

        if (newTreatment == null || newTreatment.trim().isEmpty()) {
            throw new MedicalRecordException("Treatment cannot be null or empty.");
        }

        this.treatment = newTreatment;

        this.recordDate = new Timestamp(System.currentTimeMillis());

        RecordUpdate update = new RecordUpdate("Treatment Update", "Updated to: " +
newTreatment);

        return "Medical record " + recordID + " updated for patient " + patientID + ". " +
            update.getUpdateSummary();
    }

// Overloaded Method: Update Treatment (With Appointment)
public String updateTreatment(Appointment appointment, String newTreatment)
    throws MedicalRecordException {
    if (appointment != null && appointment.getAppointmentID() != this.appointmentID) {
        throw new MedicalRecordException("Mismatched appointment ID for record update.");
    }

    if (newTreatment == null || newTreatment.trim().isEmpty()) {
        throw new MedicalRecordException("Treatment cannot be null or empty.");
    }

    this.treatment = newTreatment;

    this.recordDate = new Timestamp(System.currentTimeMillis());

    RecordUpdate update = new RecordUpdate("Treatment Update",
        "Updated to: " + newTreatment +
        " for Appointment " + appointmentID);

    return "Medical record " + recordID + " updated for patient " + patientID + ". " +

```

```

        update.getUpdateSummary();
    }

// Method to Get Record Details
public String getRecordDetails() {
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    RecordUpdate update = new RecordUpdate("N/A", "N/A");
    return "Medical Record [ID=" + recordID + ", PatientID=" + patientID +
        ", Diagnosis=" + diagnosis + ", Treatment=" + treatment +
        ", RecordDate=" + sdf.format(recordDate) +
        ", AppointmentID=" + (appointmentID == 0 ? "None" : appointmentID) +
        ", UpdateSummary=" + update.getUpdateSummary() + "]";
}

// User Defined Exception
public static class MedicalRecordException extends Exception {
    public MedicalRecordException(String message) {
        super(message);
    }
}

// Getters and Setters
public int getRecordID() { return recordID; }
public String getPatientID() { return patientID; }
public String getDiagnosis() { return diagnosis; }

```

```

public String getTreatment() { return treatment; }

public Timestamp getRecordDate() { return recordDate; }

public int getAppointmentID() { return appointmentID; }

public void setDiagnosis(String diagnosis) {
    this.diagnosis = diagnosis != null ? diagnosis : "No diagnosis provided";
}

public void setTreatment(String treatment) {
    this.treatment = treatment != null ? treatment : "No treatment provided";
}

// Override toString
@Override
public String toString() {
    return getRecordDetails();
}
}

```

//File Name Patient.java

//Accessing Package comhms.src.hms.patient.model

```

package comhms.src.hms.patient.model;

import comhms.src.hms.auth.model.UserLogin;
import comhms.src.hms.reception.model.Appointment;

```

```

import comhms.src.hms.admin.model.Schedule;

import java.util.List;


public class Patient extends UserLogin {

    private String patientID; // E.g., PAT001

    private String medicalHistory; // Simplified as a string (e.g., "Diabetes, Hypertension")


    // Constructor

    public Patient(int userID, String username, String password, String name, int age,

        String patientID, String medicalHistory) {

        super(userID, username, password, "Patient", "Patient", name, age, medicalHistory,
medicalHistory);

        this.patientID = patientID;

        this.medicalHistory = medicalHistory != null ? medicalHistory : "No medical history
provided";

    }


    // Overloaded Constructor

    public Patient(int userID, String username, String password, String name, int age,

        String patientID, String medicalHistory, String securityQuestion,

        String securityAnswer) {

        super(userID, username, password, "Patient", "Patient", name, age,

            securityQuestion, securityAnswer);

        this.patientID = patientID;

        this.medicalHistory = medicalHistory != null ? medicalHistory : "No medical history
provided";

```

```
}
```

```
// Inner Class for Patient Task
```

```
private class PatientTask {
```

```
    private String taskType;
```

```
    private String details;
```

```
    PatientTask(String taskType, String details) {
```

```
        this.taskType = taskType;
```

```
        this.details = details;
```

```
    }
```

```
    String getTaskSummary() {
```

```
        return "Task: " + taskType + ", Details: " + details;
```

```
    }
```

```
}
```

```
// Method with Object Reference
```

```
public String viewAppointments(List<Appointment> appointments) throws  
PatientAccessException {
```

```
    if (appointments == null) {
```

```
        throw new PatientAccessException("Appointment list cannot be null.");
```

```
    }
```

```
    StringBuilder result = new StringBuilder("Appointments for Patient " + patientID + ":\n");
```

```
    boolean hasAppointments = false;
```

```
    for (Appointment appointment : appointments) {
```



```

        if (patientID.equals(appointment.getPatientID()) &&
            !"Canceled".equals(appointment.getStatus())) {
            result.append(appointment.toString()).append("\n");
            hasAppointments = true;
        }
    }

    if (!hasAppointments) {
        result.append("No active appointments found.");
    }

    PatientTask task = new PatientTask("View Appointments", "Retrieved for " + patientID);
    return result.toString() + "\n" + task.getTaskSummary();
}

```

// Method without Object Reference (Static)

```

public static String getPatientRoleDescription() {
    return "Patient: Views appointments and medical records.";
}

```

// Overloaded Method: View Medical Record (Basic)

```

public String viewMedicalRecord() throws PatientAccessException {
    if (medicalHistory == null || medicalHistory.trim().isEmpty()) {
        throw new PatientAccessException("No medical history available.");
    }

    PatientTask task = new PatientTask("View Medical Record", "Retrieved for " + patientID);

    return "Medical History for " + patientID + ": " + medicalHistory + "\n" +
        task.getTaskSummary();
}

```

```
}
```

```
// Overloaded Method: View Medical Record (With Appointment Context)
```

```
public String viewMedicalRecord(Appointment appointment) throws PatientAccessException
{
    if (appointment == null || !patientID.equals(appointment.getPatientID())) {
        throw new PatientAccessException("Invalid or mismatched appointment for patient.");
    }
    if (medicalHistory == null || medicalHistory.trim().isEmpty()) {
        throw new PatientAccessException("No medical history available.");
    }
    PatientTask task = new PatientTask("View Medical Record",
        "Retrieved for " + patientID + " with Appointment " +
        appointment.getAppointmentsID());
    return "Medical History for " + patientID + " (Appointment " +
        appointment.getAppointmentsID() + "): " + medicalHistory + "\n" +
    task.getTaskSummary();
}
```

```
// Method to View Schedule Details for an Appointment
```

```
public String viewAppointmentSchedule(Appointment appointment, List<Schedule>
schedules)
    throws PatientAccessException {
    if (appointment == null || schedules == null) {
        throw new PatientAccessException("Appointment or schedule list cannot be null.");
    }
}
```

```

if (!patientID.equals(appointment.getPatientID())) {
    throw new PatientAccessException("Appointment does not belong to this patient.");
}

for (Schedule schedule : schedules) {
    if (schedule.getScheduleID() == appointment.getScheduleID()) {
        PatientTask task = new PatientTask("View Appointment Schedule",
            "Retrieved for Appointment " +
            appointment.getAppointmentsID());
        return "Schedule for Appointment " + appointment.getAppointmentsID() + ":\n" +
            schedule.toString() + "\n" + task.getTaskSummary();
    }
}

throw new PatientAccessException("No matching schedule found for appointment.");
}

```

// User Defined Exception

```

public static class PatientAccessException extends Exception {
    public PatientAccessException(String message) {
        super(message);
    }
}

```

// Getters and Setters

```

public String getPatientID() { return patientID; }

public String getMedicalHistory() { return medicalHistory; }

```

```

    public void setMedicalHistory(String medicalHistory) {

        this.medicalHistory = medicalHistory != null ? medicalHistory : "No medical history
provided";

    }

    // Override toString

    @Override

    public String toString() {

        PatientTask task = new PatientTask("N/A", "N/A");

        return "Patient [Username=" + getUsername() + ", PatientID=" + patientID +

            ", MedicalHistory=" + medicalHistory + ", TaskSummary=" + task.getTaskSummary()

+ "];"

    }

}

```

//File Name MedicalRecordFormatter.java

//Accessing Package comhms.src.hms.patient.util

```

package comhms.src.hms.patient.util;

import comhms.src.hms.patient.model.MedicalRecord;

import comhms.src.hms.patient.model.Patient;

import java.text.SimpleDateFormat;

import java.util.List;

```

```

public class MedicalRecordFormatter {

    private String dateFormat; // Configurable date format


    // Constructor

    public MedicalRecordFormatter() {

        this.dateFormat = "yyyy-MM-dd HH:mm:ss"; // Default format

    }


    // Overloaded Constructor

    public MedicalRecordFormatter(String dateFormat) {

        this.dateFormat = dateFormat != null && !dateFormat.trim().isEmpty() ?

            dateFormat : "yyyy-MM-dd HH:mm:ss";

    }


    // Inner Class for Format Configuration

    private class FormatConfig {

        private boolean includeDetails;

        private boolean patientFriendly;


        FormatConfig(boolean includeDetails, boolean patientFriendly) {

            this.includeDetails = includeDetails;

            this.patientFriendly = patientFriendly;

        }


        String getFormatDescription() {

```

```

        return (patientFriendly ? "Patient-Friendly" : "Standard") +
            (includeDetails ? ", Detailed" : ", Summary");
    }
}

```

// Method with Object Reference

```

public String formatRecord(MedicalRecord record, Patient patient)
    throws MedicalRecordFormatterException {
    if (record == null || patient == null) {
        throw new MedicalRecordFormatterException("Medical record or patient cannot be
null.");
    }
    if (!patient.getPatientID().equals(record.getPatientID())) {
        throw new MedicalRecordFormatterException("Medical record does not belong to
patient.");
    }
    FormatConfig config = new FormatConfig(true, false); // Detailed, standard
    return formatRecordInternal(record, config);
}

```

// Method without Object Reference (Static)

```

public static String getFormatterDescription() {
    return "MedicalRecordFormatter: Formats medical records for display.";
}

```

// Overloaded Method: Format Single Record (Summary)

```

    public String formatRecordSummary(MedicalRecord record) throws
    MedicalRecordFormatterException {

        if (record == null) {

            throw new MedicalRecordFormatterException("Medical record cannot be null.");

        }

        FormatConfig config = new FormatConfig(false, false); // Summary, standard

        return formatRecordInternal(record, config);

    }

// Overloaded Method: Format Multiple Records (Patient-Friendly)

public String formatPatientFriendlyRecords(List<MedicalRecord> records, Patient patient)

    throws MedicalRecordFormatterException {

    if (records == null || patient == null) {

        throw new MedicalRecordFormatterException("Records or patient cannot be null.");

    }

    StringBuilder result = new StringBuilder("Medical Records for " + patient.getUsername() +
    ":\n");

    FormatConfig config = new FormatConfig(true, true); // Detailed, patient-friendly

    boolean hasRecords = false;

    for (MedicalRecord record : records) {

        if (patient.getPatientID().equals(record.getPatientID())) {

            result.append(formatRecordInternal(record, config)).append("\n");

            hasRecords = true;

        }

    }

    if (!hasRecords) {

```

```

        result.append("No medical records found.");
    }
    return result.toString();
}

```

// Private Helper Method to Format Record

```

private String formatRecordInternal(MedicalRecord record, FormatConfig config) {
    SimpleDateFormat sdf = new SimpleDateFormat(dateFormat);
    StringBuilder formatted = new StringBuilder();
    if (config.patientFriendly) {
        formatted.append("Record ID: ").append(record.getRecordID()).append("\n");
        formatted.append("Date: ").append(sdf.format(record.getRecordDate())).append("\n");
        formatted.append("Condition: ").append(record.getDiagnosis()).append("\n");
        formatted.append("Treatment Plan: ").append(record.getTreatment()).append("\n");
        if (record.getAppointmentID() != 0) {
            formatted.append("Related Appointment:
").append(record.getAppointmentID()).append("\n");
        }
    } else if (config.includeDetails) {
        formatted.append("Medical Record [ID=").append(record.getRecordID())
            .append(", PatientID=").append(record.getPatientID())
            .append(", Diagnosis=").append(record.getDiagnosis())
            .append(", Treatment=").append(record.getTreatment())
            .append(", Date=").append(sdf.format(record.getRecordDate()))
            .append(", AppointmentID=").append(record.getAppointmentID() == 0 ?
                "None" : record.getAppointmentID())
    }
}

```



```

        .append("]");
    } else {
        formatted.append("Record ID=").append(record.getRecordID())
            .append(", Diagnosis=").append(record.getDiagnosis())
            .append(", Date=").append(sdf.format(record.getRecordDate()));
    }
    return formatted.toString() + " [" + config.getFormatDescription() + "];"
}

```

// User Defined Exception

```

public static class MedicalRecordFormatterException extends Exception {
    public MedicalRecordFormatterException(String message) {
        super(message);
    }
}

```

// Getter and Setter for Testing

```

public String getDateFormat() { return dateFormat; }

public void setDateFormat(String dateFormat) {
    this.dateFormat = dateFormat != null && !dateFormat.trim().isEmpty() ?
        dateFormat : "yyyy-MM-dd HH:mm:ss";
}
}

```

//File Name PatientView.java

//Accessing Package comhms.src.hms.patient.view

```
package comhms.src.hms.patient.view;
```

```
import comhms.src.hms.patient.model.Patient;
```

```
import comhms.src.hms.admin.model.Schedule.InvalidScheduleException;
```

```
import comhms.src.hms.auth.controller.UserLoginService;
```

```
import java.util.Scanner;
```

```
public class PatientView {
```

```
    private Scanner scanner;
```

```
    private Patient patient;
```

```
    private UserLoginService loginService; // For authentication
```

```
    private PatientService patientService; // For backend logic
```

```
    // Constructor
```

```
    public PatientView(Patient patient, UserLoginService loginService) {
```

```
        this.scanner = new Scanner(System.in);
```

```
        this.patient = patient;
```

```
        this.loginService = loginService;
```

```
        this.patientService = new PatientService(loginService); // Placeholder
```

```
    }
```

```
    // Overloaded Constructor
```

```
public PatientView(Patient patient, UserLoginService loginService, PatientService
patientService) {
```

```
    this.scanner = new Scanner(System.in);
```

```
    this.patient = patient;
```

```
    this.loginService = loginService;
```

```
    this.patientService = patientService;
```

```
}
```

```
// Inner Class for Menu Options
```

```
private class MenuOption {
```

```
    private String description;
```

```
    private Runnable action;
```

```
MenuOption(String description, Runnable action) {
```

```
    this.description = description;
```

```
    this.action = action;
```

```
}
```

```
void display(int index) {
```

```
    System.out.println(index + ". " + description);
```

```
}
```

```
void execute() {
```

```
    action.run();
```

```
}
```

```
}
```

// Method with Object Reference

```
public void displayPatientMenu(Patient patient) {  
    this.patient = patient; // Update patient reference  
    MenuOption[] options = {  
        new MenuOption("View Appointments", this::handleViewAppointments),  
        new MenuOption("View Medical Record", this::handleViewMedicalRecord),  
        new MenuOption("View Appointment Schedule",  
this::handleViewAppointmentSchedule),  
        new MenuOption("Update Profile", this::handleUpdateProfile),  
        new MenuOption("Logout", () -> System.out.println("Logging out..."))  
    };  
  
    while (true) {  
        System.out.println("\n=== Patient Dashboard ===");  
        System.out.println("Welcome, " + patient.getUsername() + " (Patient)");  
        for (int i = 0; i < options.length; i++) {  
            options[i].display(i + 1);  
        }  
  
        int choice = getValidChoice(options.length);  
        if (choice == options.length) { // Logout  
            options[choice - 1].execute();  
            break;  
        }  
        options[choice - 1].execute();  
    }  
}
```

```
}  
}
```

```
// Method without Object Reference (Static)
```

```
public static String getWelcomeMessage() {  
    return "Welcome to the Patient Dashboard!";  
}
```

```
// Handle View Appointments
```

```
private void handleViewAppointments() {  
    try {  
        String appointments = patientService.viewAppointments(patient);  
        System.out.println(appointments);  
    } catch (Exception e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
}
```

```
// Handle View Medical Record
```

```
private void handleViewMedicalRecord() {  
    try {  
        System.out.println("1. View General Medical Record");  
        System.out.println("2. View Medical Record for Appointment");  
        System.out.println("3. Back");  
        int choice = getValidChoice(3);
```

```

        if (choice == 3) return;

        if (choice == 1) {
            String record = patientService.viewMedicalRecord(patient);
            System.out.println(record);
        } else {
            System.out.print("Enter Appointment ID (e.g., 1): ");
            int appointmentID = Integer.parseInt(scanner.nextLine());
            String record = patientService.viewMedicalRecord(patient, appointmentID);
            System.out.println(record);
        }
    } catch (NumberFormatException e) {
        System.out.println("Error: Please enter a valid number for Appointment ID.");
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

// Handle View Appointment Schedule
private void handleViewAppointmentSchedule() {
    try {
        System.out.print("Enter Appointment ID (e.g., 1): ");
        int appointmentID = Integer.parseInt(scanner.nextLine());
        String schedule = patientService.viewAppointmentSchedule(patient, appointmentID);
        System.out.println(schedule);
    }
}

```

```

    } catch (NumberFormatException e) {
        System.out.println("Error: Please enter a valid number for Appointment ID.");
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

// Handle Update Profile
private void handleUpdateProfile() {
    System.out.println("\n=== Update Profile ===");
    System.out.println("1. Update Medical History");
    System.out.println("2. Back");
    int choice = getValidChoice(2);
    if (choice == 2) return;

    try {
        if (choice == 1) {
            System.out.print("Enter New Medical History (e.g., Diabetes, Hypertension): ");
            String newMedicalHistory = scanner.nextLine();
            patient.setMedicalHistory(newMedicalHistory);
            System.out.println("Medical history updated to: " + newMedicalHistory);
        }
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

```
}
```

```
// Helper Method
```

```
private int getValidChoice(int max) {
```

```
    while (true) {
```

```
        try {
```

```
            System.out.print("Enter choice (1-" + max + "): ");
```

```
            int choice = Integer.parseInt(scanner.nextLine());
```

```
            if (choice >= 1 && choice <= max) {
```

```
                return choice;
```

```
            }
```

```
            System.out.println("Invalid choice! Try again.");
```

```
        } catch (NumberFormatException e) {
```

```
            System.out.println("Please enter a number!");
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```
// Placeholder PatientService
```

```
class PatientService {
```

```
    private comhms.src.hms.auth.controller.UserLoginService loginService;
```

```
    public PatientService(comhms.src.hms.auth.controller.UserLoginService loginService) {
```

```
        this.loginService = loginService;
```



```
}
```

```
public String viewAppointments(Patient patient)

    throws comhms.src.hms.patient.model.Patient.PatientAccessException {

    comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

        1, patient.getPatientID(), 1, "Confirmed");

    return patient.viewAppointments(java.util.Arrays.asList(appointment));

}
```

```
public String viewMedicalRecord(Patient patient)

    throws comhms.src.hms.patient.model.Patient.PatientAccessException {

    return patient.viewMedicalRecord();

}
```

```
public String viewMedicalRecord(Patient patient, int appointmentID)

    throws comhms.src.hms.patient.model.Patient.PatientAccessException {

    comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

        appointmentID, patient.getPatientID(), 1, "Confirmed");

    return patient.viewMedicalRecord(appointment);

}
```

```
public String viewAppointmentSchedule(Patient patient, int appointmentID)

    throws comhms.src.hms.patient.model.Patient.PatientAccessException,
InvalidScheduleException {
```

```
        comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

        appointmentID, patient.getPatientID(), 1, "Confirmed");

        comhms.src.hms.admin.model.Schedule schedule = new
comhms.src.hms.admin.model.Schedule(

        1, 2, "2025-05-04 09:00", "2025-05-04 10:00", "Booked", "Room 101");

        return patient.viewAppointmentSchedule(appointment, java.util.Arrays.asList(schedule));

    }

}
```

//File Name ReceptionService.java

//Accessing Package comhms.src.hms.reception.controller

```
package comhms.src.hms.reception.controller;

import comhms.src.hms.reception.model.Receptionist;
import comhms.src.hms.reception.model.Appointment;
import comhms.src.hms.admin.model.Schedule;
import comhms.src.hms.auth.controller.UserLoginService;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
```

```

public class ReceptionService {

    private List<Appointment> appointmentDatabase; // Simulated Appointment table

    private List<Schedule> scheduleDatabase; // Simulated Schedule table

    private UserLoginService loginService; // For authentication and user data


    // Constructor

    public ReceptionService(UserLoginService loginService) {

        this.appointmentDatabase = new ArrayList<>();

        this.scheduleDatabase = new ArrayList<>();

        this.loginService = loginService;

        initializeDefaultData();

    }


    // Overloaded Constructor

    public ReceptionService(UserLoginService loginService, List<Appointment>
initialAppointments,

                           List<Schedule> initialSchedules) {

        this.appointmentDatabase = initialAppointments != null ? initialAppointments : new
ArrayList<>();

        this.scheduleDatabase = initialSchedules != null ? initialSchedules : new ArrayList<>();

        this.loginService = loginService;

    }


    // Inner Class for Service Operation Result

    private class OperationResult {

        private boolean success;

```

```
private String message;
```

```
OperationResult(boolean success, String message) {
```

```
    this.success = success;
```

```
    this.message = message;
```

```
}
```

```
String getMessage() {
```

```
    return message;
```

```
}
```

```
}
```

```
// Method with Object Reference
```

```
public String bookAppointment(Receptionist receptionist, int appointmentID, String  
patientID,
```

```
int scheduleID) throws Receptionist.ReceptionistAccessException,
```

```
Appointment.AppointmentException {
```

```
    Schedule schedule = findScheduleByID(scheduleID);
```

```
    if (schedule == null) {
```

```
        throw new Receptionist.ReceptionistAccessException("Schedule ID " + scheduleID + "  
not found.");
```

```
    }
```

```
    Appointment appointment = new Appointment(appointmentID, patientID, scheduleID,  
"Pending");
```

```
    appointmentDatabase.add(appointment);
```

```
    String result = appointment.confirmAppointment(schedule);
```

```
        OperationResult opResult = new OperationResult(true, result);  
        return opResult.getMessage();  
    }
```

```
// Method without Object Reference (Static)
```

```
public static String getServiceDescription() {  
    return "ReceptionService: Manages receptionist tasks like appointments and patient  
registration.";  
}
```

```
// Overloaded Method: Register Patient (Basic)
```

```
public String registerPatient(Receptionist receptionist, String patientID, String patientName)  
    throws Receptionist.ReceptionistAccessException {  
    String result = receptionist.registerPatient(patientID, patientName);  
    OperationResult opResult = new OperationResult(true, result);  
    return opResult.getMessage();  
}
```

```
// Overloaded Method: Register Patient (With Contact)
```

```
public String registerPatient(Receptionist receptionist, String patientID, String patientName,  
    String contactInfo) throws Receptionist.ReceptionistAccessException {  
    String result = receptionist.registerPatient(patientID, patientName, contactInfo);  
    OperationResult opResult = new OperationResult(true, result);  
    return opResult.getMessage();  
}
```

```
// Method to View Available Schedules
```

```
public String viewAvailableSchedules(Receptionist receptionist)
    throws Receptionist.ReceptionistAccessException {
    String result = receptionist.viewAvailableSchedules(scheduleDatabase);
    OperationResult opResult = new OperationResult(true, result);
    return opResult.getMessage();
}
```

```
// Overloaded Method: Cancel Appointment (Basic)
```

```
public String cancelAppointment(Receptionist receptionist, int appointmentID)
    throws Appointment.AppointmentException {
    Appointment appointment = findAppointmentByID(appointmentID);
    if (appointment == null) {
        throw new Appointment.AppointmentException("Appointment ID " + appointmentID + "
not found.");
    }
    String result = appointment.cancelAppointment();
    OperationResult opResult = new OperationResult(true, result);
    return opResult.getMessage();
}
```

```
// Overloaded Method: Cancel Appointment (With Reason)
```

```
public String cancelAppointment(Receptionist receptionist, int appointmentID, String reason)
    throws Appointment.AppointmentException {
    Appointment appointment = findAppointmentByID(appointmentID);
    if (appointment == null) {
```

```
        throw new Appointment.AppointmentException("Appointment ID " + appointmentID + " not found.");
```

```
    }
```

```
    String result = appointment.cancelAppointment(reason);
```

```
    OperationResult opResult = new OperationResult(true, result);
```

```
    return opResult.getMessage();
```

```
}
```

```
// User Defined Exception
```

```
public static class ReceptionServiceException extends Exception {
```

```
    public ReceptionServiceException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
// Helper Method to Find Appointment by ID
```

```
private Appointment findAppointmentByID(int appointmentID) {
```

```
    for (Appointment appointment : appointmentDatabase) {
```

```
        if (appointment.getAppointmentID() == appointmentID) {
```

```
            return appointment;
```

```
        }
```

```
    }
```

```
    return null;
```

```
}
```

```
// Helper Method to Find Schedule by ID
```

```

private Schedule findScheduleByID(int scheduleID) {
    for (Schedule schedule : scheduleDatabase) {
        if (schedule.getScheduleID() == scheduleID) {
            return schedule;
        }
    }
    return null;
}

```

// Private Helper Method to Initialize Default Data

```

private void initializeDefaultData() {
    try {
        scheduleDatabase.add(new Schedule(1, 2, "2025-05-04 09:00", "2025-05-04 10:00",
            "Available", "Room 101"));
        scheduleDatabase.add(new Schedule(2, 3, "2025-05-04 08:00", "2025-05-04 16:00",
            "Available", "Ward A"));
    } catch (Schedule.InvalidScheduleException e) {
        // Ignore for now
    }
}

```

@Override

```

public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof ReceptionService)) return false;
}

```



```

        ReceptionService that = (ReceptionService) o;
        return Objects.equals(appointmentDatabase, that.appointmentDatabase) &&
            Objects.equals(scheduleDatabase, that.scheduleDatabase);
    }

    @Override
    public int hashCode() {
        return Objects.hash(appointmentDatabase, scheduleDatabase);
    }
}

```

//File Name Appointment.java

//Accessing Package comhms.src.hms.reception.model

```

package comhms.src.hms.reception.model;

import comhms.src.hms.admin.model.Schedule;
import java.sql.Timestamp;
import java.text.SimpleDateFormat;

public class Appointment {
    private int appointmentID; // E.g., 1
    private String patientID; // E.g., PAT001
    private int scheduleID; // Links to Schedule

```

```

private String status; // E.g., Confirmed, Canceled, Pending

private Timestamp bookingTimestamp; // When the appointment was booked


// Constructor

public Appointment(int appointmentID, String patientID, int scheduleID, String status) {
    this.appointmentID = appointmentID;
    this.patientID = patientID;
    this.scheduleID = scheduleID;
    this.status = validateStatus(status);
    this.bookingTimestamp = new Timestamp(System.currentTimeMillis());
}


// Overloaded Constructor

public Appointment(int appointmentID, String patientID, int scheduleID, String status,
    Timestamp bookingTimestamp) {
    this.appointmentID = appointmentID;
    this.patientID = patientID;
    this.scheduleID = scheduleID;
    this.status = validateStatus(status);
    this.bookingTimestamp = bookingTimestamp != null ? bookingTimestamp :
        new Timestamp(System.currentTimeMillis());
}


// Inner Class for Appointment Details

private class AppointmentDetails {

```

```
private String purpose;
```

```
private String notes;
```

```
AppointmentDetails(String purpose, String notes) {
```

```
    this.purpose = purpose;
```

```
    this.notes = notes;
```

```
}
```

```
String getDetailsSummary() {
```

```
    return "Purpose: " + purpose + ", Notes: " + notes;
```

```
}
```

```
}
```

```
// Method with Object Reference
```

```
public String confirmAppointment(Schedule schedule) throws AppointmentException {
```

```
    if (schedule == null || schedule.getScheduleID() != this.scheduleID) {
```

```
        throw new AppointmentException("Invalid or mismatched schedule for appointment.");
```

```
}
```

```
    if (!"Available".equals(schedule.getStatus())) {
```

```
        throw new AppointmentException("Schedule is not available for booking.");
```

```
}
```

```
    try {
```

```
        schedule.updateStatus("Booked", "Confirmed for appointment " + appointmentID);
```

```
        this.status = "Confirmed";
```

```
        AppointmentDetails details = new AppointmentDetails("Medical Consultation",  
"Confirmed by system");
```

```

        return "Appointment " + appointmentID + " confirmed for patient " + patientID + ". " +
            details.getDetailsSummary();
    } catch (Schedule.InvalidScheduleException e) {
        throw new AppointmentException("Failed to confirm appointment: " + e.getMessage());
    }
}

```

// Method without Object Reference (Static)

```

public static String getAppointmentDescription() {
    return "Appointment: Represents a booked slot for a patient with a staff member.";
}

```

// Overloaded Method: Cancel Appointment (Basic)

```

public String cancelAppointment() throws AppointmentException {
    if ("Canceled".equals(status)) {
        throw new AppointmentException("Appointment is already canceled.");
    }
    this.status = "Canceled";
    AppointmentDetails details = new AppointmentDetails("N/A", "Canceled by system");
    return "Appointment " + appointmentID + " canceled for patient " + patientID + ". " +
        details.getDetailsSummary();
}

```

// Overloaded Method: Cancel Appointment (With Reason)

```

public String cancelAppointment(String reason) throws AppointmentException {

```

```

        if ("Canceled".equals(status)) {
            throw new AppointmentException("Appointment is already canceled.");
        }
        if (reason == null || reason.trim().isEmpty()) {
            throw new AppointmentException("Cancellation reason cannot be empty.");
        }
        this.status = "Canceled";
        AppointmentDetails details = new AppointmentDetails("N/A", "Canceled: " + reason);
        return "Appointment " + appointmentID + " canceled for patient " + patientID + ". " +
            details.getDetailsSummary();
    }

```

// User Defined Exception

```

public static class AppointmentException extends Exception {
    public AppointmentException(String message) {
        super(message);
    }
}

```

// Getters and Setters

```

public int getAppointmentID() { return appointmentID; }
public String getPatientID() { return patientID; }
public int getScheduleID() { return scheduleID; }
public String getStatus() { return status; }
public Timestamp getBookingTimestamp() { return bookingTimestamp; }

```

```
public void setStatus(String status) { this.status = validateStatus(status); }
```

```
// Helper Method to Validate Status
```

```
private String validateStatus(String status) {  
    if (status == null || !(status.equals("Pending") || status.equals("Confirmed") ||  
        status.equals("Canceled"))) {  
        return "Pending"; // Default status  
    }  
    return status;  
}
```

```
// Override toString
```

```
@Override
```

```
public String toString() {  
    AppointmentDetails details = new AppointmentDetails("Medical Consultation", "N/A");  
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");  
    return "Appointment [ID=" + appointmentID + ", PatientID=" + patientID +  
        ", ScheduleID=" + scheduleID + ", Status=" + status +  
        ", BookingTimestamp=" + sdf.format(bookingTimestamp) +  
        ", Details=" + details.getDetailsSummary() + "];"  
}
```

//File Name Receptionist.java

//Accessing Package comhms.src.hms.reception.model

```
package comhms.src.hms.reception.model;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import java.util.List;
```

```
public class Receptionist extends UserLogin {
```

```
    private String deskNumber; // E.g., DSK001
```

```
    // Constructor
```

```
    public Receptionist(int userID, String username, String password, String name, int age,
```

```
        String deskNumber) {
```

```
        super(userID, username, password, "Receptionist", "Receptionist", name, age, deskNumber, deskNumber);
```

```
        this.deskNumber = deskNumber;
```

```
    }
```

```
    // Overloaded Constructor
```

```
    public Receptionist(int userID, String username, String password, String name, int age,
```

```
        String deskNumber, String securityQuestion, String securityAnswer) {
```

```
        super(userID, username, password, "Receptionist", "Receptionist", name, age,
```

```
            securityQuestion, securityAnswer);
```

```
        this.deskNumber = deskNumber;
    }
}
```

```
// Inner Class for Reception Task
```

```
private class ReceptionTask {
    private String taskType;
    private String details;
```

```
    ReceptionTask(String taskType, String details) {
        this.taskType = taskType;
        this.details = details;
    }
}
```

```
    String getTaskSummary() {
        return "Task: " + taskType + ", Details: " + details;
    }
}
```

```
// Method with Object Reference
```

```
    public String bookAppointment(Schedule schedule, String patientID) throws
ReceptionistAccessException {
        if (schedule == null || patientID == null || patientID.trim().isEmpty()) {
            throw new ReceptionistAccessException("Schedule and patient ID cannot be null or
empty.");
        }

        ReceptionTask task = new ReceptionTask("Appointment Booking", "Patient: " + patientID);
```



```

    try {
        String result = schedule.updateStatus("Booked", "Booked by receptionist for patient " +
patientID);

        return result + "\n" + task.getTaskSummary();
    } catch (Schedule.InvalidScheduleException e) {

        throw new ReceptionistAccessException("Failed to book appointment: " +
e.getMessage());
    }
}

```

// Method without Object Reference (Static)

```

public static String getReceptionistRoleDescription() {
    return "Receptionist: Manages appointments and patient registrations.";
}

```

// Overloaded Method: Register Patient (Basic)

```

public String registerPatient(String patientID, String patientName) throws
ReceptionistAccessException {
    if (patientID == null || patientName == null || patientID.trim().isEmpty() ||
patientName.trim().isEmpty()) {
        throw new ReceptionistAccessException("Patient ID and name cannot be null or
empty.");
    }

    ReceptionTask task = new ReceptionTask("Patient Registration", "ID: " + patientID + ",
Name: " + patientName);

    return "Patient registered: " + patientName + " (" + patientID + "). " +
task.getTaskSummary();
}

```

```

// Overloaded Method: Register Patient (With Contact)

public String registerPatient(String patientID, String patientName, String contactInfo)
    throws ReceptionistAccessException {
    if (patientID == null || patientName == null || contactInfo == null ||
        patientID.trim().isEmpty() || patientName.trim().isEmpty() ||
        contactInfo.trim().isEmpty()) {
        throw new ReceptionistAccessException("Patient ID, name, and contact cannot be null or
        empty.");
    }

    ReceptionTask task = new ReceptionTask("Patient Registration",
        "ID: " + patientID + ", Name: " + patientName + ", Contact: " +
        contactInfo);

    return "Patient registered: " + patientName + " (" + patientID + "). " +
        task.getTaskSummary();
}

```

```

// Method to View Available Schedules

public String viewAvailableSchedules(List<Schedule> schedules) throws
ReceptionistAccessException {
    if (schedules == null) {
        throw new ReceptionistAccessException("Schedule list cannot be null.");
    }

    StringBuilder result = new StringBuilder("Available Schedules:\n");

    boolean hasAvailable = false;

    for (Schedule schedule : schedules) {
        if ("Available".equals(schedule.getStatus())) {

```

```

        result.append(schedule.toString()).append("\n");
        hasAvailable = true;
    }
}
if (!hasAvailable) {
    result.append("No available schedules.");
}
return result.toString();
}

```

// User Defined Exception

```

public static class ReceptionistAccessException extends Exception {
    public ReceptionistAccessException(String message) {
        super(message);
    }
}

```

// Getters and Setters

```

public String getDeskNumber() { return deskNumber; }
public void setDeskNumber(String deskNumber) { this.deskNumber = deskNumber; }

```

// Override toString

@Override

```

public String toString() {
    ReceptionTask task = new ReceptionTask("N/A", "N/A");

```

```

        return "Receptionist [Username=" + getUsername() + ", DeskNumber=" + deskNumber +
            ", TaskSummary=" + task.getTaskSummary() + "]\n";
    }
}

```

//File Name ReceptionistTaskLogger.java

//Accessing Package comhms.src.hms.reception.util

```

package comhms.src.hms.reception.util;

```

```

import java.sql.Timestamp;

```

```

import java.text.SimpleDateFormat;

```

```

import java.util.ArrayList;

```

```

import java.util.List;

```

```

public class ReceptionTaskLogger {

```

```

    private List<LogEntry> logDatabase; // Simulated log storage

```

```

    // Constructor

```

```

    public ReceptionTaskLogger() {

```

```

        this.logDatabase = new ArrayList<>();

```

```

    }

```

```

    // Overloaded Constructor

```

```

public ReceptionTaskLogger(List<LogEntry> initialLogs) {
    this.logDatabase = initialLogs != null ? initialLogs : new ArrayList<>();
}

```

// Inner Class for Log Entry

```

public class LogEntry {

```

```

    private int logID;

```

```

    private String taskType; // E.g., "Book Appointment", "Register Patient"

```

```

    private int receptionistID;

```

```

    private String details; // E.g., "Appointment ID: 1, Patient: PAT001"

```

```

    private Timestamp timestamp;

```

```

    public LogEntry(int logID, String taskType, int receptionistID, String details) {

```

```

        this.logID = logID;

```

```

        this.taskType = taskType;

```

```

        this.receptionistID = receptionistID;

```

```

        this.details = details;

```

```

        this.timestamp = new Timestamp(System.currentTimeMillis());

```

```

    }

```

```

    public String getLogDetails() {

```

```

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");

```

```

        return "Log [ID=" + logID + ", Task=" + taskType + ", ReceptionistID=" + receptionistID

```

+

```

            ", Details=" + details + ", Timestamp=" + sdf.format(timestamp) + "];"

```

```

    }

```

```
}
```

```
// Method with Object Reference
```

```
public void logTask(String taskType, int receptionistID, String details)
    throws ReceptionTaskLoggerException {
    if (taskType == null || taskType.trim().isEmpty() || details == null || details.trim().isEmpty())
    {
        throw new ReceptionTaskLoggerException("Task type and details cannot be null or
empty.");
    }

    int logID = logDatabase.size() + 1; // Simple incremental ID

    LogEntry entry = new LogEntry(logID, taskType, receptionistID, details);

    logDatabase.add(entry);
}
```

```
// Method without Object Reference (Static)
```

```
public static String getLoggerDescription() {
    return "ReceptionTaskLogger: Logs receptionist tasks with timestamps and details.";
}
```

```
// Overloaded Method: Retrieve Logs (All)
```

```
public String getLogs() {
    StringBuilder result = new StringBuilder("Reception Task Logs:\n");

    if (logDatabase.isEmpty()) {
        result.append("No logs found.");
    } else {
```

```

        for (LogEntry entry : logDatabase) {
            result.append(entry.getLogDetails()).append("\n");
        }
    }
    return result.toString();
}

// Overloaded Method: Retrieve Logs (By Receptionist ID)
public String getLogs(int receptionistID) {
    StringBuilder result = new StringBuilder("Reception Task Logs for Receptionist " +
        receptionistID + ":\n");

    boolean hasLogs = false;
    for (LogEntry entry : logDatabase) {
        if (entry.receptionistID == receptionistID) {
            result.append(entry.getLogDetails()).append("\n");
            hasLogs = true;
        }
    }

    if (!hasLogs) {
        result.append("No logs found for Receptionist " + receptionistID + ".");
    }

    return result.toString();
}

// User Defined Exception

```

```

public static class ReceptionTaskLoggerException extends Exception {
    public ReceptionTaskLoggerException(String message) {
        super(message);
    }
}

// Getter for Testing
public List<LogEntry> getLogDatabase() {
    return new ArrayList<>(logDatabase); // Return a copy to protect encapsulation
}
}

```

//File Name ReceptionistView.java

//Accessing Package comhms.src.hms.reception.view

```

package comhms.src.hms.reception.view;

import comhms.src.hms.reception.model.Receptionist;
import comhms.src.hms.admin.model.Schedule.InvalidScheduleException;
import comhms.src.hms.auth.controller.UserLoginService;
import java.util.Scanner;

public class ReceptionView {
    private Scanner scanner;

```



```
private Receptionist receptionist;

private UserLoginService loginService; // For authentication

private ReceptionService receptionService; // For backend logic


// Constructor

public ReceptionView(Receptionist receptionist, UserLoginService loginService) {

    this.scanner = new Scanner(System.in);

    this.receptionist = receptionist;

    this.loginService = loginService;

    this.receptionService = new ReceptionService(loginService); // Placeholder
}


// Overloaded Constructor

public ReceptionView(Receptionist receptionist, UserLoginService loginService,
                    ReceptionService receptionService) {

    this.scanner = new Scanner(System.in);

    this.receptionist = receptionist;

    this.loginService = loginService;

    this.receptionService = receptionService;
}


// Inner Class for Menu Options

private class MenuOption {

    private String description;

    private Runnable action;
```

```

MenuOption(String description, Runnable action) {
    this.description = description;
    this.action = action;
}

void display(int index) {
    System.out.println(index + ". " + description);
}

void execute() {
    action.run();
}
}

// Method with Object Reference
public void displayReceptionMenu(Receptionist receptionist) {
    this.receptionist = receptionist; // Update receptionist reference
    MenuOption[] options = {
        new MenuOption("Book Appointment", this::handleBookAppointment),
        new MenuOption("Register Patient", this::handleRegisterPatient),
        new MenuOption("View Available Schedules", this::handleViewSchedules),
        new MenuOption("Cancel Appointment", this::handleCancelAppointment),
        new MenuOption("Update Profile", this::handleUpdateProfile),
        new MenuOption("Logout", () -> System.out.println("Logging out..."))
    }
}

```

```
};
```

```
while (true) {
```

```
    System.out.println("\n=== Receptionist Dashboard ===");
```

```
    System.out.println("Welcome, " + receptionist.getUsername() + " (Receptionist)");
```

```
    for (int i = 0; i < options.length; i++) {
```

```
        options[i].display(i + 1);
```

```
    }
```

```
    int choice = getValidChoice(options.length);
```

```
    if (choice == options.length) { // Logout
```

```
        options[choice - 1].execute();
```

```
        break;
```

```
    }
```

```
    options[choice - 1].execute();
```

```
}
```

```
}
```

```
// Method without Object Reference (Static)
```

```
public static String getWelcomeMessage() {
```

```
    return "Welcome to the Receptionist Dashboard!";
```

```
}
```

```
// Handle Book Appointment
```

```
private void handleBookAppointment() {
```

```

try {
    System.out.print("Enter Appointment ID (e.g., 1): ");
    int appointmentID = Integer.parseInt(scanner.nextLine());
    System.out.print("Enter Patient ID (e.g., PAT001): ");
    String patientID = scanner.nextLine();
    System.out.print("Enter Schedule ID (e.g., 1): ");
    int scheduleID = Integer.parseInt(scanner.nextLine());

    String result = receptionService.bookAppointment(receptionist, appointmentID,
patientID, scheduleID);

    System.out.println(result);
} catch (NumberFormatException e) {
    System.out.println("Error: Please enter valid numbers for IDs.");
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}

```

// Handle Register Patient

```

private void handleRegisterPatient() {
    try {
        System.out.print("Enter Patient ID (e.g., PAT002): ");
        String patientID = scanner.nextLine();
        System.out.print("Enter Patient Name: ");
        String patientName = scanner.nextLine();
        System.out.print("Enter Contact Info (or press Enter to skip): ");
        String contactInfo = scanner.nextLine();
    }
}

```

```

        String result = contactInfo.isEmpty() ?
            receptionService.registerPatient(receptionist, patientID, patientName) :
            receptionService.registerPatient(receptionist, patientID, patientName,
contactInfo);

        System.out.println(result);
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

// Handle View Available Schedules

```

private void handleViewSchedules() {
    try {
        String schedules = receptionService.viewAvailableSchedules(receptionist);
        System.out.println(schedules);
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

// Handle Cancel Appointment

```

private void handleCancelAppointment() {
    try {
        System.out.print("Enter Appointment ID (e.g., 1): ");
        int appointmentID = Integer.parseInt(scanner.nextLine());
        System.out.print("Enter Cancellation Reason (or press Enter to skip): ");
    }
}

```

```

String reason = scanner.nextLine();

String result = reason.isEmpty() ?

    receptionService.cancelAppointment(receptionist, appointmentID) :

    receptionService.cancelAppointment(receptionist, appointmentID, reason);

System.out.println(result);
} catch (NumberFormatException e) {

    System.out.println("Error: Please enter a valid number for Appointment ID.");
} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());
}
}

```

// Handle Update Profile

```

private void handleUpdateProfile() {

    System.out.println("\n=== Update Profile ===");

    System.out.println("1. Update Desk Number");

    System.out.println("2. Back");

    int choice = getValidChoice(2);

    if (choice == 2) return;

    try {

        if (choice == 1) {

            System.out.print("Enter New Desk Number (e.g., DSK002): ");

            String newDeskNumber = scanner.nextLine();

            receptionist.setDeskNumber(newDeskNumber);

```

```

        System.out.println("Desk number updated to: " + newDeskNumber);
    }
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}

// Helper Method
private int getValidChoice(int max) {
    while (true) {
        try {
            System.out.print("Enter choice (1-" + max + "): ");
            int choice = Integer.parseInt(scanner.nextLine());
            if (choice >= 1 && choice <= max) {
                return choice;
            }
            System.out.println("Invalid choice! Try again.");
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number!");
        }
    }
}

// Placeholder ReceptionService

```

```

class ReceptionService {

    private comhms.src.hms.auth.controller.UserLoginService loginService;

    public ReceptionService(comhms.src.hms.auth.controller.UserLoginService loginService) {

        this.loginService = loginService;

    }

    public String bookAppointment(Receptionist receptionist, int appointmentID, String
patientID, int scheduleID)

        throws comhms.src.hms.reception.model.Receptionist.ReceptionistAccessException,

            comhms.src.hms.reception.model.Appointment.AppointmentException,
InvalidScheduleException {

        comhms.src.hms.admin.model.Schedule schedule = new
comhms.src.hms.admin.model.Schedule(

            scheduleID, 2, "2025-05-04 09:00", "2025-05-04 10:00", "Available", "Room 101");

        comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

            appointmentID, patientID, scheduleID, "Pending");

        return appointment.confirmAppointment(schedule);

    }

    public String registerPatient(Receptionist receptionist, String patientID, String patientName)

        throws comhms.src.hms.reception.model.Receptionist.ReceptionistAccessException {

        return receptionist.registerPatient(patientID, patientName);

    }

    public String registerPatient(Receptionist receptionist, String patientID, String patientName,

```



```

        String contactInfo)

        throws comhms.src.hms.reception.model.Receptionist.ReceptionistAccessException {
    return receptionist.registerPatient(patientID, patientName, contactInfo);
}

public String viewAvailableSchedules(Receptionist receptionist)

    throws comhms.src.hms.reception.model.Receptionist.ReceptionistAccessException,
InvalidScheduleException {

    return receptionist.viewAvailableSchedules(java.util.Arrays.asList(

        new comhms.src.hms.admin.model.Schedule(1, 2, "2025-05-04 09:00", "2025-05-04
10:00",

            "Available", "Room 101")));

}

public String cancelAppointment(Receptionist receptionist, int appointmentID)

    throws comhms.src.hms.reception.model.Appointment.AppointmentException {

    comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

        appointmentID, "PAT001", 1, "Confirmed");

    return appointment.cancelAppointment();

}

public String cancelAppointment(Receptionist receptionist, int appointmentID, String reason)

    throws comhms.src.hms.reception.model.Appointment.AppointmentException {

    comhms.src.hms.reception.model.Appointment appointment = new
comhms.src.hms.reception.model.Appointment(

        appointmentID, "PAT001", 1, "Confirmed");

```

```
        return appointment.cancelAppointment(reason);
    }
}
```

//File Name StaffService.java

//Accessing Package comhms.src.hms.staff.controller

```
package comhms.src.hms.staff.controller;

import comhms.src.hms.staff.model.Staff;
import comhms.src.hms.staff.model.ClinicalStaff;
import comhms.src.hms.staff.model.Doctor;
import comhms.src.hms.staff.model.Nurse;
import comhms.src.hms.admin.model.Schedule;
import comhms.src.hms.auth.controller.UserLoginService;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
```

```

public class StaffService {

    private List<Schedule> scheduleDatabase; // Simulated Schedule table

    private UserLoginService loginService; // For authentication and user data


    // Constructor

    public StaffService(UserLoginService loginService) {

        this.scheduleDatabase = new ArrayList<>();

        this.loginService = loginService;

        initializeDefaultSchedules();

    }


    // Overloaded Constructor

    public StaffService(UserLoginService loginService, List<Schedule> initialSchedules) {

        this.scheduleDatabase = initialSchedules != null ? initialSchedules : new ArrayList<>();

        this.loginService = loginService;

    }


    // Inner Class for Service Operation Result

    private class OperationResult {

        private boolean success;

        private String message;


        OperationResult(boolean success, String message) {

            this.success = success;

            this.message = message;

```

```
}
```

```
String getMessage() {
```

```
    return message;
```

```
}
```

```
}
```

```
// Method with Object Reference
```

```
public OperationResult performClinicalTask(Staff staff, String patientID, String taskDetails)
```

```
    throws Staff.StaffAccessException {
```

```
    if (!(staff instanceof ClinicalStaff)) {
```

```
        throw new Staff.StaffAccessException("Only clinical staff can perform clinical tasks.");
```

```
    }
```

```
    String result = ((ClinicalStaff) staff).performClinicalTask(patientID, taskDetails);
```

```
    return new OperationResult(true, result);
```

```
}
```

```
// Method without Object Reference (Static)
```

```
public static String getServiceDescription() {
```

```
    return "StaffService: Manages staff tasks like schedules, clinical tasks, and reports.";
```

```
}
```

```
// Overloaded Method: Generate Report (Doctor)
```

```
public String generateReport(Doctor doctor, String patientID) throws  
Staff.StaffAccessException {
```

```
    return doctor.generateConsultationReport(scheduleDatabase, patientID);
```

```
}
```

```
// Overloaded Method: Generate Report (Nurse)
```

```
public String generateReport(Nurse nurse, String shiftDate) throws  
Staff.StaffAccessException {
```

```
    return nurse.generateShiftReport(scheduleDatabase, shiftDate);
```

```
}
```

```
// Method to Get Schedules
```

```
public List<Schedule> getSchedules() {
```

```
    return new ArrayList<>(scheduleDatabase);
```

```
}
```

```
// Method to Update Profile (Specialization)
```

```
public String updateProfile(Staff staff, String newSpecialization) throws  
Staff.StaffAccessException {
```

```
    return staff.updateProfile(newSpecialization);
```

```
}
```

```
// Method to Update Profile (Specialization and Department)
```

```
public String updateProfile(Staff staff, String newSpecialization, String newDepartment)
```

```
    throws Staff.StaffAccessException {
```

```
    return staff.updateProfile(newSpecialization, newDepartment);
```

```
}
```

```
// Method to Update Credentials (Clinical Staff)
```

```
public String updateCredentials(ClinicalStaff staff, String newLicenseNumber)
    throws Staff.StaffAccessException {
    return staff.updateCredentials(newLicenseNumber);
}
```

// Method to Update Consultation Fee (Doctor)

```
public String updateConsultationFee(Doctor doctor, double newFee, String reason)
    throws Staff.StaffAccessException {
    return reason == null || reason.isEmpty() ? doctor.updateConsultationFee(newFee) :
        doctor.updateConsultationFee(newFee, reason);
}
```

// Method to Update Shift Type (Nurse)

```
public String updateShiftType(Nurse nurse, String newShiftType, String reason)
    throws Staff.StaffAccessException {
    return reason == null || reason.isEmpty() ? nurse.updateShiftType(newShiftType) :
        nurse.updateShiftType(newShiftType, reason);
}
```

// User Defined Exception

```
public static class StaffServiceException extends Exception {
    public StaffServiceException(String message) {
        super(message);
    }
}
```

```

// Private Helper Method

private void initializeDefaultSchedules() {

    try {

        scheduleDatabase.add(new Schedule(1, 2, "2025-05-04 09:00", "2025-05-04 10:00",
"Booked", "Room 101"));

        scheduleDatabase.add(new Schedule(2, 3, "2025-05-04 08:00", "2025-05-04 16:00",
"Booked", "Ward A"));

    } catch (Schedule.InvalidScheduleException e) {

        // Ignore for now

    }

}

```

@Override

```

public boolean equals(Object o) {

    if (this == o) return true;

    if (!(o instanceof StaffService)) return false;

    StaffService that = (StaffService) o;

    return Objects.equals(scheduleDatabase, that.scheduleDatabase);

}

```

@Override

```

public int hashCode() {

    return Objects.hash(scheduleDatabase);

}

}

```

//File Name ClinicalStaff.java

//Accessing Package comhms.src.hms.staff.model

```
package comhms.src.hms.staff.model;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import java.util.List;
```

```
public abstract class ClinicalStaff extends Staff {
```

```
    private String licenseNumber; // E.g., LIC12345
```

```
    // Constructor
```

```
    public ClinicalStaff(int userID, String username, String password, String subRole, String  
name, int age,
```

```
        String employeeID, String specialization, String department, String  
licenseNumber) {
```

```
        super(userID, username, password, subRole, name, age, employeeID, specialization,  
department);
```

```
        this.licenseNumber = licenseNumber;
```

```
    }
```

```
    // Overloaded Constructor
```

```
    public ClinicalStaff(int userID, String username, String password, String subRole, String  
name, int age,
```

```
        String employeeID, String specialization, String department, String  
licenseNumber,
```

```
        String securityQuestion, String securityAnswer) {
```



```
        super(userID, username, password, subRole, name, age, employeeID, specialization,  
department,
```

```
        securityQuestion, securityAnswer);
```

```
        this.licenseNumber = licenseNumber;
```

```
    }
```

```
// Inner Class for Clinical Credentials
```

```
private class ClinicalCredentials {
```

```
    private String credentialType;
```

```
    private String issuer;
```

```
ClinicalCredentials(String credentialType, String issuer) {
```

```
    this.credentialType = credentialType;
```

```
    this.issuer = issuer;
```

```
}
```

```
String getCredentialSummary() {
```

```
    return "Credential: " + credentialType + ", Issued by: " + issuer;
```

```
}
```

```
}
```

```
// Abstract Method
```

```
public abstract String performClinicalTask(String patientID, String taskDetails) throws  
Staff.StaffAccessException;
```

```
// Method with Object Reference
```

```

public String validateLicense(List<Schedule> schedules) throws Staff.StaffAccessException {
    if (licenseNumber == null || licenseNumber.trim().isEmpty()) {
        throw new Staff.StaffAccessException("Invalid license number.");
    }

    ClinicalCredentials creds = new ClinicalCredentials("Medical License", "State Board");

    StringBuilder result = new StringBuilder("License Valid: " + licenseNumber + ", " +
creds.getCredentialSummary() + "\n");

    result.append("Assigned Schedules:\n");

    boolean hasSchedules = false;

    for (Schedule schedule : schedules) {
        if (schedule.getStaffID() == getUserID()) {
            result.append(schedule.toString()).append("\n");

            hasSchedules = true;
        }
    }

    if (!hasSchedules) {
        result.append("No schedules assigned.");
    }

    return result.toString();
}

```

// Method without Object Reference (Static)

```

public static String getClinicalStaffRoleDescription() {
    return "ClinicalStaff: Performs clinical tasks such as prescribing or recording vitals.";
}

```

// Overloaded Method: Update Credentials (License Number)

```
public String updateCredentials(String newLicenseNumber) throws
Staff.StaffAccessException {

    if (newLicenseNumber == null || newLicenseNumber.trim().isEmpty()) {

        throw new Staff.StaffAccessException("License number cannot be empty.");

    }

    this.licenseNumber = newLicenseNumber;

    ClinicalCredentials creds = new ClinicalCredentials("Medical License", "State Board");

    return "Credentials updated: " + creds.getCredentialSummary() + ", License: " +
newLicenseNumber;

}
```

// Overloaded Method: Update Credentials (License Number and Issuer)

```
public String updateCredentials(String newLicenseNumber, String issuer) throws
Staff.StaffAccessException {

    if (newLicenseNumber == null || newLicenseNumber.trim().isEmpty() ||

        issuer == null || issuer.trim().isEmpty()) {

        throw new Staff.StaffAccessException("License number and issuer cannot be empty.");

    }

    this.licenseNumber = newLicenseNumber;

    ClinicalCredentials creds = new ClinicalCredentials("Medical License", issuer);

    return "Credentials updated: " + creds.getCredentialSummary() + ", License: " +
newLicenseNumber;

}
```

// Getters and Setters

```
public String getLicenseNumber() { return licenseNumber; }
```

```

    public void setLicenseNumber(String licenseNumber) { this.licenseNumber = licenseNumber;
}

// Override toString

@Override

public String toString() {

    ClinicalCredentials creds = new ClinicalCredentials("Medical License", "State Board");

    return "ClinicalStaff [Username=" + getUsername() + ", EmployeeID=" + getEmployeeID()
+
        ", Specialization=" + getSpecialization() + ", Department=" + getDepartment() +
        ", LicenseNumber=" + licenseNumber + ", Credentials=" +
creds.getCredentialSummary() + "]\n";

}

}

```

//File Name Doctor.java

//Accessing Package comhms.src.hms.staff.model

```
package comhms.src.hms.staff.model;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import java.util.List;
```

```
public class Doctor extends ClinicalStaff {  
    private double consultationFee; // E.g., 100.0 (in USD)  
  
    // Constructor  
    public Doctor(int userID, String username, String password, String name, int age,  
        String employeeID, String specialization, String department, String licenseNumber,  
        double consultationFee) {  
        super(userID, username, password, "Doctor", name, age, employeeID, specialization,  
department, licenseNumber);  
        this.consultationFee = consultationFee;  
    }  
  
    // Overloaded Constructor  
    public Doctor(int userID, String username, String password, String name, int age,  
        String employeeID, String specialization, String department, String licenseNumber,  
        double consultationFee, String securityQuestion, String securityAnswer) {  
        super(userID, username, password, "Doctor", name, age, employeeID, specialization,  
department,  
        licenseNumber, securityQuestion, securityAnswer);  
        this.consultationFee = consultationFee;  
    }  
  
    // Inner Class for Consultation Details  
    private class ConsultationDetails {  
        private String patientID;  
        private String prescription;
```

```

    ConsultationDetails(String patientID, String prescription) {
        this.patientID = patientID;
        this.prescription = prescription;
    }

    String getConsultationSummary() {
        return "Patient: " + patientID + ", Prescription: " + prescription + ", Fee: $" +
consultationFee;
    }
}

// Implement Abstract Method

@Override

    public String performClinicalTask(String patientID, String taskDetails) throws
Staff.StaffAccessException {
        if (patientID == null || taskDetails == null || patientID.trim().isEmpty() ||
taskDetails.trim().isEmpty()) {
            throw new Staff.StaffAccessException("Patient ID and task details cannot be null or
empty.");
        }

        ConsultationDetails consultation = new ConsultationDetails(patientID, taskDetails);
        return "Doctor prescribed: " + taskDetails + " for patient " + patientID + ". " +
            consultation.getConsultationSummary();
    }

// Method with Object Reference

```

```

public String generateConsultationReport(List<Schedule> schedules, String patientID)

    throws Staff.StaffAccessException {

    if (patientID == null || patientID.trim().isEmpty()) {

        throw new Staff.StaffAccessException("Patient ID cannot be null or empty.");

    }

    StringBuilder report = new StringBuilder("Consultation Report for Patient: " + patientID +
"\n");

    report.append("Doctor: ").append(getUsername()).append("
")
.append(getSpecialization()).append("
");

    report.append("Consultation Fee: $").append(consultationFee).append("
");

    report.append("Assigned Schedules:
");

    boolean hasSchedules = false;

    for (Schedule schedule : schedules) {

        if (schedule.getStaffID() == getUserID()) {

            report.append(schedule.toString()).append("
");

            hasSchedules = true;

        }

    }

    if (!hasSchedules) {

        report.append("No schedules assigned.");

    }

    return report.toString();

}

```

// Method without Object Reference (Static)

```

public static String getDoctorRoleDescription() {

```

```
    return "Doctor: Diagnoses patients, prescribes medications, and generates consultation reports.";
```

```
}
```

```
// Overloaded Method: Update Consultation Fee
```

```
public String updateConsultationFee(double newFee) throws Staff.StaffAccessException {
```

```
    if (newFee < 0) {
```

```
        throw new Staff.StaffAccessException("Consultation fee cannot be negative.");
```

```
    }
```

```
    this.consultationFee = newFee;
```

```
    ConsultationDetails consultation = new ConsultationDetails("N/A", "N/A");
```

```
    return "Consultation fee updated: " + consultation.getConsultationSummary();
```

```
}
```

```
// Overloaded Method: Update Consultation Fee with Reason
```

```
public String updateConsultationFee(double newFee, String reason) throws  
Staff.StaffAccessException {
```

```
    if (newFee < 0) {
```

```
        throw new Staff.StaffAccessException("Consultation fee cannot be negative.");
```

```
    }
```

```
    if (reason == null || reason.trim().isEmpty()) {
```

```
        throw new Staff.StaffAccessException("Reason cannot be empty.");
```

```
    }
```

```
    this.consultationFee = newFee;
```

```
    ConsultationDetails consultation = new ConsultationDetails("N/A", "N/A");
```



```
        return "Consultation fee updated: " + consultation.getConsultationSummary() + " (Reason: " + reason + ")";
```

```
    }
```

```
// Getters and Setters
```

```
public double getConsultationFee() { return consultationFee; }
```

```
public void setConsultationFee(double consultationFee) { this.consultationFee = consultationFee; }
```

```
// Override toString
```

```
@Override
```

```
public String toString() {
```

```
    ConsultationDetails consultation = new ConsultationDetails("N/A", "N/A");
```

```
    return "Doctor [Username=" + getUsername() + ", EmployeeID=" + getEmployeeID() +
```

```
        ", Specialization=" + getSpecialization() + ", Department=" + getDepartment() +
```

```
        ", LicenseNumber=" + getLicenseNumber() + ", ConsultationFee=$" + consultationFee
```

```
+
```

```
        ", Details=" + consultation.getConsultationSummary() + "];
```

```
    }
```

```
}
```

//File Name Nurse.java

//Accessing Package comhms.src.hms.staff.model

```
package comhms.src.hms.staff.model;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import java.util.List;
```

```
public class Nurse extends ClinicalStaff {
```

```
    private String shiftType; // E.g., "Day", "Night"
```

```
    // Constructor
```

```
    public Nurse(int userID, String username, String password, String name, int age,
                  String employeeID, String specialization, String department, String licenseNumber,
                  String shiftType) {
        super(userID, username, password, "Nurse", name, age, employeeID, specialization,
              department, licenseNumber);
        this.shiftType = shiftType;
    }
```

```
    // Overloaded Constructor
```

```
    public Nurse(int userID, String username, String password, String name, int age,
                  String employeeID, String specialization, String department, String licenseNumber,
                  String shiftType, String securityQuestion, String securityAnswer) {
        super(userID, username, password, "Nurse", name, age, employeeID, specialization,
              department,
```

```

        licenseNumber, securityQuestion, securityAnswer);
    this.shiftType = shiftType;
}

// Inner Class for Nursing Task Details
private class NursingTaskDetails {

    private String patientID;

    private String task;

    NursingTaskDetails(String patientID, String task) {

        this.patientID = patientID;

        this.task = task;

    }

    String getTaskSummary() {

        return "Patient: " + patientID + ", Task: " + task + ", Shift: " + shiftType;

    }

}

// Implement Abstract Method

@Override

    public String performClinicalTask(String patientID, String taskDetails) throws
Staff.StaffAccessException {

        if (patientID == null || taskDetails == null || patientID.trim().isEmpty() ||
taskDetails.trim().isEmpty()) {

```

```
        throw new Staff.StaffAccessException("Patient ID and task details cannot be null or empty.");
```

```
    }
```

```
    NursingTaskDetails task = new NursingTaskDetails(patientID, taskDetails);
```

```
    return "Nurse performed: " + taskDetails + " for patient " + patientID + ". " + task.getTaskSummary();
```

```
}
```

```
// Method with Object Reference
```

```
public String generateShiftReport(List<Schedule> schedules, String shiftDate)
```

```
    throws Staff.StaffAccessException {
```

```
    if (shiftDate == null || shiftDate.trim().isEmpty()) {
```

```
        throw new Staff.StaffAccessException("Shift date cannot be null or empty.");
```

```
    }
```

```
    StringBuilder report = new StringBuilder("Shift Report for Nurse: " + getUsername() + " on " + shiftDate + "\n");
```

```
    report.append("Shift Type: ").append(shiftType).append("\n");
```

```
    report.append("Specialization: ").append(getSpecialization()).append("\n");
```

```
    report.append("Assigned Schedules:\n");
```

```
    boolean hasSchedules = false;
```

```
    for (Schedule schedule : schedules) {
```

```
        if (schedule.getStaffID() == getUserID()) {
```

```
            report.append(schedule.toString()).append("\n");
```

```
            hasSchedules = true;
```

```
        }
```

```
    }
```

```
    if (!hasSchedules) {  
        report.append("No schedules assigned.");  
    }  
    return report.toString();  
}
```

// Method without Object Reference (Static)

```
public static String getNurseRoleDescription() {  
    return "Nurse: Records patient vitals, administers medications, and generates shift reports.";  
}
```

// Overloaded Method: Update Shift Type

```
public String updateShiftType(String newShiftType) throws Staff.StaffAccessException {  
    if (newShiftType == null || newShiftType.trim().isEmpty()) {  
        throw new Staff.StaffAccessException("Shift type cannot be empty.");  
    }  
    this.shiftType = newShiftType;  
    NursingTaskDetails task = new NursingTaskDetails("N/A", "N/A");  
    return "Shift type updated: " + task.getTaskSummary();  
}
```

// Overloaded Method: Update Shift Type with Reason

```
public String updateShiftType(String newShiftType, String reason) throws  
Staff.StaffAccessException {  
    if (newShiftType == null || newShiftType.trim().isEmpty()) {  
        throw new Staff.StaffAccessException("Shift type cannot be empty.");  
    }  
}
```

```

    }

    if (reason == null || reason.trim().isEmpty()) {
        throw new Staff.StaffAccessException("Reason cannot be empty.");
    }

    this.shiftType = newShiftType;

    NursingTaskDetails task = new NursingTaskDetails("N/A", "N/A");

    return "Shift type updated: " + task.getTaskSummary() + " (Reason: " + reason + ")";
}

// Getters and Setters

public String getShiftType() { return shiftType; }

public void setShiftType(String shiftType) { this.shiftType = shiftType; }

// Override toString

@Override

public String toString() {

    NursingTaskDetails task = new NursingTaskDetails("N/A", "N/A");

    return "Nurse [Username=" + getUsername() + ", EmployeeID=" + getEmployeeID() +
        ", Specialization=" + getSpecialization() + ", Department=" + getDepartment() +
        ", LicenseNumber=" + getLicenseNumber() + ", ShiftType=" + shiftType +
        ", Details=" + task.getTaskSummary() + "]\n";

}

}

```

//File Name Staff.java

//Accessing Package comhms.src.hms.staff.model

```
package comhms.src.hms.staff.model;
```

```
import comhms.src.hms.auth.model.UserLogin;
```

```
import comhms.src.hms.admin.model.Schedule;
```

```
import java.util.List;
```

```
public class Staff extends UserLogin {
```

```
    private String employeeID; // E.g., EMP0001
```

```
    private String specialization; // E.g., "Cardiology", "General Nursing"
```

```
    private String department; // E.g., "Medical", "Surgical"
```

```
    // Constructor
```

```
    public Staff(int userID, String username, String password, String subRole, String name, int age,
```

```
        String employeeID, String specialization, String department) {
```

```
        super(userID, username, password, "Staff", subRole, name, age, department, department);
```

```
        this.employeeID = employeeID;
```

```
        this.specialization = specialization;
```

```
        this.department = department;
```

```
    }
```

```
    // Overloaded Constructor
```

```

    public Staff(int userID, String username, String password, String subRole, String name, int
age,

        String employeeID, String specialization, String department, String securityQuestion,

        String securityAnswer) {

        super(userID, username, password, "Staff", subRole, name, age, securityQuestion,
securityAnswer);

        this.employeeID = employeeID;

        this.specialization = specialization;

        this.department = department;

    }

```

// Inner Class for Staff Profile

```

private class StaffProfile {

    private String title;

    private String contactInfo;

    StaffProfile(String title, String contactInfo) {

        this.title = title;

        this.contactInfo = contactInfo;

    }

    String getProfileSummary() {

        return "Title: " + title + ", Contact: " + contactInfo;

    }

}

```


// Method with Object Reference

```
public String viewAssignedSchedules(List<Schedule> schedules) throws
StaffAccessException {

    if (schedules == null) {

        throw new StaffAccessException("Schedule list cannot be null.");

    }

    StringBuilder result = new StringBuilder("Assigned Schedules for " + getUsername() +
":\n");

    boolean hasSchedules = false;

    for (Schedule schedule : schedules) {

        if (schedule.getStaffID() == getUserID()) {

            result.append(schedule.toString()).append("\n");

            hasSchedules = true;

        }

    }

    if (!hasSchedules) {

        result.append("No schedules assigned.");

    }

    return result.toString();

}
```

// Method without Object Reference (Static)

```
public static String getStaffRoleDescription() {

    return "Staff: Provides medical or support services, such as doctors or nurses.";

}
```

// Overloaded Method: Update Profile (Specialization)

```
public String updateProfile(String newSpecialization) throws StaffAccessException {  
    if (newSpecialization == null || newSpecialization.trim().isEmpty()) {  
        throw new StaffAccessException("Specialization cannot be empty.");  
    }  
    this.specialization = newSpecialization;  
    StaffProfile profile = new StaffProfile(getSubRole(), "N/A");  
    return "Profile updated: " + profile.getProfileSummary() + ", Specialization: " +  
newSpecialization;  
}
```

// Overloaded Method: Update Profile (Specialization and Department)

```
public String updateProfile(String newSpecialization, String newDepartment) throws  
StaffAccessException {  
    if (newSpecialization == null || newSpecialization.trim().isEmpty() ||  
        newDepartment == null || newDepartment.trim().isEmpty()) {  
        throw new StaffAccessException("Specialization and department cannot be empty.");  
    }  
    this.specialization = newSpecialization;  
    this.department = newDepartment;  
    StaffProfile profile = new StaffProfile(getSubRole(), "N/A");  
    return "Profile updated: " + profile.getProfileSummary() + ", Specialization: " +  
newSpecialization +  
        ", Department: " + newDepartment;  
}
```

// User Defined Exception

```
public static class StaffAccessException extends Exception {  
    public StaffAccessException(String message) {  
        super(message);  
    }  
}
```

// Getters and Setters

```
public String getEmployeeID() { return employeeID; }  
public String getSpecialization() { return specialization; }  
public String getDepartment() { return department; }  
public void setEmployeeID(String employeeID) { this.employeeID = employeeID; }  
public void setSpecialization(String specialization) { this.specialization = specialization; }  
public void setDepartment(String department) { this.department = department; }
```

// Override toString

@Override

```
public String toString() {  
    StaffProfile profile = new StaffProfile(getSubRole(), "N/A");  
    return "Staff [Username=" + getUsername() + ", EmployeeID=" + employeeID +  
        ", Specialization=" + specialization + ", Department=" + department +  
        ", Profile=" + profile.getProfileSummary() + "];"  
}  
}
```

//File Name StaffView.java

//Accessing Package comhms.src.hms.staff.view

```
package comhms.src.hms.staff.view;
```

```
import comhms.src.hms.staff.model.Staff;
```

```
import comhms.src.hms.staff.model.Doctor;
```

```
import comhms.src.hms.staff.model.Nurse;
```

```
import comhms.src.hms.staff.model.ClinicalStaff;
```

```
import comhms.src.hms.auth.controller.UserLoginService;
```

```
import java.util.*;
```

```
public class StaffView {
```

```
    private Scanner scanner;
```

```
    private Staff staff;
```

```
    private UserLoginService loginService; // For authentication and data access
```

```
    private StaffService staffService; // Placeholder for backend logic
```

```
    // Constructor
```

```
    public StaffView(Staff staff, UserLoginService loginService) {
```

```
        this.scanner = new Scanner(System.in);
```

```
        this.staff = staff;
```

```
        this.loginService = loginService;
```

```
        this.staffService = new StaffService(); // Placeholder
```

```
}
```

```
// Overloaded Constructor
```

```
public StaffView(Staff staff, UserLoginService loginService, StaffService staffService) {
```

```
    this.scanner = new Scanner(System.in);
```

```
    this.staff = staff;
```

```
    this.loginService = loginService;
```

```
    this.staffService = staffService;
```

```
}
```

```
// Inner Class for Menu Options
```

```
private class MenuOption {
```

```
    private String description;
```

```
    private Runnable action;
```

```
    MenuOption(String description, Runnable action) {
```

```
        this.description = description;
```

```
        this.action = action;
```

```
    }
```

```
    void display(int index) {
```

```
        System.out.println(index + ". " + description);
```

```
    }
```

```
    void execute() {
```

```
        action.run();
    }
}
```

// Method with Object Reference

```
public void displayStaffMenu(Staff staff) {
    this.staff = staff; // Update staff reference
    MenuOption[] options;
    if (staff instanceof Doctor) {
        options = new MenuOption[] {
            new MenuOption("View Assigned Schedules", this::handleViewSchedules),
            new MenuOption("Perform Clinical Task (Prescribe)",
this::handlePerformClinicalTask),
            new MenuOption("Generate Consultation Report", this::handleGenerateReport),
            new MenuOption("Update Profile", this::handleUpdateProfile),
            new MenuOption("Logout", () -> System.out.println("Logging out..."))
        };
    } else if (staff instanceof Nurse) {
        options = new MenuOption[] {
            new MenuOption("View Assigned Schedules", this::handleViewSchedules),
            new MenuOption("Perform Clinical Task (Vitals/Medication)",
this::handlePerformClinicalTask),
            new MenuOption("Generate Shift Report", this::handleGenerateReport),
            new MenuOption("Update Profile", this::handleUpdateProfile),
            new MenuOption("Logout", () -> System.out.println("Logging out..."))
        };
    }
}
```

```

    } else {
        options = new MenuOption[] {
            new MenuOption("View Assigned Schedules", this::handleViewSchedules),
            new MenuOption("Update Profile", this::handleUpdateProfile),
            new MenuOption("Logout", () -> System.out.println("Logging out..."))
        };
    }

    while (true) {
        System.out.println("\n=== Staff Dashboard ===");
        System.out.println("Welcome, " + staff.getUsername() + " (" + staff.getSubRole() + ")");
        for (int i = 0; i < options.length; i++) {
            options[i].display(i + 1);
        }

        int choice = getValidChoice(options.length);
        if (choice == options.length) { // Logout
            options[choice - 1].execute();
            break;
        }
        options[choice - 1].execute();
    }
}

```

// Method without Object Reference (Static)

```
public static String getWelcomeMessage() {  
    return "Welcome to the Staff Dashboard!";  
}
```

```
// Handle View Schedules
```

```
private void handleViewSchedules() {  
    try {  
        String schedules = staff.viewAssignedSchedules(staffService.getSchedules());  
        System.out.println(schedules);  
    } catch (Staff.StaffAccessException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
}
```

```
// Handle Perform Clinical Task
```

```
private void handlePerformClinicalTask() {  
    if (!(staff instanceof ClinicalStaff)) {  
        System.out.println("Error: Only clinical staff can perform clinical tasks.");  
        return;  
    }  
    try {  
        System.out.print("Enter Patient ID (e.g., PAT001): ");  
        String patientID = scanner.nextLine();  
        System.out.print("Enter Task Details (e.g., Aspirin 100mg daily): ");  
        String taskDetails = scanner.nextLine();  
    }  
}
```



```

        String result = ((ClinicalStaff) staff).performClinicalTask(patientID, taskDetails);

        System.out.println(result);
    } catch (Staff.StaffAccessException e) {

        System.out.println("Error: " + e.getMessage());
    }
}

// Handle Generate Report
private void handleGenerateReport() {
    try {
        if (staff instanceof Doctor) {

            System.out.print("Enter Patient ID (e.g., PAT001): ");

            String patientID = scanner.nextLine();

            String report = ((Doctor)
staff).generateConsultationReport(staffService.getSchedules(), patientID);

            System.out.println(report);
        } else if (staff instanceof Nurse) {

            System.out.print("Enter Shift Date (e.g., 2025-05-04): ");

            String shiftDate = scanner.nextLine();

            String report = ((Nurse) staff).generateShiftReport(staffService.getSchedules(),
shiftDate);

            System.out.println(report);
        } else {

            System.out.println("Error: Report generation not available for this role.");
        }
    } catch (Staff.StaffAccessException e) {

```

```

        System.out.println("Error: " + e.getMessage());
    }
}

// Handle Update Profile
private void handleUpdateProfile() {
    System.out.println("\n=== Update Profile ===");
    System.out.println("1. Update Specialization");
    System.out.println("2. Update Department");
    if (staff instanceof ClinicalStaff) {
        System.out.println("3. Update License Number");
    }
    if (staff instanceof Doctor) {
        System.out.println("4. Update Consultation Fee");
    } else if (staff instanceof Nurse) {
        System.out.println("4. Update Shift Type");
    }
    System.out.println("5. Back");
    int choice = getValidChoice(staff instanceof ClinicalStaff ? 5 : 3);

    if (choice == 5 || choice == 3 && !(staff instanceof ClinicalStaff)) return;

    try {
        if (choice == 1) {
            System.out.print("Enter New Specialization: ");

```

```

String newSpecialization = scanner.nextLine();

String result = staff.updateProfile(newSpecialization);

System.out.println(result);

} else if (choice == 2) {

    System.out.print("Enter New Department: ");

    String newDepartment = scanner.nextLine();

    String result = staff.updateProfile(staff.getSpecialization(), newDepartment);

    System.out.println(result);

} else if (choice == 3 && staff instanceof ClinicalStaff) {

    System.out.print("Enter New License Number: ");

    String newLicense = scanner.nextLine();

    String result = ((ClinicalStaff) staff).updateCredentials(newLicense);

    System.out.println(result);

} else if (choice == 4 && staff instanceof Doctor) {

    System.out.print("Enter New Consultation Fee: ");

    double newFee = Double.parseDouble(scanner.nextLine());

    System.out.print("Enter Reason (or press Enter to skip): ");

    String reason = scanner.nextLine();

    String result = reason.isEmpty() ? ((Doctor) staff).updateConsultationFee(newFee) :

        ((Doctor) staff).updateConsultationFee(newFee, reason);

    System.out.println(result);

} else if (choice == 4 && staff instanceof Nurse) {

    System.out.print("Enter New Shift Type (e.g., Day/Night): ");

    String newShift = scanner.nextLine();

    System.out.print("Enter Reason (or press Enter to skip): ");

```

```

        String reason = scanner.nextLine();

        String result = reason.isEmpty() ? ((Nurse) staff).updateShiftType(newShift) :
            ((Nurse) staff).updateShiftType(newShift, reason);

        System.out.println(result);
    }
} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());

}
}

```

// Helper Method

```

private int getValidChoice(int max) {
    while (true) {
        try {
            System.out.print("Enter choice (1-" + max + "): ");

            int choice = Integer.parseInt(scanner.nextLine());

            if (choice >= 1 && choice <= max) {
                return choice;
            }

            System.out.println("Invalid choice! Try again.");
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number!");
        }
    }
}
}

```

```
}
```

```
// Placeholder StaffService
```

```
class StaffService {
```

```
    public List<comhms.src.hms.admin.model.Schedule> getSchedules() {
```

```
        try {
```

```
            return Arrays.asList(
```

```
                new comhms.src.hms.admin.model.Schedule(1, 2, "2025-05-04 09:00", "2025-05-04  
10:00", "Booked", "Room 101"),
```

```
                new comhms.src.hms.admin.model.Schedule(2, 3, "2025-05-04 08:00", "2025-05-04  
16:00", "Booked", "Ward A")
```

```
            );
```

```
        } catch (comhms.src.hms.admin.model.Schedule.InvalidScheduleException e) {
```

```
            return new ArrayList<>();
```

```
        }
```

```
    }
```

```
}
```

//File Name Main.java

//Accessing Package comhms.src.hms

```
package comhms.src.hms;

import comhms.src.hms.auth.view.LoginView;
import comhms.src.hms.auth.controller.UserLoginService;
import comhms.src.hms.auth.util.CaptchaValidator;
import comhms.src.hms.auth.util.DashboardRouter;
import comhms.src.hms.auth.model.UserLogin;
import comhms.src.hms.admin.model.Admin;
import comhms.src.hms.reception.model.Receptionist;
import comhms.src.hms.staff.model.Staff;
import comhms.src.hms.patient.model.Patient;
import comhms.src.hms.admin.view.AdminView;
import comhms.src.hms.reception.view.ReceptionView;
import comhms.src.hms.staff.view.StaffView;
import comhms.src.hms.patient.view.PatientView;

public class Main {

    private UserLoginService loginService;

    private CaptchaValidator captchaValidator;

    private LoginView loginView;

    // Constructor
```

```

public Main() {

    DashboardRouter router = new DashboardRouter() {

        @Override

        public void redirect(UserLogin user) {

            System.out.println("Redirecting to " + user.getRole() + " Dashboard...");

            if (user.getRole().equals("Admin")) {

                Admin admin = new Admin(user.getUserID(), user.getUsername(),

                    user.getPassword(), user.getName(), user.getAge(), null, false);

                AdminView adminView = new AdminView(admin, loginService);

                adminView.displayAdminMenu(admin);

            } else if (user.getRole().equals("Receptionist")) {

                Receptionist receptionist = new Receptionist(user.getUserID(), user.getUsername(),

                    user.getPassword(), user.getName(),

                    user.getAge(),

                    "DSK" + String.format("%03d", user.getUserID()));

                ReceptionView receptionView = new ReceptionView(receptionist, loginService);

                receptionView.displayReceptionMenu(receptionist);

            } else if (user.getRole().equals("Staff")) {

                Staff staff = new Staff(user.getUserID(), user.getUsername(), user.getPassword(),

                    user.getName(), null, user.getAge(),

                    "STF" + String.format("%03d", user.getUserID()),

                    "General", null);

                StaffView staffView = new StaffView(staff, loginService);

                staffView.displayStaffMenu(staff);

            } else if (user.getRole().equals("Patient")) {

```

```

        Patient patient = new Patient(user.getUserID(), user.getUsername(),
                                      user.getPassword(), user.getName(),
                                      user.getAge(),
                                      "PAT" + String.format("%03d", user.getUserID()),
                                      "No medical history provided");

        PatientView patientView = new PatientView(patient, loginService);
        patientView.displayPatientMenu(patient);
    }
}

};

this.loginService = new UserLoginService(router);
this.captchaValidator = new CaptchaValidator();
this.loginView = new LoginView(loginService, captchaValidator);
}

// Overloaded Constructor

public Main(UserLoginService loginService, CaptchaValidator captchaValidator,
            LoginView loginView) {
    this.loginService = loginService;
    this.captchaValidator = captchaValidator;
    this.loginView = loginView;
}

// Inner Class for System Info

private class SystemInfo {

```



```

private String version;

private String description;

SystemInfo(String version, String description) {
    this.version = version;
    this.description = description;
}

String getSystemSummary() {
    return "HMS Version: " + version + ", Description: " + description;
}
}

// Method with Object Reference
public void startSystem(UserLoginService loginService) {
    SystemInfo info = new SystemInfo("1.0.0", "Hospital Management System");
    System.out.println(info.getSystemSummary());
    this.loginService = loginService;
    loginView.displayLoginMenu(loginService);
}

// Method without Object Reference (Static)
public static String getSystemWelcomeMessage() {
    return "Welcome to the Hospital Management System!";
}
}

```

// Main Method

```
public static void main(String[] args) {  
    try {  
        Main hms = new Main();  
        hms.startSystem(new UserLoginService());  
    } catch (Exception e) {  
        System.out.println("System Error: " + e.getMessage());  
    }  
}
```

// Getter for Testing

```
public LoginView getLoginView() {  
    return loginView;  
}  
}
```

//File Name Person.java

```
public abstract class Person implements Detailable {  
    protected String id;  
    protected String name;  
    protected String contact;
```

```
public Person(String id, String name, String contact) {  
    this.id = id;  
    this.name = name;  
    this.contact = contact;  
}  
  
public String getId() {  
    return id;  
}  
  
public String getName() {  
    return name;  
}  
}
```

//File Name HospitalException.java

```
class HospitalException extends Exception {  
    public HospitalException(String message) {  
        super(message);  
    }  
}
```

Sql code:

-- Schema for Hospital Management System

Create database hospital_management_system;

USE hospital_management_system;

-- Table for Users (to store user credentials and roles)

CREATE TABLE Users (

 user_id INTEGER PRIMARY KEY AUTO_INCREMENT,

 username VARCHAR(50) NOT NULL UNIQUE,

 password VARCHAR(255) NOT NULL,

 name VARCHAR(100) NOT NULL,

 age INTEGER CHECK (age >= 0),

 role ENUM('ADMIN', 'DOCTOR', 'RECEPTIONIST', 'STAFF', 'PATIENT') NOT NULL

);

CREATE TABLE Patients (

 patient_id VARCHAR(10) PRIMARY KEY,

 user_id INTEGER NOT NULL UNIQUE,

 gender VARCHAR(20) NOT NULL,

 contact VARCHAR(10) NOT NULL CHECK (LENGTH(contact) = 10 AND contact REGEXP '^[0-9]+\$'),

 medical_history TEXT,

 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE

);

CREATE TABLE Doctors (

 doctor_id VARCHAR(10) PRIMARY KEY,

 user_id INTEGER NOT NULL UNIQUE,

 specialization VARCHAR(100) NOT NULL,

 contact VARCHAR(10) NOT NULL CHECK (LENGTH(contact) = 10 AND contact
REGEXP '^[0-9]+\$'),

 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE

);

CREATE TABLE Receptionists (

 receptionist_id VARCHAR(10) PRIMARY KEY,

 user_id INTEGER NOT NULL UNIQUE,

 desk_id VARCHAR(10) NOT NULL UNIQUE,

 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE

);

CREATE TABLE Staff (

 staff_id VARCHAR(10) PRIMARY KEY,

 user_id INTEGER NOT NULL UNIQUE,

 department VARCHAR(100) NOT NULL,

 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE

);

CREATE TABLE Appointments (

```
appointment_id VARCHAR(10) PRIMARY KEY,  
patient_id VARCHAR(10) NOT NULL,  
doctor_id VARCHAR(10) NOT NULL,  
time_slot VARCHAR(50) NOT NULL,  
is_emergency BOOLEAN NOT NULL DEFAULT FALSE,  
follow_up_due_date VARCHAR(50),  
follow_up_notes TEXT,  
FOREIGN KEY (patient_id) REFERENCES Patients(patient_id) ON DELETE CASCADE,  
FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE BillItems (  
    bill_item_id INTEGER PRIMARY KEY AUTO_INCREMENT,  
    appointment_id VARCHAR(10) NOT NULL,  
    description TEXT NOT NULL,  
    cost DECIMAL(10, 2) NOT NULL CHECK (cost >= 0),  
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id) ON  
DELETE CASCADE  
);
```

```
CREATE TABLE AuditLogs (  
    log_id INTEGER PRIMARY KEY AUTO_INCREMENT,  
    user_id INTEGER,  
    log_message TEXT NOT NULL,  
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE SET NULL
```

);

```
INSERT INTO Users (username, password, name, age, role)
VALUES ('admin', 'admin123', 'Administrator', NULL, 'ADMIN');
```

-- Simplified SELECT Queries for Hospital Management System

-- 1. Retrieve all users (for authentication or user management)

```
SELECT * FROM Users ;
```

-- 2. Retrieve patient details by ID

```
SELECT * FROM Patients p JOIN Users u ON p.user_id = u.user_id;
```

-- 3. Retrieve doctor details by ID

```
SELECT * FROM Doctors d JOIN Users u ON d.user_id = u.user_id ;
```

-- 4. Retrieve receptionist details by ID

```
SELECT * FROM Receptionists r JOIN Users u ON r.user_id = u.user_id ;
```

-- 5. Retrieve staff details by ID

```
SELECT * FROM Staff s JOIN Users u ON s.user_id = u.user_id ;
```

-- 6. Retrieve all appointments

```
SELECT * FROM Appointments ;
```

-- 7. Retrieve all bill items for an appointment

```
SELECT * FROM BillItems ;
```

-- 8. Retrieve all audit logs

```
SELECT * FROM AuditLogs ORDER BY timestamp DESC;
```

-- 9. Retrieve users by role

```
SELECT * FROM Users
```

GUI Code:

//File Name HospitalSystemGUI.java

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import javax.swing.*;
```

```
import java.util.ArrayList;
```

```
import java.util.HashMap;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.stream.Collectors;
```



```
// Custom exception for hospital-specific errors
```

```
class HospitalException extends Exception {  
    public HospitalException(String message) {  
        super(message);  
    }  
}
```

```
// Enum for user roles
```

```
enum UserRole {  
    ADMIN, DOCTOR, RECEPTIONIST;  
  
    public static UserRole fromString(String name) {  
        if (name == null || name.trim().isEmpty()) {  
            throw new IllegalArgumentException("Role cannot be null or empty");  
        }  
        try {  
            return UserRole.valueOf(name.toUpperCase());  
        } catch (IllegalArgumentException e) {  
            throw new IllegalArgumentException("Invalid role: " + name);  
        }  
    }  
}
```

```
// Interface for entities with details
```

```
interface Detailable {
```

```
String getDetails();  
}  
  
// Base class for Person entities  
abstract class Person implements Detailable {  
    protected String id;  
    protected String name;  
    protected String contact;  
  
    public Person(String id, String name, String contact) {  
        this.id = id;  
        this.name = name;  
        this.contact = contact;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
}  
  
// Patient class
```

```
class Patient extends Person {  
    private int age;  
    private String gender;  
  
    public Patient(String id, String name, int age, String gender, String contact) {  
        super(id, name, contact);  
        this.age = age;  
        this.gender = gender;  
    }  
  
    @Override  
    public String getDetails() {  
        return String.format("Patient ID: %s, Name: %s, Age: %d, Gender: %s, Contact: %s",  
            id, name, age, gender, contact);  
    }  
}  
  
// Doctor class  
class Doctor extends Person {  
    private String specialization;  
  
    public Doctor(String id, String name, String specialization, String contact) {  
        super(id, name, contact);  
        this.specialization = specialization;  
    }  
}
```

@Override

```
public String getDetails() {  
    return String.format("Doctor ID: %s, Name: %s, Specialization: %s, Contact: %s",  
        id, name, specialization, contact);  
}  
}
```

// Appointment class

```
class Appointment {  
    private String id;  
    private String patientId;  
    private String doctorId;  
    private String timeSlot;  
    private boolean isEmergency;  
    private List<BillItem> billItems;  
    private String followUpDueDate;  
    private String followUpNotes;  
  
    public Appointment(String id, String patientId, String doctorId, String timeSlot, boolean  
isEmergency) {  
        this.id = id;  
        this.patientId = patientId;  
        this.doctorId = doctorId;  
        this.timeSlot = timeSlot;  
        this.isEmergency = isEmergency;  
    }  
}
```

```

        this.billItems = new ArrayList<>();
    }

    public String getId() {
        return id;
    }

    public void addBillItem(BillItem item) {
        billItems.add(item);
    }

    public double getTotalBill() {
        return billItems.stream().mapToDouble(BillItem::getCost).sum();
    }

    public void setFollowUp(String dueDate, String notes) {
        this.followUpDueDate = dueDate;
        this.followUpNotes = notes;
    }
}

// BillItem class
class BillItem {
    private String description;
    private double cost;

```

```

public BillItem(String description, double cost) {
    this.description = description;
    this.cost = cost;
}

public double getCost() {
    return cost;
}
}

class HospitalSystem {
    private static HospitalSystem hms = new HospitalSystem();
    private Map<String, Patient> patients = new HashMap<>();
    private Map<String, Doctor> doctors = new HashMap<>();
    private Map<String, Appointment> appointments = new HashMap<>();
    private List<String> auditLogs = new ArrayList<>();
    private Map<String, String> userCredentials = new HashMap<>();
    private int patientCounter = 1;
    private int doctorCounter = 1;
    private int appointmentCounter = 1;

    private HospitalSystem() {
        userCredentials.put("admin:ADMIN", "admin123");
    }
}

```

```
public static HospitalSystem getInstance() {  
    return hms;  
}
```

```
public boolean authenticate(String userId, String password, UserRole role) {  
    String key = userId + ":" + role.name();  
    return userCredentials.containsKey(key) && userCredentials.get(key).equals(password);  
}
```

```
public String addPatient(String name, int age, String gender, String contact) throws  
HospitalException {  
    if (contact.length() != 10 || !contact.matches("\\d+")) {  
        throw new HospitalException("Invalid contact number");  
    }  
    String id = "P" + patientCounter++;  
    Patient patient = new Patient(id, name, age, gender, contact);  
    patients.put(id, patient);  
    auditLogs.add(String.format("User %s added patient %s",  
HospitalSystemGUI.loggedInUserId, id));  
    userCredentials.put(id + ":PATIENT", "pass123");  
    return id;  
}
```

```
public String addDoctor(String name, String specialization, String contact) throws  
HospitalException {
```

```

        if (contact.length() != 10 || !contact.matches("\\d+")) {
            throw new HospitalException("Invalid contact number");
        }

        String id = "D" + doctorCounter++;

        Doctor doctor = new Doctor(id, name, specialization, contact);

        doctors.put(id, doctor);

        auditLogs.add(String.format("User %s added doctor %s",
HospitalSystemGUI.loggedInUserId, id));

        userCredentials.put(id + ":DOCTOR", "doc123");

        return id;
    }

    public String bookAppointment(String patientId, String doctorId, String slot, boolean
isEmergency) throws HospitalException {

        if (!patients.containsKey(patientId)) {

            throw new HospitalException("Invalid patient ID");

        }

        if (!doctors.containsKey(doctorId)) {

            throw new HospitalException("Invalid doctor ID");

        }

        String id = "A" + appointmentCounter++;

        Appointment appointment = new Appointment(id, patientId, doctorId, slot, isEmergency);

        appointments.put(id, appointment);

        auditLogs.add(String.format("User %s booked appointment %s",
HospitalSystemGUI.loggedInUserId, id));

        return id;
    }

```



```
}
```

```
public void addFollowUp(String pId, String appId, String dueDate, String notes) throws  
HospitalException {
```

```
    if (!patients.containsKey(pId)) {
```

```
        throw new HospitalException("Invalid patient ID");
```

```
    }
```

```
    if (!appointments.containsKey(appId)) {
```

```
        throw new HospitalException("Invalid appointment ID");
```

```
    }
```

```
    Appointment appointment = appointments.get(appId);
```

```
    appointment.setFollowUp(dueDate, notes);
```

```
    auditLogs.add(String.format("User %s added follow-up for appointment %s",  
HospitalSystemGUI.loggedInUserId, appId));
```

```
}
```

```
public void addBillItem(String appointmentId, String item, double cost) throws  
HospitalException {
```

```
    if (!appointments.containsKey(appointmentId)) {
```

```
        throw new HospitalException("Invalid appointment ID");
```

```
    }
```

```
    Appointment appointment = appointments.get(appointmentId);
```

```
    appointment.addBillItem(new BillItem(item, cost));
```

```
    auditLogs.add(String.format("User %s added bill item to appointment %s",  
HospitalSystemGUI.loggedInUserId, appointmentId));
```

```
}
```

```

public double generateBill(String appointmentId) throws HospitalException {
    if (!appointments.containsKey(appointmentId)) {
        throw new HospitalException("Invalid appointment ID");
    }

    Appointment appointment = appointments.get(appointmentId);

    auditLogs.add(String.format("User %s generated bill for appointment %s",
HospitalSystemGUI.loggedInUserId, appointmentId));

    return appointment.getTotalBill();
}

```

```

public Person findById(String id) {
    if (patients.containsKey(id)) return patients.get(id);
    if (doctors.containsKey(id)) return doctors.get(id);
    return null;
}

```

```

public List<Person> findByName(String name) {
    List<Person> results = new ArrayList<>();
    results.addAll(patients.values().stream()
        .filter(p -> p.getName().toLowerCase().contains(name.toLowerCase()))
        .collect(Collectors.toList()));
    results.addAll(doctors.values().stream()
        .filter(d -> d.getName().toLowerCase().contains(name.toLowerCase()))
        .collect(Collectors.toList()));
    return results;
}

```

```
public void viewAuditLogs(String userId) {  
    auditLogs.forEach(System.out::println);  
}  
}
```

```
public class HospitalSystemGUI {  
    private HospitalSystem hms;  
    private JFrame frame;  
    private JPanel mainPanel;  
    private JTextArea outputArea;  
    static String loggedInUserId;  
    static UserRole loggedInRole;  
  
    public HospitalSystemGUI() {  
        hms = HospitalSystem.getInstance();  
        frame = new JFrame("Hospital Management System");  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setSize(600, 400);  
        frame.setLayout(new BorderLayout());  
  
        // Login panel  
        showLoginPanel();  
  
        frame.setVisible(true);  
    }  
}
```

```
}
```

```
private void showLoginPanel() {  
    JPanel loginPanel = new JPanel(new GridLayout(4, 2, 10, 10));  
    JTextField userIdField = new JTextField(15);  
    JPasswordField passwordField = new JPasswordField(15);  
    JTextField roleField = new JTextField(15);  
    JButton loginButton = new JButton("Login");  
  
    loginPanel.add(new JLabel("User ID:"));  
    loginPanel.add(userIdField);  
    loginPanel.add(new JLabel("Password:"));  
    loginPanel.add(passwordField);  
    loginPanel.add(new JLabel("Role (ADMIN/DOCTOR/RECEPTIONIST:)"));  
    loginPanel.add(roleField);  
    loginPanel.add(new JLabel());  
    loginPanel.add(loginButton);  
  
    outputArea = new JTextArea(5, 30);  
    outputArea.setEditable(false);  
    JScrollPane scrollPane = new JScrollPane(outputArea);  
  
    frame.getContentPane().removeAll();  
    frame.add(loginPanel, BorderLayout.CENTER);  
    frame.add(scrollPane, BorderLayout.SOUTH);  
}
```

```
frame.revalidate();
```

```
frame.repaint();
```

```
loginButton.addActionListener(e -> {  
    String userId = userIdField.getText();  
    String password = new String(passwordField.getPassword());  
    String role = roleField.getText();  
    try {  
        UserRole userRole = UserRole.fromString(role);  
        if (hms.authenticate(userId, password, userRole)) {  
            loggedInUserId = userId;  
            loggedInRole = userRole;  
            outputArea.append("Login successful!\n");  
            showMainPanel();  
        } else {  
            outputArea.append("Authentication failed!\n");  
        }  
    } catch (IllegalArgumentException ex) {  
        outputArea.append("Invalid role!\n");  
    }  
});  
}
```

```
private void showMainPanel() {  
    mainPanel = new JPanel(new GridLayout(9, 1, 10, 10));
```

```
JButton addPatientButton = new JButton("Add Patient");
JButton addDoctorButton = new JButton("Add Doctor");
JButton bookAppointmentButton = new JButton("Book Appointment");
JButton addFollowUpButton = new JButton("Add Follow Up");
JButton addBillItemButton = new JButton("Add Bill Item");
JButton generateBillButton = new JButton("Generate Bill");
JButton searchByIdButton = new JButton("Search by ID");
JButton searchByNameButton = new JButton("Search by Name");
JButton enterMultiplePatientsButton = new JButton("Enter Multiple Patients");
JButton enterMultipleDoctorsButton = new JButton("Enter Multiple Doctors");
JButton logoutButton = new JButton("Logout");
JButton exitButton = new JButton("Exit");
```

```
mainPanel.add(addPatientButton);
mainPanel.add(addDoctorButton);
mainPanel.add(bookAppointmentButton);
mainPanel.add(addFollowUpButton);
mainPanel.add(addBillItemButton);
mainPanel.add(generateBillButton);
mainPanel.add(searchByIdButton);
mainPanel.add(searchByNameButton);
mainPanel.add(enterMultiplePatientsButton);
mainPanel.add(enterMultipleDoctorsButton);
mainPanel.add(logoutButton);
mainPanel.add(exitButton);
```

```
frame.getContentPane().removeAll();

frame.add(mainPanel, BorderLayout.CENTER);

frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

frame.revalidate();

frame.repaint();


addPatientButton.addActionListener(e -> showAddPatientPanel());
addDoctorButton.addActionListener(e -> showAddDoctorPanel());
bookAppointmentButton.addActionListener(e -> showBookAppointmentPanel());
addFollowUpButton.addActionListener(e -> showAddFollowUpPanel());
addBillItemButton.addActionListener(e -> showAddBillItemPanel());
generateBillButton.addActionListener(e -> showGenerateBillPanel());
searchByIdButton.addActionListener(e -> showSearchByIdPanel());
searchByNameButton.addActionListener(e -> showSearchByNamePanel());
enterMultiplePatientsButton.addActionListener(e -> showEnterMultiplePatientsPanel());
enterMultipleDoctorsButton.addActionListener(e -> showEnterMultipleDoctorsPanel());
logoutButton.addActionListener(e -> {
    loggedInUserId = null;
    loggedInRole = null;
    outputArea.append("Logged out!\n");
    showLoginPanel();
});
exitButton.addActionListener(e -> System.exit(0));
}
```

```
private void showAddPatientPanel() {  
    JPanel patientPanel = new JPanel(new GridLayout(5, 2, 10, 10));  
    JTextField nameField = new JTextField(15);  
    JTextField ageField = new JTextField(15);  
    JTextField genderField = new JTextField(15);  
    JTextField contactField = new JTextField(15);  
    JButton submitButton = new JButton("Submit");  
    JButton backButton = new JButton("Back");  
  
    patientPanel.add(new JLabel("Name:"));  
    patientPanel.add(nameField);  
    patientPanel.add(new JLabel("Age:"));  
    patientPanel.add(ageField);  
    patientPanel.add(new JLabel("Gender:"));  
    patientPanel.add(genderField);  
    patientPanel.add(new JLabel("Contact (10 digits):"));  
    patientPanel.add(contactField);  
    patientPanel.add(submitButton);  
    patientPanel.add(backButton);  
  
    frame.getContentPane().removeAll();  
    frame.add(patientPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
}
```



```
frame.repaint();
```

```
submitButton.addActionListener(e -> {  
    try {  
        String name = nameField.getText();  
        int age = Integer.parseInt(ageField.getText());  
        String gender = genderField.getText();  
        String contact = contactField.getText();  
        String patientId = hms.addPatient(name, age, gender, contact);  
        outputArea.append("Patient added! ID: " + patientId + "\n");  
        nameField.setText("");  
        ageField.setText("");  
        genderField.setText("");  
        contactField.setText("");  
    } catch (NumberFormatException ex) {  
        outputArea.append("Invalid age!\n");  
    } catch (HospitalException ex) {  
        outputArea.append("Error: " + ex.getMessage() + "\n");  
    }  
});
```

```
backButton.addActionListener(e -> {  
    frame.getContentPane().removeAll();  
    frame.add(mainPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
```

```
        frame.revalidate();  
        frame.repaint();  
    });  
}
```

```
private void showAddDoctorPanel() {  
    JPanel doctorPanel = new JPanel(new GridLayout(4, 2, 10, 10));  
    JTextField nameField = new JTextField(15);  
    JTextField specializationField = new JTextField(15);  
    JTextField contactField = new JTextField(15);  
    JButton submitButton = new JButton("Submit");  
    JButton backButton = new JButton("Back");  
  
    if (loggedInRole == UserRole.RECEPTIONIST) {  
        outputArea.append("Permission denied!\n");  
        return;  
    }  
}
```

```
    doctorPanel.add(new JLabel("Name:"));  
    doctorPanel.add(nameField);  
    doctorPanel.add(new JLabel("Specialization:"));  
    doctorPanel.add(specializationField);  
    doctorPanel.add(new JLabel("Contact (10 digits):"));  
    doctorPanel.add(contactField);  
    doctorPanel.add(submitButton);
```

```
doctorPanel.add(backButton);
```

```
frame.getContentPane().removeAll();
```

```
frame.add(doctorPanel, BorderLayout.CENTER);
```

```
frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
```

```
frame.revalidate();
```

```
frame.repaint();
```

```
submitButton.addActionListener(e -> {
```

```
    try {
```

```
        String name = nameField.getText();
```

```
        String specialization = specializationField.getText();
```

```
        String contact = contactField.getText();
```

```
        String doctorId = hms.addDoctor(name, specialization, contact);
```

```
        outputArea.append("Doctor added! ID: " + doctorId + "\n");
```

```
        nameField.setText("");
```

```
        specializationField.setText("");
```

```
        contactField.setText("");
```

```
    } catch (HospitalException ex) {
```

```
        outputArea.append("Error: " + ex.getMessage() + "\n");
```

```
    }
```

```
});
```

```
backButton.addActionListener(e -> {
```

```
    frame.getContentPane().removeAll();
```

```

        frame.add(mainPanel, BorderLayout.CENTER);

        frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

        frame.revalidate();

        frame.repaint();

    });
}

private void showBookAppointmentPanel() {
    JPanel appointmentPanel = new JPanel(new GridLayout(5, 2, 10, 10));

    JTextField patientIdField = new JTextField(15);
    JTextField doctorIdField = new JTextField(15);
    JTextField timeSlotField = new JTextField(15);
    JCheckBox emergencyCheckBox = new JCheckBox("Is Emergency?");
    JButton submitButton = new JButton("Submit");
    JButton backButton = new JButton("Back");

    appointmentPanel.add(new JLabel("Patient ID:"));
    appointmentPanel.add(patientIdField);
    appointmentPanel.add(new JLabel("Doctor ID:"));
    appointmentPanel.add(doctorIdField);
    appointmentPanel.add(new JLabel("Time Slot:"));
    appointmentPanel.add(timeSlotField);
    appointmentPanel.add(new JLabel());
    appointmentPanel.add(emergencyCheckBox);
    appointmentPanel.add(submitButton);

```

```
appointmentPanel.add(backButton);
```

```
frame.getContentPane().removeAll();
```

```
frame.add(appointmentPanel, BorderLayout.CENTER);
```

```
frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
```

```
frame.revalidate();
```

```
frame.repaint();
```

```
submitButton.addActionListener(e -> {
```

```
    try {
```

```
        String patientId = patientIdField.getText();
```

```
        String doctorId = doctorIdField.getText();
```

```
        String timeSlot = timeSlotField.getText();
```

```
        boolean isEmergency = emergencyCheckBox.isSelected();
```

```
        String appointmentId = hms.bookAppointment(patientId, doctorId, timeSlot,  
isEmergency);
```

```
        outputArea.append("Appointment booked! ID: " + appointmentId + "\n");
```

```
        patientIdField.setText("");
```

```
        doctorIdField.setText("");
```

```
        timeSlotField.setText("");
```

```
        emergencyCheckBox.setSelected(false);
```

```
    } catch (HospitalException ex) {
```

```
        outputArea.append("Error: " + ex.getMessage() + "\n");
```

```
    }
```

```
});
```

```
backButton.addActionListener(e -> {  
    frame.getContentPane().removeAll();  
    frame.add(mainPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
    frame.repaint();  
});  
}
```

```
private void showAddFollowUpPanel() {  
    JPanel followUpPanel = new JPanel(new GridLayout(5, 2, 10, 10));  
    JTextField patientIdField = new JTextField(15);  
    JTextField appointmentIdField = new JTextField(15);  
    JTextField dueDateField = new JTextField(15);  
    JTextField notesField = new JTextField(15);  
    JButton submitButton = new JButton("Submit");  
    JButton backButton = new JButton("Back");  
  
    followUpPanel.add(new JLabel("Patient ID:"));  
    followUpPanel.add(patientIdField);  
    followUpPanel.add(new JLabel("Appointment ID:"));  
    followUpPanel.add(appointmentIdField);  
    followUpPanel.add(new JLabel("Due Date:"));  
    followUpPanel.add(dueDateField);  
    followUpPanel.add(new JLabel("Notes:"));
```

```
followUpPanel.add(notesField);

followUpPanel.add(submitButton);

followUpPanel.add(backButton);


frame.getContentPane().removeAll();

frame.add(followUpPanel, BorderLayout.CENTER);

frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

frame.revalidate();

frame.repaint();


submitButton.addActionListener(e -> {
    try {
        String patientId = patientIdField.getText();

        String appointmentId = appointmentIdField.getText();

        String dueDate = dueDateField.getText();

        String notes = notesField.getText();

        hms.addFollowUp(patientId, appointmentId, dueDate, notes);

        outputArea.append("Follow-up added for appointment " + appointmentId + "\n");

        patientIdField.setText("");

        appointmentIdField.setText("");

        dueDateField.setText("");

        notesField.setText("");
    } catch (HospitalException ex) {
        outputArea.append("Error: " + ex.getMessage() + "\n");
    }
}
```

```
});

backButton.addActionListener(e -> {

    frame.getContentPane().removeAll();

    frame.add(mainPanel, BorderLayout.CENTER);

    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

    frame.revalidate();

    frame.repaint();

});

}
```

```
private void showAddBillItemPanel() {

    JPanel billItemPanel = new JPanel(new GridLayout(4, 2, 10, 10));

    JTextField appointmentIdField = new JTextField(15);

    JTextField descriptionField = new JTextField(15);

    JTextField costField = new JTextField(15);

    JButton submitButton = new JButton("Submit");

    JButton backButton = new JButton("Back");


    billItemPanel.add(new JLabel("Appointment ID:"));

    billItemPanel.add(appointmentIdField);

    billItemPanel.add(new JLabel("Description:"));

    billItemPanel.add(descriptionField);

    billItemPanel.add(new JLabel("Cost:"));

    billItemPanel.add(costField);

}
```



```
billItemPanel.add(submitButton);

billItemPanel.add(backButton);


frame.getContentPane().removeAll();

frame.add(billItemPanel, BorderLayout.CENTER);

frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

frame.revalidate();

frame.repaint();


submitButton.addActionListener(e -> {

    try {

        String appointmentId = appointmentIdField.getText();

        String description = descriptionField.getText();

        double cost = Double.parseDouble(costField.getText());

        hms.addBillItem(appointmentId, description, cost);

        outputArea.append("Bill item added to appointment " + appointmentId + "\n");

        appointmentIdField.setText("");

        descriptionField.setText("");

        costField.setText("");

    } catch (NumberFormatException ex) {

        outputArea.append("Invalid cost!\n");

    } catch (HospitalException ex) {

        outputArea.append("Error: " + ex.getMessage() + "\n");

    }

});
```

```
backButton.addActionListener(e -> {  
    frame.getContentPane().removeAll();  
    frame.add(mainPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
    frame.repaint();  
});  
}
```

```
private void showGenerateBillPanel() {  
    JPanel billPanel = new JPanel(new GridLayout(3, 2, 10, 10));  
    JTextField appointmentIdField = new JTextField(15);  
    JButton submitButton = new JButton("Generate");  
    JButton backButton = new JButton("Back");  
  
    billPanel.add(new JLabel("Appointment ID:"));  
    billPanel.add(appointmentIdField);  
    billPanel.add(submitButton);  
    billPanel.add(backButton);  
  
    frame.getContentPane().removeAll();  
    frame.add(billPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
}
```

```

frame.repaint();

submitButton.addActionListener(e -> {
    try {
        String appointmentId = appointmentIdField.getText();
        double total = hms.generateBill(appointmentId);
        outputArea.append("Total bill for appointment " + appointmentId + ": $" + total +
"\n");
        appointmentIdField.setText("");
    } catch (HospitalException ex) {
        outputArea.append("Error: " + ex.getMessage() + "\n");
    }
});

```

```

backButton.addActionListener(e -> {
    frame.getContentPane().removeAll();
    frame.add(mainPanel, BorderLayout.CENTER);
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
    frame.revalidate();
    frame.repaint();
});
}

```

```

private void showSearchByIdPanel() {
    JPanel searchPanel = new JPanel(new GridLayout(3, 2, 10, 10));
    JTextField idField = new JTextField(15);

```

```
JButton submitButton = new JButton("Search");
```

```
JButton backButton = new JButton("Back");
```

```
searchPanel.add(new JLabel("ID (e.g., P1 or D1:"));
```

```
searchPanel.add(idField);
```

```
searchPanel.add(submitButton);
```

```
searchPanel.add(backButton);
```

```
frame.getContentPane().removeAll();
```

```
frame.add(searchPanel, BorderLayout.CENTER);
```

```
frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
```

```
frame.revalidate();
```

```
frame.repaint();
```

```
submitButton.addActionListener(e -> {
```

```
    String id = idField.getText();
```

```
    Person person = hms.findById(id);
```

```
    if (person != null) {
```

```
        outputArea.append(person.getDetails() + "\n");
```

```
    } else {
```

```
        outputArea.append("No person found with ID: " + id + "\n");
```

```
    }
```

```
    idField.setText("");
```

```
});
```

```
backButton.addActionListener(e -> {  
    frame.getContentPane().removeAll();  
    frame.add(mainPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
    frame.repaint();  
});  
}
```

```
private void showSearchByNamePanel() {  
    JPanel searchPanel = new JPanel(new GridLayout(3, 2, 10, 10));  
    JTextField nameField = new JTextField(15);  
    JButton submitButton = new JButton("Search");  
    JButton backButton = new JButton("Back");  
  
    searchPanel.add(new JLabel("Name:"));  
    searchPanel.add(nameField);  
    searchPanel.add(submitButton);  
    searchPanel.add(backButton);  
  
    frame.getContentPane().removeAll();  
    frame.add(searchPanel, BorderLayout.CENTER);  
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);  
    frame.revalidate();  
    frame.repaint();  
}
```

```

submitButton.addActionListener(e -> {
    String name = nameField.getText();
    List<Person> persons = hms.findByName(name);
    if (persons.isEmpty()) {
        outputArea.append("No persons found with name: " + name + "\n");
    } else {
        outputArea.append("Found persons:\n");
        for (Person person : persons) {
            outputArea.append(person.getDetails() + "\n");
        }
    }
    nameField.setText("");
});

```

```

backButton.addActionListener(e -> {
    frame.getContentPane().removeAll();
    frame.add(mainPanel, BorderLayout.CENTER);
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
    frame.revalidate();
    frame.repaint();
});
}

```

```

private void showEnterMultiplePatientsPanel() {

```

```
JPanel patientPanel = new JPanel(new GridLayout(6, 2, 10, 10));

JTextField nameField = new JTextField(15);

JTextField ageField = new JTextField(15);

JTextField genderField = new JTextField(15);

JTextField contactField = new JTextField(15);

JButton addAnotherButton = new JButton("Add Another Patient");

JButton doneButton = new JButton("Done");


patientPanel.add(new JLabel("Name:"));
patientPanel.add(nameField);
patientPanel.add(new JLabel("Age:"));
patientPanel.add(ageField);
patientPanel.add(new JLabel("Gender:"));
patientPanel.add(genderField);
patientPanel.add(new JLabel("Contact (10 digits):"));
patientPanel.add(contactField);
patientPanel.add(addAnotherButton);
patientPanel.add(doneButton);


frame.getContentPane().removeAll();
frame.add(patientPanel, BorderLayout.CENTER);
frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
frame.revalidate();
frame.repaint();
```

```

addAnotherButton.addActionListener(e -> {
    try {
        String name = nameField.getText();
        int age = Integer.parseInt(ageField.getText());
        String gender = genderField.getText();
        String contact = contactField.getText();
        String patientId = hms.addPatient(name, age, gender, contact);
        outputArea.append("Patient added! ID: " + patientId + "\n");
        nameField.setText("");
        ageField.setText("");
        genderField.setText("");
        contactField.setText("");
    } catch (NumberFormatException ex) {
        outputArea.append("Invalid age!\n");
    } catch (HospitalException ex) {
        outputArea.append("Error: " + ex.getMessage() + "\n");
    }
});

```

```

doneButton.addActionListener(e -> {
    frame.getContentPane().removeAll();
    frame.add(mainPanel, BorderLayout.CENTER);
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);
    frame.revalidate();
    frame.repaint();
});

```



```
});  
}
```

```
private void showEnterMultipleDoctorsPanel() {  
    JPanel doctorPanel = new JPanel(new GridLayout(5, 2, 10, 10));  
    JTextField nameField = new JTextField(15);  
    JTextField specializationField = new JTextField(15);  
    JTextField contactField = new JTextField(15);  
    JButton addAnotherButton = new JButton("Add Another Doctor");  
    JButton doneButton = new JButton("Done");  
  
    if (loggedInRole == UserRole.RECEPTIONIST) {  
        outputArea.append("Permission denied!\n");  
        return;  
    }  
  
    doctorPanel.add(new JLabel("Name:"));  
    doctorPanel.add(nameField);  
    doctorPanel.add(new JLabel("Specialization:"));  
    doctorPanel.add(specializationField);  
    doctorPanel.add(new JLabel("Contact (10 digits):"));  
    doctorPanel.add(contactField);  
    doctorPanel.add(addAnotherButton);  
    doctorPanel.add(doneButton);  
}
```

```

frame.getContentPane().removeAll();

frame.add(doctorPanel, BorderLayout.CENTER);

frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

frame.revalidate();

frame.repaint();


addAnotherButton.addActionListener(e -> {

    try {

        String name = nameField.getText();

        String specialization = specializationField.getText();

        String contact = contactField.getText();

        String doctorId = hms.addDoctor(name, specialization, contact);

        outputArea.append("Doctor added! ID: " + doctorId + "\n");

        nameField.setText("");

        specializationField.setText("");

        contactField.setText("");

    } catch (HospitalException ex) {

        outputArea.append("Error: " + ex.getMessage() + "\n");

    }

});


doneButton.addActionListener(e -> {

    frame.getContentPane().removeAll();

    frame.add(mainPanel, BorderLayout.CENTER);

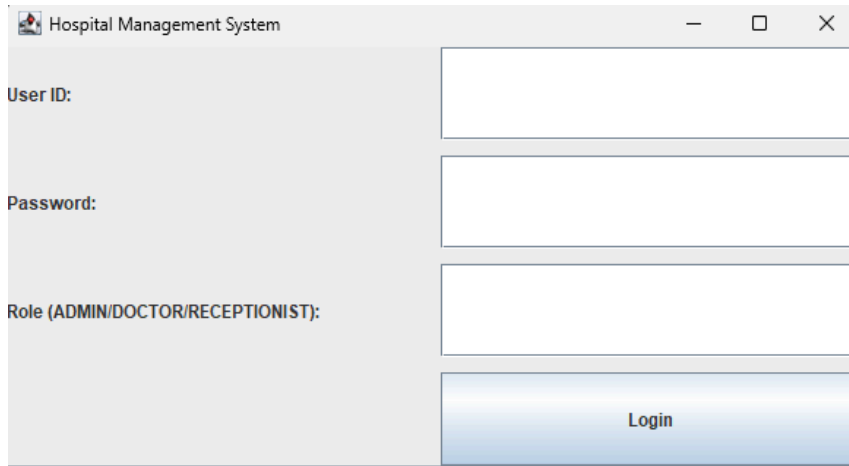
    frame.add(new JScrollPane(outputArea), BorderLayout.SOUTH);

```

```
        frame.revalidate();  
        frame.repaint();  
    });  
}  
  
public static void main(String[] args) {  
    try {  
        SwingUtilities.invokeLater(() -> new HospitalSystemGUI());  
    } catch (Exception e) {  
        System.err.println("Failed to start GUI: " + e.getMessage());  
        e.printStackTrace();  
    }  
}}
```

Output:

Login interface:



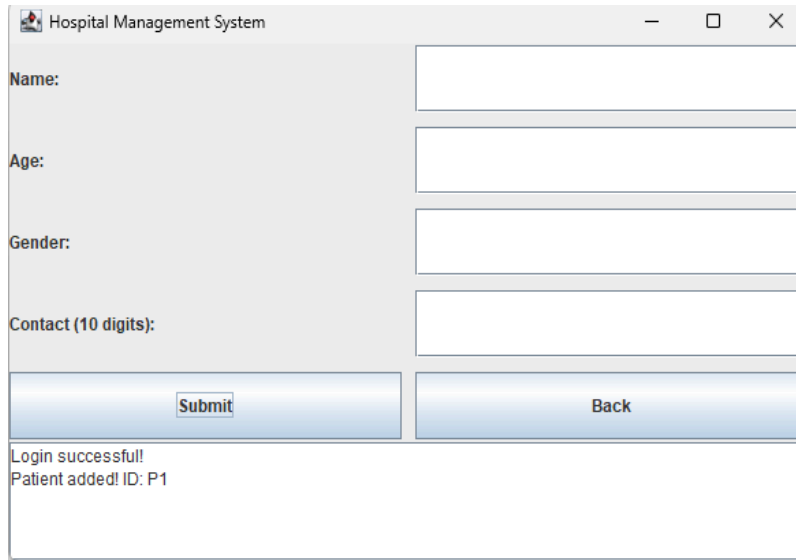
A screenshot of a window titled "Hospital Management System". The window contains three input fields on the left, each with a label: "User ID:", "Password:", and "Role (ADMIN/DOCTOR/RECEPTIONIST):". To the right of these fields is a large blue button with the text "Login".

Options:



A screenshot of a window titled "Hospital Management System". The window displays a grid of 12 buttons arranged in 6 rows and 2 columns. The buttons are: "Add Patient", "Add Doctor", "Book Appointment", "Add Follow Up", "Add Bill Item", "Generate Bill", "Search by ID", "Search by Name", "Enter Multiple Patients", "Enter Multiple Doctors", "Logout", and "Exit". Below the grid is a text area containing the message "Login successfull!".

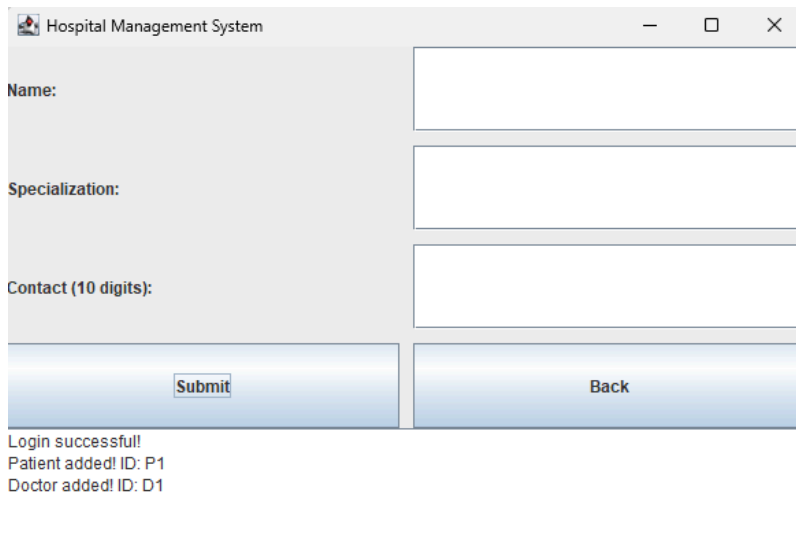
Option 1: Adding patient



The screenshot shows a window titled "Hospital Management System" with a standard Windows-style title bar (minimize, maximize, close buttons). The main content area is divided into two columns. The left column contains labels for "Name:", "Age:", "Gender:", and "Contact (10 digits):". The right column contains four corresponding text input fields. Below the input fields, there are two buttons: "Submit" and "Back". At the bottom of the window, there is a status bar with the text "Login successful!" and "Patient added! ID: P1".

Name:	
Age:	
Gender:	
Contact (10 digits):	
Submit Back	
Login successful! Patient added! ID: P1	

Patient and Doctor Added:



The screenshot shows a window titled "Hospital Management System" with a standard Windows-style title bar (minimize, maximize, close buttons). The main content area is divided into two columns. The left column contains labels for "Name:", "Specialization:", and "Contact (10 digits):". The right column contains three corresponding text input fields. Below the input fields, there are two buttons: "Submit" and "Back". At the bottom of the window, there is a status bar with the text "Login successful!", "Patient added! ID: P1", and "Doctor added! ID: D1".

Name:	
Specialization:	
Contact (10 digits):	
Submit Back	
Login successful! Patient added! ID: P1 Doctor added! ID: D1	

Booking appointment:

The screenshot shows a web application window titled "Hospital Management System". The form contains three input fields for "Patient ID:", "Doctor ID:", and "Time Slot:". Below these fields is a checkbox labeled "Is Emergency?". At the bottom of the form are two buttons: "Submit" and "Back". A status bar at the very bottom of the window displays the following messages: "Login successful!", "Patient added! ID: P1", "Doctor added! ID: D1", and "Appointment booked! ID: A1".

Hospital Management System

Patient ID:

Doctor ID:

Time Slot:

☐ Is Emergency?

Login successful!
Patient added! ID: P1
Doctor added! ID: D1
Appointment booked! ID: A1

Add follow up:

The screenshot shows a web application window titled "Hospital Management System". The form contains four input fields: "Patient ID:", "Appointment ID:", "Due Date:", and "Notes:". Below these fields are two buttons: "Submit" and "Back". A status bar at the bottom of the window displays the following messages: "Patient added! ID: P1", "Doctor added! ID: D1", "Appointment booked! ID: A1", and "Follow-up added for appointment A1".

Hospital Management System

Patient ID:

Appointment ID:

Due Date:

Notes:

Patient added! ID: P1
Doctor added! ID: D1
Appointment booked! ID: A1
Follow-up added for appointment A1

Add Bill Item:

The screenshot shows a window titled "Hospital Management System" with a standard Windows title bar (minimize, maximize, close buttons). The window contains three input fields on the left, each with a label: "Appointment ID:", "Description:", and "Cost:". To the right of each label is a corresponding empty text box. Below these fields are two buttons: "Submit" and "Back". At the bottom of the window is a scrollable log area containing the following text: "Doctor added! ID: D1", "Appointment booked! ID: A1", "Follow-up added for appointment A1", and "Bill item added to appointment A1".

Generate Bill:

The screenshot shows a window titled "Hospital Management System" with a standard Windows title bar. The window contains one input field on the left with the label "Appointment ID:". To the right of the label is an empty text box. Below this field are two buttons: "Generate" and "Back". At the bottom of the window is a scrollable log area containing the following text: "Appointment booked! ID: A1", "Follow-up added for appointment A1", "Bill item added to appointment A1", and "Total bill for appointment A1: \$100.0".

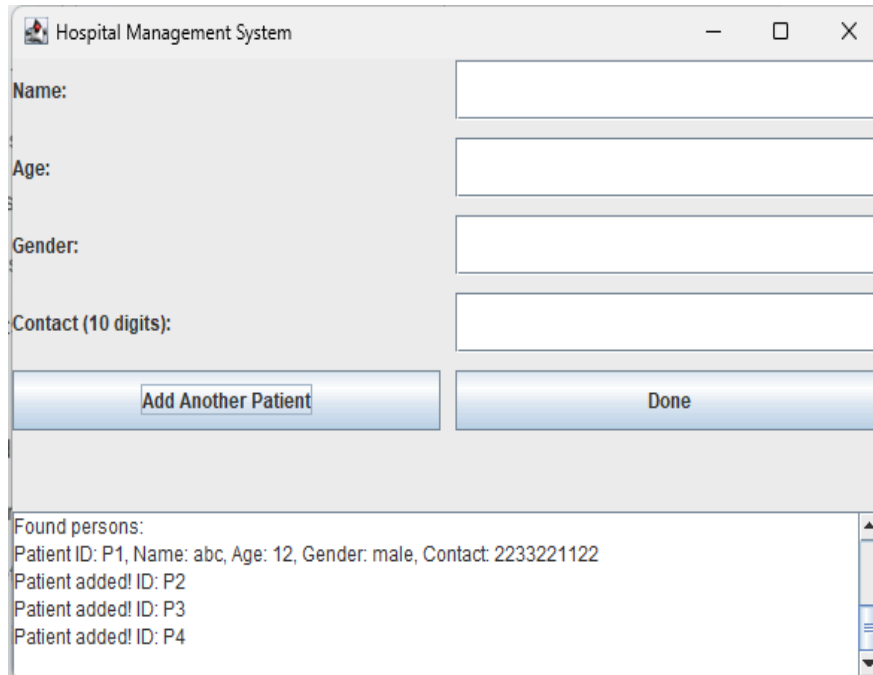
Search By Id:

The screenshot shows a window titled "Hospital Management System" with standard minimize, maximize, and close buttons. The main area contains a label "ID (e.g., P1 or D1):" next to a large empty text input field. Below the input field are two blue buttons with white text: "Search" and "Back". At the bottom of the window, there is a scrollable text area containing the following information: "Follow-up added for appointment A1", "Bill item added to appointment A1", "Total bill for appointment A1: \$100.0", and "Patient ID: P1, Name: abc, Age: 12, Gender: male, Contact: 2233221122".

Search By Name:

The screenshot shows a window titled "Hospital Management System" with standard minimize, maximize, and close buttons. The main area contains a label "Name:" next to a large empty text input field. Below the input field are two blue buttons with white text: "Search" and "Back". At the bottom of the window, there is a scrollable text area containing the following information: "Bill item added to appointment A1", "Total bill for appointment A1: \$100.0", "Patient ID: P1, Name: abc, Age: 12, Gender: male, Contact: 2233221122", "Found persons:", and "Patient ID: P1, Name: abc, Age: 12, Gender: male, Contact: 2233221122".

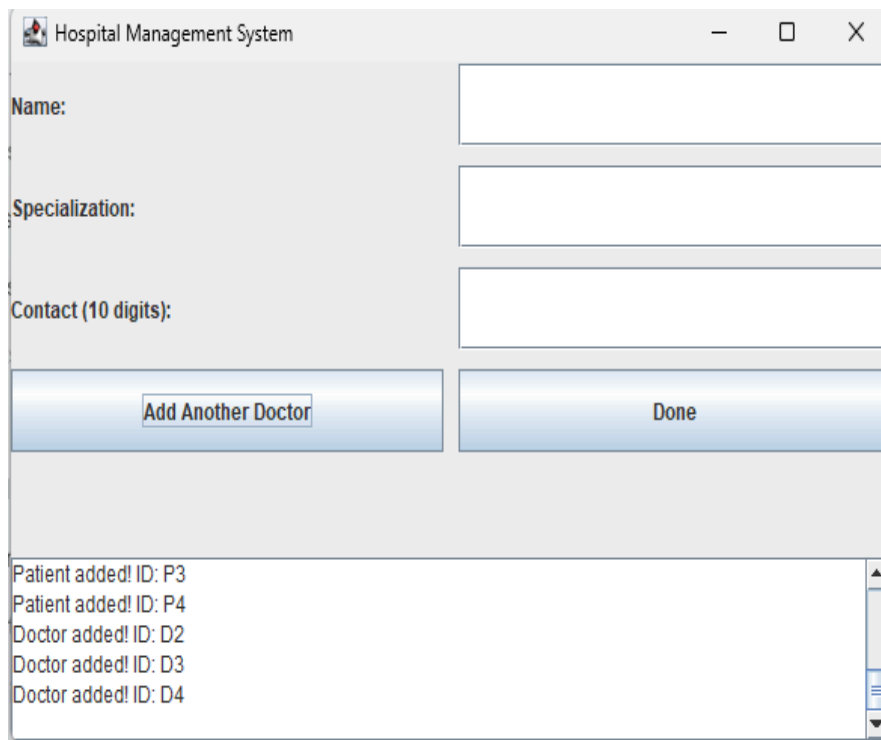
Enter Multiple Patients:



The screenshot shows a window titled "Hospital Management System" with a standard Windows title bar (minimize, maximize, close buttons). The window contains a form for entering patient information. The form has four input fields with labels: "Name:", "Age:", "Gender:", and "Contact (10 digits):". Below these fields are two buttons: "Add Another Patient" and "Done". At the bottom of the window is a scrollable text area containing the following text:

Found persons:
Patient ID: P1, Name: abc, Age: 12, Gender: male, Contact: 2233221122
Patient added! ID: P2
Patient added! ID: P3
Patient added! ID: P4

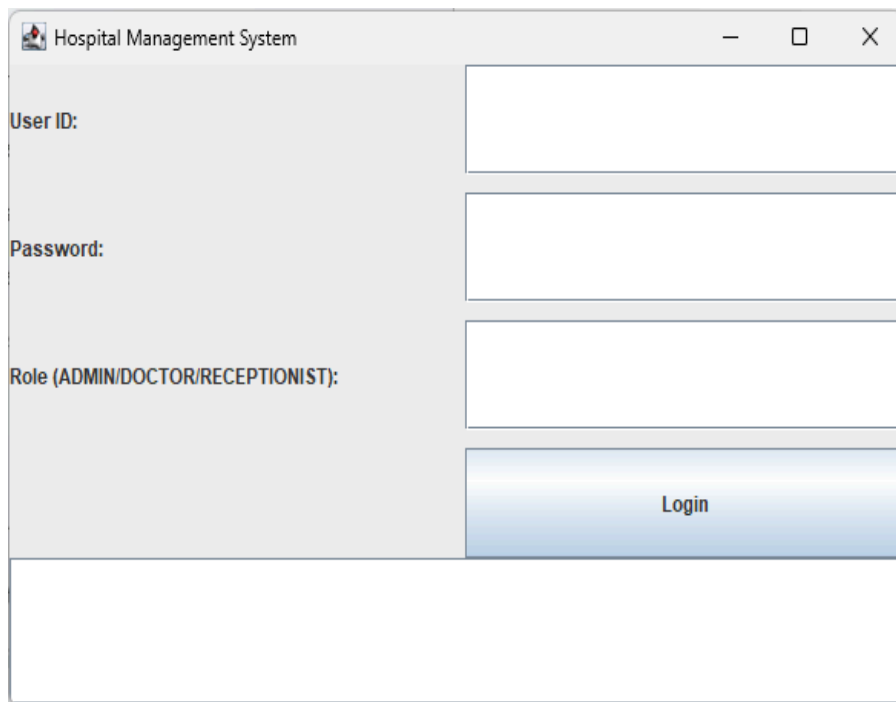
Enter Multiple Doctors:



The screenshot shows a window titled "Hospital Management System" with a standard Windows title bar (minimize, maximize, close buttons). The window contains a form for entering doctor information. The form has three input fields with labels: "Name:", "Specialization:", and "Contact (10 digits):". Below these fields are two buttons: "Add Another Doctor" and "Done". At the bottom of the window is a scrollable text area containing the following text:

Patient added! ID: P3
Patient added! ID: P4
Doctor added! ID: D2
Doctor added! ID: D3
Doctor added! ID: D4

Logout:



A screenshot of a web application window titled "Hospital Management System". The window has a standard title bar with minimize, maximize, and close buttons. The main content area is divided into two columns. The left column is a light gray panel containing three labels: "User ID:", "Password:", and "Role (ADMIN/DOCTOR/RECEPTIONIST):". The right column contains three white input fields corresponding to these labels, followed by a blue "Login" button. Below the input fields and button is a large white rectangular area, likely for a message or error display.

User ID:	
Password:	
Role (ADMIN/DOCTOR/RECEPTIONIST):	
	Login

Testing vs Requirement Mapping:

Aspect	Requirement Mapping (Hospital Management System)	Testing (Hospital Management System)
Focus	Mapping features like patient registration, doctor management, appointment scheduling to functional requirements.	Verifying if modules like patient registration, billing, and reports behave correctly under different conditions.
Goal	Ensure all system requirements (e.g., "register patient", "view doctor list") are implemented in the application.	Ensure the system functions correctly without bugs – e.g., appointments get scheduled properly, invalid data is rejected.
Artifacts Used	Functional Requirement Specs (FR-01: Add patient, FR-02: Schedule appointment), Design Documents.	Test Cases (e.g., TC-01: Register patient), Bug Reports, Test Logs.
Done By	System Analyst, Project Manager, Developer (during design phase).	QA Team, Test Engineers, sometimes developers (during implementation/testing phase).
Outcome	Requirement Traceability Matrix linking each feature to its requirement.	Test Results: e.g., "Patient form validation passed", "Doctor availability fails under load".

AI Contribution:

We have used Chat Gpt to write report faster,accurately and to make the content more relatable.

Below is the brief information about how we used AI in our project.

Area of Work	AI Contribution (ChatGPT)
Requirements & System Design	Helped define and structure system requirements, and explained how requirement mapping works.
Use Case Diagram	Provided PlantUML code and a detailed explanation of the use case diagram
Class Diagram	Explained the structure and generated PlantUML code for the class diagram.

Sequence Diagram	Gave PlantUML code and full explanation of sequence flow between system components
Abstraction (OOP Concept)	Explained abstract classes and interfaces with examples from Hospital Management System
Access Specifiers	Provided detailed explanation with real examples in context.
Testing vs Requirement Mapping	Explained concepts and provided a customized table for the Hospital Management context
Report Writing Support	Helped structure technical descriptions and summary content clearly and professionally.

Future Enhancements:

- **Role-Based Access Control (RBAC):** Assign different access levels for Admins, Doctors, Nurses, and Receptionists.
- **SMS/Email Notifications:** Send appointment reminders, billing info, and test reports to patients automatically.
- **Online Appointment Booking:** Allow patients to book, reschedule, or cancel appointments via a web or mobile app.
- **Electronic Medical Records (EMR):** Digitally store patient history, prescriptions, test results, and diagnosis reports.
- **Billing & Insurance Integration:** Automate billing processes and integrate with insurance companies for claim handling.
- **Data Analytics & Reporting:** Generate reports and insights on hospital operations, patient trends, and staff performance.

Conclusion:

The Hospital Management System is a comprehensive solution designed to streamline the daily operations of a hospital or clinic. It effectively manages critical functions such as patient registration, doctor assignment, appointment scheduling, and billing. By integrating core object-oriented programming concepts like abstraction, inheritance, interfaces, and exception handling, the system ensures modularity, scalability, and ease of maintenance.

This project demonstrates how technology can improve healthcare delivery by reducing manual errors, enhancing record-keeping, and increasing efficiency. With features that support both administrative and medical staff, the system contributes to better patient care and operational transparency.

Although the current version meets essential functional requirements, there is significant scope for future enhancements such as online booking, EMR integration, and cloud-based access. Overall, the project lays a solid foundation for a digital healthcare platform that can be continuously evolved to meet the growing needs of the healthcare sector.